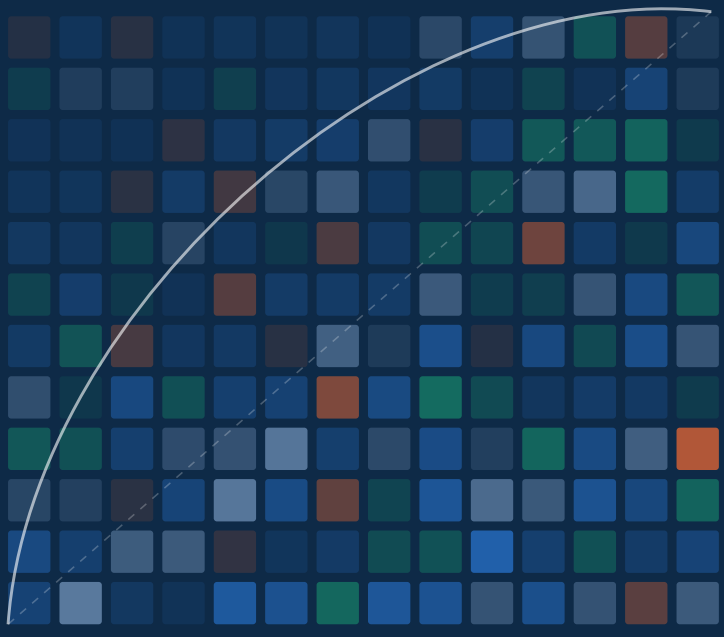
GIÁO TRÌNH · TẬP 1

KỸ THUẬT HỌC MÁY

cho Dữ liệu Bảng

Hướng dẫn thực hành từng bước với Python:
từ dữ liệu thô, cân bằng dữ liệu, chọn đặc trưng,
tối ưu siêu tham số, đến giải thích mô hình
và đánh giá độ tin cậy của dự đoán.



MẤT CÂN BẰNG · CHỌN ĐẶC TRƯNG · SIÊU THAM SỐ
ENSEMBLE · XAI: SHAP, LIME · CALIBRATION · CONFORMAL

LÊ NHẬT TÙNG
Giảng viên – Khoa học Dữ liệu & Trí tuệ nhân tạo

2026

Kỹ thuật Học máy cho Dữ liệu Bảng

Tập 1 – Giáo trình thực hành

Tác giả: Lê Nhật Tùng

Bản thảo, 2026. Mã nguồn, dữ liệu và notebook đi kèm nằm trong các thư mục `code/`, `data/`.
Phông chữ: Be Vietnam Pro, Noto Serif, JetBrains Mono (SIL Open Font License).

Mục lục

Lời nói đầu **viii**

I Nền tảng **1**

1 Bắt đầu từ con số không: dữ liệu, mô hình đầu tiên và cách chấm điểm **2**

1.1 Trước khi bắt đầu 2

1.2 Giai đoạn 1 – Chuẩn bị môi trường 3

1.2.1 Bước 1. Cài Python và tạo môi trường ảo 4

1.2.2 Bước 2. Cài các thư viện và kiểm tra phiên bản 4

1.2.3 Bước 3. Tạo cấu trúc thư mục và đặt dữ liệu vào đúng chỗ 5

1.2.4 Bước 4. Nạp thư viện và thiết lập chung 6

1.3 Giai đoạn 2 – Làm quen với dữ liệu 6

1.3.1 Bước 5. Đọc dữ liệu vào bảng 6

1.3.2 Bước 6. Kiểm tra “sức khỏe” của dữ liệu 7

1.3.3 Bước 7. Đổi tên cột cho dễ dùng 8

1.3.4 Bước 8. Nhìn vào biến mục tiêu 8

1.3.5 Bước 9. Kiểm tra và làm sạch các biến phân loại 10

1.3.6 Bước 10. Tìm hiểu biến trạng thái trả nợ 11

1.3.7 Bước 11. Thống kê mô tả và phân bố của các biến số 11

1.3.8 Bước 12. Tìm tín hiệu đầu tiên 12

1.4 Giai đoạn 3 – Chia dữ liệu thành tập huấn luyện và tập kiểm tra 14

1.4.1 Bước 13. Chia train/test 14

1.5 Giai đoạn 4 – Mô hình đầu tiên và cách chấm điểm 15

1.5.1 Bước 14. Mô hình “ngây thơ” và cái bẫy Accuracy 15

1.5.2 Bước 15. Mô hình thật đầu tiên: hồi quy logistic 16

1.5.3 Bước 16. Ma trận nhầm lẫn 17

1.5.4 Bước 17. Precision, Recall và F1 – tự tay tính 18

1.5.5 Bước 18. Xác suất dự đoán và ngưỡng quyết định 19

1.5.6 Bước 19. ROC-AUC và PR-AUC – chấm điểm trên mọi ngưỡng 21

1.6 Giai đoạn 5 – Đánh giá chéo 23

1.6.1 Bước 20. Vì sao một lần chia là chưa đủ? Đánh giá chéo k-fold 23

1.6.2 Bước 21. Viết gọn bằng `cross_validate` và vì sao cần pipeline 24

1.7 Giai đoạn 6 – So sánh nhiều mô hình 25

1.7.1 Bước 22. So sánh bốn mô hình bằng cùng một quy trình 25

1.7.2 Bước 23. Đánh giá một lần duy nhất trên tập test 26

1.7.3 Bước 24. Chênh lệch có phải do may rủi? Kiểm định thống kê 27
1.8 Giai đoạn 7 – Làm lại với bài toán hồi quy 28
1.8.1 Bước 25. Đọc dữ liệu và phát hiện mức chặn trên 28
1.8.2 Bước 26. Chỉ số đánh giá cho hồi quy – tự tay tính 30
1.8.3 Bước 27. So sánh mô hình hồi quy và đánh giá trên test 31
1.9 Tổng kết: giao thức chuẩn của giáo trình 33
1.10 Những lỗi thường gặp 34
1.11 Khi viết bài báo khoa học 35
1.12 Bài tập 35
1.13 Đáp án phần Tự kiểm tra 36
Thuật ngữ của chương 37

II Xử lý mất cân bằng dữ liệu 38

2 SMOTE – Sinh mẫu tổng hợp cho lớp thiểu số 39
2.1 Trước khi bắt đầu 39
2.2 Giai đoạn 1 – Nhìn lại vấn đề 40
2.2.1 Bước 1. Nạp dữ liệu đã chia ở Chương 1 40
2.2.2 Bước 2. Mô hình hiện tại bỏ sót bao nhiêu người? 41
2.3 Giai đoạn 2 – Cách ngây thơ: nhân bản dữ liệu 41
2.3.1 Bước 3. Một bộ dữ liệu "đồ chơi" để nhìn bằng mắt 42
2.3.2 Bước 4. Nhân bản ngẫu nhiên và điểm yếu của nó 42
2.4 Giai đoạn 3 – SMOTE từng bước bằng tay 43
2.4.1 Bước 5. Tìm láng giềng gần nhất 43
2.4.2 Bước 6. Sinh một điểm mới bằng tay 44
2.4.3 Bước 7. Tự viết hàm SMOTE hoàn chỉnh 45
2.4.4 Bước 8. Dùng SMOTE của thư viện và nhìn kết quả 46
2.5 Giai đoạn 4 – SMOTE trên dữ liệu thật 48
2.5.1 Bước 9. Vì sao phải chuẩn hóa trước khi dùng SMOTE? 48
2.5.2 Bước 10. SMOTE thay đổi tập train như thế nào? 49
2.5.3 Bước 11. Đánh giá chéo có và không có SMOTE 49
2.5.4 Bước 12. Kiểm tra trên tập test 51
2.6 Giai đoạn 5 – Cái bẫy rò rỉ dữ liệu 51
2.6.1 Bước 13. Làm sai thử một lần 52
2.6.2 Bước 14. Vì sao làm sai lại cho điểm cao như vậy? 52
2.7 Giai đoạn 6 – SMOTE thật ra làm gì? 53
2.7.1 Bước 15. Ảnh hưởng của số láng giềng k 53
2.7.2 Bước 16. Một đối thủ đơn giản: dời ngưỡng 55
2.7.3 Bước 17. SMOTE hay dời ngưỡng? 56

2.7.4 Bước 18. SMOTE làm “phình” xác suất . . . 56
2.8 Giai đoạn 7 – SMOTE và biến phân loại . . . 57
2.8.1 Bước 19. SMOTE “phá hỏng” biến phân loại . . . 57
2.8.2 Bước 20. Dùng SMOTE-NC cho dữ liệu hỗn hợp . . . 58
2.9 Tổng kết . . . 59
2.10 Những lỗi thường gặp . . . 60
2.11 Khi viết bài báo khoa học . . . 60
2.12 Bài tập . . . 60
2.13 Đáp án phần Tự kiểm tra . . . 61
Thuật ngữ của chương . . . 62

3 Borderline-SMOTE – Chỉ sinh mẫu ở vùng biên 63
3.1 Trước khi bắt đầu . . . 63
3.2 Giai đoạn 1 – Chuẩn bị . . . 63
3.2.1 Bước 1. Nạp dữ liệu . . . 64
3.3 Giai đoạn 2 – Phân loại các điểm thiểu số . . . 64
3.3.1 Bước 2. “Hỏi” các láng giềng . . . 64
3.3.2 Bước 3. Quy tắc phân loại . . . 65
3.3.3 Bước 4. Nhìn ba loại điểm trên mặt phẳng . . . 66
3.4 Giai đoạn 3 – Sinh mẫu ở vùng biên . . . 67
3.4.1 Bước 5. Tự viết Borderline-SMOTE-1 . . . 67
3.4.2 Bước 6. So sánh ba thuật toán bằng hình . . . 68
3.4.3 Bước 7. Đo mức “lấn” vào lớp đa số . . . 69
3.5 Giai đoạn 4 – Chẩn đoán dữ liệu thật . . . 70
3.5.1 Bước 8. Khách hàng vỡ nợ thuộc loại nào? . . . 70
3.5.2 Bước 9. Nhìn kỹ hơn: phân bố của m' . . . 71
3.6 Giai đoạn 5 – So sánh trên Credit Default . . . 72
3.6.1 Bước 10. Đánh giá chéo bốn chiến lược với hai mô hình . . . 72
3.6.2 Bước 11. Biểu đồ so sánh . . . 73
3.6.3 Bước 12. Kiểm tra trên tập test . . . 73
3.6.4 Bước 13. Đối chiếu với phương án dời ngưỡng . . . 74
3.7 Giai đoạn 6 – Khi nào Borderline-SMOTE có lợi? . . . 75
3.7.1 Bước 14. Thử với nhiều mức chồng lấn . . . 75
3.8 Tổng kết . . . 76
3.9 Những lỗi thường gặp . . . 76
3.10 Khi viết bài báo khoa học . . . 77
3.11 Bài tập . . . 77
3.12 Đáp án phần Tự kiểm tra . . . 78
Thuật ngữ của chương . . . 78

4 ADASYN – Sinh mẫu thích nghi theo độ khó **79**
4.1 Trước khi bắt đầu . . . 79
4.2 Giai đoạn 1 – Chuẩn bị . . . 80
4.2.1 Bước 1. Nạp dữ liệu . . . 80
4.3 Giai đoạn 2 – ADASYN bằng tay . . . 81
4.3.1 Bước 2. Đo độ khó của một điểm . . . 81
4.3.2 Bước 3. Chia ngân sách theo độ khó . . . 81
4.3.3 Bước 4. Nhìn ngân sách trên mặt phẳng . . . 83
4.3.4 Bước 5. Tự viết ADASYN . . . 83
4.4 Giai đoạn 3 – Ngân sách đi về đâu? . . . 84
4.4.1 Bước 6. So sánh ba thuật toán bằng hình . . . 84
4.4.2 Bước 7. Đếm ngân sách theo nhóm độ khó . . . 85
4.4.3 Bước 8. Trên dữ liệu thật . . . 86
4.5 Giai đoạn 4 – So sánh trên Credit Default . . . 87
4.5.1 Bước 9. Đánh giá chéo . . . 87
4.5.2 Bước 10. Ảnh hưởng của K . . . 88
4.6 Giai đoạn 5 – Thử thách: nhãn bị ghi sai . . . 89
4.6.1 Bước 11. Khi một phần dữ liệu train bị ghi nhãn sai . . . 89
4.7 Giai đoạn 6 – Tổng kết ba chương về lấy mẫu . . . 90
4.7.1 Bước 12. Bảng tổng kết trên tập test . . . 90
4.8 Tổng kết . . . 91
4.9 Những lỗi thường gặp . . . 91
4.10 Khi viết bài báo khoa học . . . 92
4.11 Bài tập . . . 92
4.12 Đáp án phần Tự kiểm tra . . . 92
Thuật ngữ của chương . . . 93

5 Class Weighting – Học có trọng số theo lớp **94**
5.1 Trước khi bắt đầu . . . 94
5.2 Giai đoạn 1 – Hàm mất mát có trọng số . . . 94
5.2.1 Bước 1. Nạp dữ liệu . . . 95
5.2.2 Bước 2. Tính tay hàm mất mát . . . 95
5.2.3 Bước 3. Trọng số bằng nhân bản . . . 96
5.2.4 Bước 4. Trọng số “balanced” . . . 97
5.3 Giai đoạn 2 – Quét trọng số . . . 98
5.3.1 Bước 5. Thử nhiều giá trị w . . . 98
5.4 Giai đoạn 3 – Trọng số tương đương dời ngưỡng . . . 99
5.4.1 Bước 6. Trọng số “uốn” thang xác suất . . . 99
5.4.2 Bước 7. Kiểm chứng trên dữ liệu thật . . . 101
5.4.3 Bước 8. Nhìn định lý bằng hình . . . 101

5.4.4 Bước 9. Cái giá phải trả: xác suất bị méo . . . 102
5.5 Giai đoạn 4 – Chọn ngưỡng từ chi phí . . . 103
5.5.1 Bước 10. Ngưỡng tối ưu khi biết chi phí . . . 103
5.5.2 Bước 11. Đường chi phí . . . 104
5.6 Giai đoạn 5 – Với hồi quy logistic . . . 105
5.6.1 Bước 12. Trọng số với một mô hình khác . . . 105
5.7 Tổng kết . . . 106
5.8 Những lỗi thường gặp . . . 106
5.9 Khi viết bài báo khoa học . . . 107
5.10 Bài tập . . . 107
Thuật ngữ của chương . . . 108

6 Focal Loss – Tập trung vào mẫu khó 109
6.1 Trước khi bắt đầu . . . 109
6.2 Giai đoạn 1 – Chuẩn bị . . . 110
6.2.1 Bước 1. Nạp dữ liệu và biến đổi đặc trưng . . . 110
6.3 Giai đoạn 2 – Focal loss bằng tay . . . 111
6.3.1 Bước 2. Tính tay cho 4 khách hàng . . . 111
6.3.2 Bước 3. Nhìn hình dạng hàm mất mát . . . 112
6.3.3 Bước 4. Viết focal loss bằng PyTorch . . . 112
6.4 Giai đoạn 3 – Mạng nơ-ron đầu tiên . . . 114
6.4.1 Bước 5. Xây và huấn luyện một MLP . . . 114
6.4.2 Bước 6. Đánh giá MLP . . . 116
6.5 Giai đoạn 4 – Focal loss trên Credit Default . . . 116
6.5.1 Bước 7. Focal loss kéo xác suất về phía 0,5 . . . 116
6.5.2 Bước 8. So sánh tám hàm mất mát . . . 117
6.5.3 Bước 9. Nhìn phân bố xác suất . . . 119
6.6 Giai đoạn 5 – So sánh công bằng sau khi dời ngưỡng . . . 120
6.6.1 Bước 10. Chọn ngưỡng riêng cho từng hàm mất mát . . . 120
6.7 Tổng kết . . . 121
6.8 Những lỗi thường gặp . . . 122
6.9 Khi viết bài báo khoa học . . . 122
6.10 Bài tập . . . 123
6.11 Đáp án phần Tự kiểm tra . . . 123
Thuật ngữ của chương . . . 124

Lời nói đầu

Giáo trình này dành cho sinh viên và người mới bắt đầu nghiên cứu Học máy, những người đã biết huấn luyện một mô hình cơ bản và muốn tiến thêm một bước: *làm cho mô hình tốt hơn, hiểu được mô hình, và tin được dự đoán của nó*, đồng thời viết được bài báo khoa học về những việc đó.

Cấu trúc

Giáo trình gồm 27 chương, chia thành bảy phần. Mỗi chương trình bày **một** kỹ thuật:

Phần	Nội dung
I. Nền tảng	Dữ liệu, mô hình cơ sở, giao thức đánh giá (Ch. 1)
II. Mất cân bằng	SMOTE, Borderline-SMOTE, ADASYN, Class weighting, Focal loss (Ch. 2–6)
III. Chọn đặc trưng	Mutual Information, LASSO, RFE, Boruta (Ch. 7–10)
IV. Tối ưu siêu tham số	Grid, Random, Bayesian Optimization, Optuna/TPE (Ch. 11–14)
V. Ensemble	Voting, Stacking, Blending (Ch. 15–17)
VI. Giải thích mô hình	Permutation, PDP, ICE, LIME, SHAP, Anchors, DiCE (Ch. 18–24)
VII. Độ tin cậy	Calibration, Conformal Prediction, MC Dropout (Ch. 25–27)

Bản này gồm những chương nào? Bản hiện tại gồm **Chương 1–6** (Phần I và toàn bộ Phần II), đã được viết hoàn chỉnh theo phong cách hướng dẫn từng bước. Các chương còn lại sẽ được bổ sung trong những bản tiếp theo. Khi một chương nhắc đến chương chưa có trong bản này (ví dụ "xem Chương 18"), số chương đó là theo dàn ý ở bảng trên.

Cách học với giáo trình này

Mỗi chương được chia thành nhiều **bước nhỏ**, gom trong vài *giai đoạn*. Đầu chương có sơ đồ lộ trình để bạn biết mình đang ở đâu. Hầu hết các bước có cùng một cấu trúc:

1. **Mục đích**: làm bước này để làm gì;
2. **Ô lệnh**: một đoạn code ngắn, thường dưới 15 dòng;
3. **Kết quả**: đúng những gì máy in ra khi chạy ô lệnh;
4. **Đọc kết quả**: giải thích từng con số, kèm ví dụ tính tay khi có công thức.

Xen kẽ là các hộp *Tự kiểm tra* (có đáp án cuối chương), *Thử tự làm* và *Mẹo*. Cuối mỗi chương có phần tổng kết, lỗi thường gặp, gợi ý khi viết bài báo khoa học, bài tập ba mức độ và bảng thuật ngữ Việt–Anh.

Hai bộ dữ liệu xuyên suốt

Toàn bộ giáo trình dùng hai bộ dữ liệu công khai, với cùng một cách chia train/test cố định, để kết quả giữa các chương so sánh được với nhau:

- **Credit Default** [22]: phân loại nhị phân, 30 000 mẫu, tỷ lệ vỡ nợ 22%.
- **California Housing** [19]: hồi quy, 20 640 mẫu.

Về các con số trong sách

Mọi hộp "Kết quả", bảng số và hình vẽ đều được sinh tự động từ mã nguồn đi kèm. Mỗi ô lệnh là một file trong thư mục `code/chXX/`; một script chạy lần lượt các ô lệnh và ghi phần máy in ra vào thư mục `outputs/`, rồi sách chèn trực tiếp những file này. Vì vậy, nếu bạn gõ lại đúng code, bạn sẽ thấy đúng các con số đã in; sai khác nhỏ ở chữ số cuối có thể có do phiên bản thư viện. Nhiều kết quả đi *ngược* kỳ vọng thông thường, chẳng hạn SMOTE không thắng phương án dời ngưỡng. Chúng được giữ nguyên vì đó chính là bài học.

Mã nguồn và notebook đi kèm

Mỗi chương có một notebook (`code/chXX/chXX_notebook.ipynb`) gồm đúng các ô lệnh trong sách, theo đúng thứ tự. Hãy đặt notebook ở *thư mục gốc* của dự án (cùng cấp với `data/`), rồi chạy lần lượt từng ô. Để chạy lại toàn bộ ô lệnh của một chương và cập nhật kết quả trong sách:

```
python code/run_chapter.py ch02        # chạy các ô lệnh của Chương 2
xelatex -shell-escape sach && bibtex sach && xelatex -shell-escape sach && xelatex
↪  -shell-escape sach
```

Gói `minted` cần Python và Pygments, đồng thời cần tùy chọn `-shell-escape`. Trên Overleaf, chọn trình biên dịch XeLaTeX. Thư mục `fonts/` chứa các phông chữ dùng trong sách (Be Vietnam Pro, Noto Serif, JetBrains Mono, giấy phép SIL Open Font License); hãy tải cả thư mục này lên cùng dự án.

Ký hiệu

$\mathbf{x}_i \in \mathbb{R}^d, y_i$	vector đặc trưng và nhãn của mẫu thứ i
n, d	số mẫu, số đặc trưng
n_0, n_1	số mẫu lớp âm (đa số), lớp dương (thiểu số)
$\hat{f}, \hat{p}(\mathbf{x})$	mô hình đã huấn luyện, xác suất dự đoán lớp dương
t	ngưỡng quyết định
$\mathbb{E}, \mathbb{P}, \mathbb{1}[\cdot]$	kỳ vọng, xác suất, hàm chỉ thị

PHẦN I

Nền tảng

CHƯƠNG 1

Bắt đầu từ con số không: dữ liệu, mô hình đầu tiên và cách chấm điểm

Chương này giúp bạn làm được gì?

Sau khi làm theo hết các bước trong chương, bạn sẽ:

- cài đặt được môi trường Python và tổ chức thư mục cho một dự án học máy;
- đọc, kiểm tra và làm sạch một bộ dữ liệu bảng có thật (30 000 khách hàng thẻ tín dụng);
- huấn luyện mô hình phân loại đầu tiên và **tự tay tính** các chỉ số Accuracy, Precision, Recall, F1;
- hiểu vì sao Accuracy có thể "nói dối", và vì sao cần ROC-AUC, PR-AUC;
- đánh giá mô hình đúng cách bằng *đánh giá chéo* (*cross-validation*);
- so sánh nhiều mô hình một cách công bằng, rồi áp dụng toàn bộ quy trình cho một bài toán hồi quy.

1.1 Trước khi bắt đầu

Bạn cần biết gì?

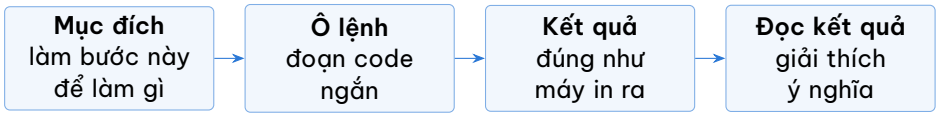
Chương này viết cho người *mới bắt đầu*. Bạn chỉ cần:

- biết Python cơ bản: biến, danh sách, vòng lặp `for`, hàm;
- đã từng nghe đến thư viện `pandas` (không cần thành thạo);
- nhớ một chút xác suất phổ thông: tỷ lệ, trung bình, phần trăm.

Mọi khái niệm học máy đều được giải thích lại từ đầu, kèm ví dụ bằng con số cụ thể.

Cách đọc chương này

Chương được chia thành **27 bước**, gom trong 7 giai đoạn (Hình 1.1). Hầu hết các bước có cùng một cấu trúc:



Các hộp *Kết quả* trong sách được tạo tự động bằng cách chạy chính các ô lệnh đó. Vì vậy,

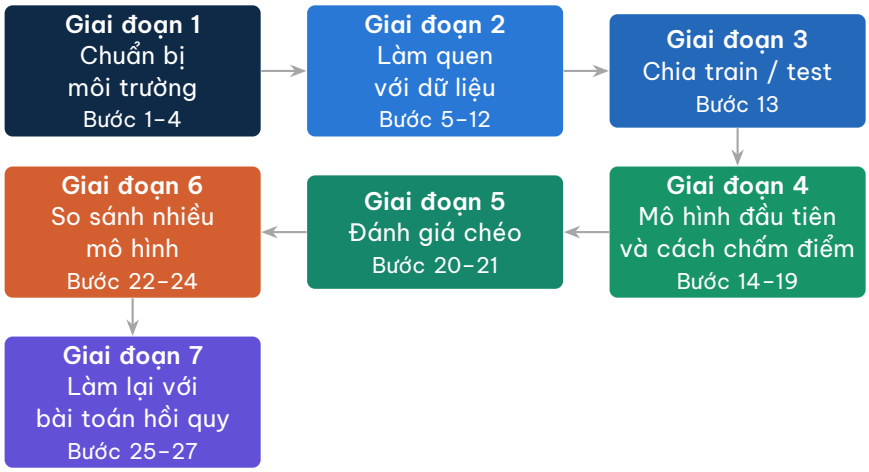
nếu bạn gõ lại đúng code, bạn sẽ thấy *đúng* những con số này. Có thể lệch ở chữ số cuối nếu phiên bản thư viện khác.

Xen kẽ trong chương có ba loại hộp nhỏ:

- **Tự kiểm tra**: câu hỏi ngắn để bạn tự đánh giá đã hiểu chưa. Đáp án ở cuối chương (Mục 1.13).
- **Thử tự làm**: gợi ý sửa một chi tiết trong code để quan sát kết quả thay đổi.
- **Mẹo**: kinh nghiệm thực tế giúp tránh mất thời gian.

Thời gian dự kiến

Khoảng 3–4 giờ nếu bạn gõ lại từng ô lệnh và làm các phần "Thử tự làm". Đừng cố đọc một mạch: hãy dừng lại sau mỗi giai đoạn.



Hình 1.1: Lộ trình của Chương 1: 7 giai đoạn, 27 bước.

Bài toán chúng ta sẽ giải

Một ngân hàng ở Đài Loan có dữ liệu của 30 000 khách hàng dùng thẻ tín dụng trong 6 tháng (tháng 4 đến tháng 9 năm 2005) [22]. Ngân hàng muốn biết trước: **khách hàng nào sẽ không trả được nợ (vỡ nợ) vào tháng 10?** Nếu biết sớm, ngân hàng có thể hạ hạn mức, nhắc nhở, hoặc cơ cấu lại khoản nợ.

Đây là bài toán **phân loại nhị phân**: mỗi khách hàng thuộc một trong hai lớp, $y = 1$ (vỡ nợ) hoặc $y = 0$ (không vỡ nợ). Mô hình nhận vào các thông tin về khách hàng, gọi là **đặc trưng** (*features*), và dự đoán y.

Ở cuối chương, ta sẽ làm lại toàn bộ quy trình với bài toán thứ hai: **dự đoán giá nhà** ở California. Ở đó, y là một con số liên tục, nên gọi là bài toán **hồi quy**. Hai bộ dữ liệu này sẽ đi cùng bạn suốt giáo trình.

1.2 Giai đoạn 1 – Chuẩn bị môi trường

1.2.1 Bước 1. Cài Python và tạo môi trường ảo

Mục đích: có một "góc làm việc" riêng cho dự án, để các thư viện cài vào không xung đột với dự án khác trên máy.

Nếu máy chưa có Python, hãy tải bản 3.11 trở lên từ `https://www.python.org` (trên Windows, nhớ tích ô *Add Python to PATH* khi cài). Sau đó mở cửa sổ lệnh (Windows: *Command Prompt* hoặc *PowerShell*; macOS/Linux: *Terminal*) và gõ:

Cửa sổ lệnh – Windows

```
mkdir giaotrinh_ml
cd giaotrinh_ml
py -m venv .venv
.venv\Scripts\activate
```

Cửa sổ lệnh – macOS / Linux

```
mkdir giaotrinh_ml
cd giaotrinh_ml
python3 -m venv .venv
source .venv/bin/activate
```

Đọc kết quả. Bốn dòng lệnh lần lượt: tạo thư mục dự án, đi vào thư mục đó, tạo một **môi trường ảo** tên `.venv`, rồi *kích hoạt* nó. Khi kích hoạt thành công, đầu dòng lệnh sẽ có chữ `(.venv)`. Mỗi lần mở cửa sổ lệnh mới để làm việc với dự án, bạn cần chạy lại lệnh kích hoạt (dòng cuối).

Dùng Google Colab nếu không muốn cài gì

Bạn có thể bỏ qua Bước 1 và dùng Google Colab (`https://colab.research.google.com`). Colab đã có sẵn Python và phần lớn thư viện. Khi đó chỉ cần tải file dữ liệu lên Colab ở Bước 3.

1.2.2 Bước 2. Cài các thư viện và kiểm tra phiên bản

Mục đích: cài những "hộp đồ nghề" mà ta sẽ dùng trong cả giáo trình.

Cửa sổ lệnh (đã kích hoạt môi trường ảo)

```
pip install numpy pandas matplotlib scikit-learn xgboost lightgbm xlrd jupyterlab
```

Mỗi thư viện có một vai trò:

Thư viện	Dùng để làm gì
`numpy`	Tính toán với mảng số (vector, ma trận)
`pandas`	Đọc và xử lý bảng dữ liệu (giống Excel nhưng bằng code)
`matplotlib`	Vẽ biểu đồ
`scikit-learn`	Thư viện học máy chính: chia dữ liệu, mô hình, chỉ số đánh giá
`xgboost`, `lightgbm`	Hai thư viện mô hình "boosting" rất mạnh cho dữ liệu bảng
`xlrd`	Giúp `pandas` đọc được file Excel đời cũ (`.xls`)
`jupyterlab`	Môi trường viết và chạy code theo từng ô (notebook)

Cài xong, khởi động Jupyter bằng lệnh `jupyter lab`. Trình duyệt sẽ mở ra; tạo một notebook

mới ở *thư mục gốc* của dự án và chạy ô lệnh đầu tiên:

Ô lệnh 1 · Kiểm tra phiên bản Python và các thư viện chính `code/ch01/s01_phien_ban.py`

```
import sys
import numpy, pandas, sklearn, matplotlib, xgboost, lightgbm

print("Python       :", sys.version.split()[0])
print("numpy        :", numpy.__version__)
print("pandas       :", pandas.__version__)
print("scikit-learn :", sklearn.__version__)
print("matplotlib   :", matplotlib.__version__)
print("xgboost      :", xgboost.__version__)
print("lightgbm     :", lightgbm.__version__)
```

Kết quả

```
Python       : 3.13.16
numpy        : 2.5.3
pandas       : 3.0.5
scikit-learn : 1.9.1
matplotlib   : 3.11.2
xgboost      : 3.4.1
lightgbm     : 4.7.0
```

Đọc kết quả. Nếu ô lệnh chạy không báo lỗi, mọi thư viện đã được cài đúng. Bạn không cần có *đúng* các phiên bản trên. Nếu bạn gặp lỗi `ModuleNotFoundError: No module named 'xgboost'`, nghĩa là thư viện đó chưa được cài vào *môi trường đang dùng*. Hãy kiểm tra lại xem notebook có đang chạy trong môi trường `.venv` không.

1.2.3 Bước 3. Tạo cấu trúc thư mục và đặt dữ liệu vào đúng chỗ

Mục đích: sắp xếp thư mục gọn gàng ngay từ đầu, để đường dẫn trong code luôn đúng.

Tạo cấu trúc thư mục như sau (các thư mục `figures/`, `code/` để trống cũng được):

Cấu trúc thư mục dự án

```
giaotrinh_ml/
|-- .venv/                    <- môi trường ảo (Bước 1)
|-- data/
|   `-- credit_default.xls    <- dữ liệu Credit Default
|-- figures/                  <- nơi lưu các hình vẽ
|-- code/
|   `-- ch01/                 <- code của Chương 1
`-- ch01.ipynb                <- notebook bạn đang làm việc
```

Tải dữ liệu. Vào trang UCI Machine Learning Repository, tìm "Default of Credit Card Clients" (mã bộ dữ liệu 350), bấm *Download*, giải nén và được file `default of credit card clients.xls`. Đổi tên thành `credit_default.xls` rồi chép vào thư mục `data/`. Bộ mã nguồn đi kèm giáo trình cũng đã có sẵn file này.

Vì sao notebook phải ở thư mục gốc?

Trong code, ta viết đường dẫn *tương đối* như `"data/credit_default.xls"`. Python sẽ tìm file này tính từ thư mục mà notebook đang đứng. Nếu bạn đặt notebook trong `code/ch01/`, Python sẽ tìm `code/ch01/data/...` và báo lỗi `FileNotFoundError`.

1.2.4 Bước 4. Nạp thư viện và thiết lập chung

Mục đích: gọi các thư viện ra dùng và đặt vài thiết lập để kết quả dễ đọc, dễ tái lập.

Ô lệnh 2 · Nạp thư viện và thiết lập chung cho cả chương `code/ch01/s02_thiet_lap.py`

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 80)       # mỗi dòng bảng in ra tối đa 80 ký tự
pd.set_option("display.max_columns", 8)  # hiện tối đa 8 cột, phần còn lại thay bằng
↪  "..."
pd.set_option("display.precision", 3)    # làm tròn 3 chữ số sau dấu phẩy

SEED = 42  # "hạt giống" ngẫu nhiên: cố định để mỗi lần chạy cho cùng một kết quả
```

Kết quả

```
(Ô lệnh này không in ra gì.)
```

Đọc kết quả. Ô lệnh này không in gì, nhưng có hai điểm quan trọng:

- `import pandas as pd`: đặt tên viết tắt `pd` cho thư viện. Đây là quy ước chung của cộng đồng; bạn sẽ thấy `pd`, `np`, `plt` trong hầu hết tài liệu.
- `SEED = 42`: nhiều bước trong học máy có yếu tố ngẫu nhiên (chia dữ liệu, khởi tạo mô hình). Cố định "hạt giống" giúp mỗi lần chạy lại cho *cùng* kết quả. Con số 42 không có gì đặc biệt; điều quan trọng là dùng *một* giá trị cố định trong suốt dự án.

1.3 Giai đoạn 2 – Làm quen với dữ liệu

Trước khi xây bất kỳ mô hình nào, ta cần *hiểu* dữ liệu: có bao nhiêu dòng, bao nhiêu cột, mỗi cột nghĩa là gì, có lỗi hay giá trị lạ không. Người làm dữ liệu có kinh nghiệm thường dành phần lớn thời gian cho giai đoạn này.

1.3.1 Bước 5. Đọc dữ liệu vào bảng

Mục đích: đưa file Excel vào Python dưới dạng một bảng (DataFrame) để xử lý.

Ô lệnh 3 · Đọc file Excel vào một bảng dữ liệu (DataFrame) code/ch01/s03_doc_du_lieu.py

```
df = pd.read_excel("data/credit_default.xls", header=1)

print("Số dòng, số cột:", df.shape)
print()
print(df.iloc[:5, :7])   # 5 dòng đầu, 7 cột đầu
```

Kết quả

```
Số dòng, số cột: (30000, 25)

   ID  LIMIT_BAL  SEX  EDUCATION  MARRIAGE  AGE  PAY_0
0   1      20000    2          2         1   24      2
1   2     120000    2          2         2   26     -1
2   3      90000    2          2         2   34      0
3   4      50000    2          2         1   37      0
4   5      50000    1          2         1   57     -1
```

Đọc kết quả.

- `df.shape` cho kết quả `(30000, 25)`: bảng có 30 000 dòng (mỗi dòng là một khách hàng) và 25 cột.
- `df.iloc[:5, :7]` lấy 5 dòng đầu và 7 cột đầu. Chữ `iloc` nghĩa là "chọn theo vị trí" (*integer location*); `:5` là "từ đầu đến trước vị trí 5".
- Cột bên trái (0, 1, 2, ...) là **chỉ số dòng** (*index*) do `pandas` tự đánh, không phải dữ liệu.

Khách hàng đầu tiên có hạn mức 20 000 Đô la Đài Loan, là nữ (`SEX` = 2), 24 tuổi, và `PAY_0` = 2, nghĩa là đang trễ hạn 2 tháng.

Vì sao `header=1`?

Dòng đầu tiên của file Excel gốc là một dòng tiêu đề phụ (`X1`, `X2`, ...), dòng thứ hai mới là tên cột thật. Tham số `header=1` báo cho `pandas` lấy dòng thứ hai (đếm từ 0) làm tên cột. Nếu quên tham số này, tên cột sẽ thành `X1`, `X2`, ... Bạn hãy thử bỏ `header=1` để thấy sự khác biệt.

1.3.2 Bước 6. Kiểm tra "sức khỏe" của dữ liệu

Mục đích: phát hiện sớm ba vấn đề hay gặp: sai kiểu dữ liệu, thiếu giá trị, trùng lặp.

Ô lệnh 4 · Kiểm tra kiểu dữ liệu, giá trị thiếu và dòng trùng lặp code/ch01/s04_kiem_tra.py

```
print("Kiểu dữ liệu của các cột:")
print(df.dtypes.value_counts())
print()
print("Tổng số ô bị thiếu (NaN):", df.isna().sum().sum())
print("Số dòng trùng lặp (bỏ qua cột ID):", df.drop(columns="ID").duplicated().sum())
```

Kết quả

```
Kiểu dữ liệu của các cột:
int64    25
Name: count, dtype: int64
```

```
Tổng số ô bị thiếu (NaN): 0
Số dòng trùng lặp (bỏ qua cột ID): 35
```

Đọc kết quả.

- **Kiểu dữ liệu:** cả 25 cột đều là số nguyên (`int64`). Không có cột nào bị đọc nhầm thành chữ, đây là tin tốt. (Dòng `Name: count, dtype: int64` chỉ là "nhãn" mà `pandas` in kèm, có thể bỏ qua.)
- **Giá trị thiếu:** 0 ô. Dữ liệu thực tế thường có ô trống; bộ dữ liệu này đã được làm sạch sẵn.
- **Trùng lặp:** có 35 dòng giống hệt một dòng khác ở *mọi* cột (khi bỏ qua mã khách hàng `ID`). Kiểm tra kỹ cho thấy phần lớn là khách hàng không phát sinh giao dịch (toàn bộ dư nợ và số tiền trả bằng 0), nên trùng nhau là tự nhiên. Với 35 trên 30 000 dòng (khoảng 0,1%), ta giữ nguyên.

1.3.3 Bước 7. Đổi tên cột cho dễ dùng

Mục đích: bỏ cột không có ích cho dự đoán và đặt tên cột nhất quán.

Ô lệnh 5 · Bỏ cột ID và đặt lại tên cột cho dễ dùng code/ch01/s05_doi_ten.py

```python
df = df.drop(columns="ID")
df = df.rename(columns={"PAY_0": "PAY_1",
                        "default payment next month": "default"})

cols = df.columns.tolist()
for i in range(0, len(cols), 6):      # in 6 tên cột trên mỗi dòng cho gọn
    print(cols[i:i + 6])
```

Kết quả

```
['LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_1']
['PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1']
['BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1']
['PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6', 'default']
```

Đọc kết quả. Còn lại 24 cột: 23 đặc trưng và 1 cột nhãn `default`. Ta làm hai việc:

- **Bỏ cột `ID`**: mã khách hàng chỉ là số thứ tự, không mang thông tin về khả năng trả nợ. Nếu để lại, mô hình có thể "học" những quy luật vô nghĩa từ nó.
- **Đổi `PAY_0` thành `PAY_1`**: trong file gốc, trạng thái trả nợ tháng 9 tên là `PAY_0`, trong khi dư nợ tháng 9 lại tên `BILL_AMT1`. Đổi tên để số thứ tự tháng khớp nhau giữa ba nhóm cột.

Ý nghĩa của từng cột được tóm tắt trong Bảng 1.1.

1.3.4 Bước 8. Nhìn vào biến mục tiêu

Mục đích: biết hai lớp có số lượng chênh lệch nhau thế nào, vì điều này quyết định cách ta chấm điểm mô hình.

Bảng 1.1: Từ điển dữ liệu Credit Default (23 đặc trưng và 1 nhãn).

Cột	Ý nghĩa
`LIMIT_BAL`	Hạn mức tín dụng (Đô la Đài Loan), gồm cả thẻ phụ của gia đình
`SEX`	Giới tính: 1 = nam, 2 = nữ
`EDUCATION`	Học vấn: 1 = sau đại học, 2 = đại học, 3 = trung học, 4 = khác
`MARRIAGE`	Hôn nhân: 1 = đã kết hôn, 2 = độc thân, 3 = khác
`AGE`	Tuổi (năm)
`PAY_1`–`PAY_6`	Trạng thái trả nợ, từ tháng 9 (`PAY_1`) lùi về tháng 4 (`PAY_6`): -1 = trả đúng hạn; 1, 2, … , 8 = trễ 1, 2, ..., 8 tháng; 9 = trễ từ 9 tháng trở lên
`BILL_AMT1`–`BILL_AMT6`	Dư nợ trên sao kê từng tháng (tháng 9 lùi về tháng 4)
`PAY_AMT1`–`PAY_AMT6`	Số tiền khách hàng đã thanh toán từng tháng
`default`	**Nhãn**: 1 = vỡ nợ vào tháng 10, 0 = không vỡ nợ

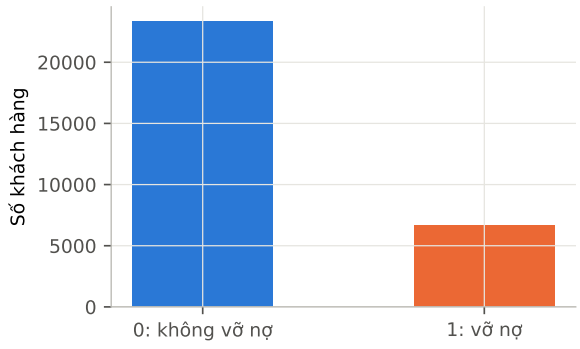
Ô lệnh 6 · Biến mục tiêu: có bao nhiêu khách hàng vỡ nợ? `code/ch01/s06_bien_muc_tieu.py`

```
counts = df["default"].value_counts()
ratio = df["default"].value_counts(normalize=True)
print(pd.DataFrame({"Số lượng": counts, "Tỷ lệ": ratio}))

fig, ax = plt.subplots(figsize=(4, 2.6))
ax.bar(["0: không vỡ nợ", "1: vỡ nợ"], counts.sort_index().values,
       color=["#2a78d6", "#eb6834"], width=0.5)
ax.set_ylabel("Số khách hàng")
plt.savefig("figures/ch01_s06_target.pdf", bbox_inches="tight")  # Jupyter: plt.show()
```

Kết quả

```
         Số lượng  Tỷ lệ
default
0           23364  0.779
1            6636  0.221
```



Hình 1.2: Số khách hàng theo nhãn `default`.

Đọc kết quả. Có 6 636 khách hàng vỡ nợ, chiếm 22,1%. Lớp "không vỡ nợ" đông gấp khoảng $23\,364/6\,636 \approx 3{,}5$ lần lớp "vỡ nợ". Ta nói dữ liệu bị **mất cân bằng** (*imbalanced*):

- lớp đông hơn gọi là **lớp đa số** (ở đây là lớp 0);
- lớp ít hơn gọi là **lớp thiểu số**, và thường là lớp ta *quan tâm* (ở đây là lớp 1, vỡ nợ). Ta cũng

gọi nó là **lớp dương**.

Hãy nhớ con số 22,1%. Nó sẽ xuất hiện lại nhiều lần trong chương.

Tự kiểm tra

Câu 1. Nếu một mô hình lúc nào cũng trả lời "không vỡ nợ" cho mọi khách hàng, nó sẽ đoán đúng bao nhiêu phần trăm khách hàng?

1.3.5 Bước 9. Kiểm tra và làm sạch các biến phân loại

Mục đích: tìm những giá trị "lạ" không có trong mô tả dữ liệu, rồi xử lý chúng.

Theo Bảng 1.1, `EDUCATION` chỉ nên nhận các giá trị 1–4 và `MARRIAGE` chỉ nên nhận 1–3. Hãy kiểm tra xem thực tế có đúng như vậy không:

Ô lệnh 7 · Ba biến phân loại nhận những giá trị nào? code/ch01/s07_bien_phan_loai.py

```
for col in ["SEX", "EDUCATION", "MARRIAGE"]:
    print(col, "->", df[col].value_counts().sort_index().to_dict())
```

Kết quả

```
SEX -> {1: 11888, 2: 18112}
EDUCATION -> {0: 14, 1: 10585, 2: 14030, 3: 4917, 4: 123, 5: 280, 6: 51}
MARRIAGE -> {0: 54, 1: 13659, 2: 15964, 3: 323}
```

Đọc kết quả. Hàm `value_counts()` đếm số lần mỗi giá trị xuất hiện; `sort_index()` sắp xếp theo giá trị; `to_dict()` in ra cho gọn trên một dòng. Ta thấy:

- `SEX` chỉ có 1 và 2. Đúng mô tả.
- `EDUCATION` có thêm các mã **0, 5, 6** (tổng cộng $14 + 280 + 51 = 345$ khách hàng) không có trong từ điển dữ liệu.
- `MARRIAGE` có thêm mã **0** (54 khách hàng).

Ta không biết các mã này nghĩa là gì. Cách xử lý thận trọng là gộp chúng vào nhóm "khác" đã có sẵn:

Ô lệnh 8 · Gộp các mã không có trong từ điển dữ liệu vào nhóm "khác" code/ch01/s08_lam_sach.py

```
df["EDUCATION"] = df["EDUCATION"].replace({0: 4, 5: 4, 6: 4})   # 4 = khác
df["MARRIAGE"] = df["MARRIAGE"].replace({0: 3})                 # 3 = khác

for col in ["EDUCATION", "MARRIAGE"]:
    print(col, "->", df[col].value_counts().sort_index().to_dict())
```

Kết quả

```
EDUCATION -> {1: 10585, 2: 14030, 3: 4917, 4: 468}
MARRIAGE -> {1: 13659, 2: 15964, 3: 377}
```

Đọc kết quả. Sau khi gộp, `EDUCATION` còn 4 nhóm (nhóm 4 có $123 + 345 = 468$ người) và `MARRIAGE` còn 3 nhóm (nhóm 3 có $323 + 54 = 377$ người). Lệnh `replace({0: 4, 5: 4, 6: 4})` có nghĩa là "thay 0 bằng 4, thay 5 bằng 4, thay 6 bằng 4".

Ghi lại mọi quyết định làm sạch

Mỗi lần thay đổi dữ liệu, hãy ghi lại *đã làm gì* và *vì sao*. Khi viết báo cáo hay bài báo, bạn sẽ cần mô tả lại các bước này. Người đọc cần biết để có thể làm lại đúng như bạn.

1.3.6 Bước 10. Tìm hiểu biến trạng thái trả nợ

Mục đích: hiểu nhóm biến `PAY_*`, thường là nhóm quan trọng nhất trong bài toán tín dụng.

Ô lệnh 9 · Trạng thái trả nợ tháng 9 (PAY_1) nhận những giá trị nào? `code/ch01/s09_pay.py`

```
print(df["PAY_1"].value_counts().sort_index().to_dict())
```

Kết quả

```
{-2: 2759, -1: 5686, 0: 14737, 1: 3688, 2: 2667, 3: 322, 4: 76, 5: 26, 6: 11, 7: 9, 8: 19}
```

Đọc kết quả. `PAY_1` nhận các giá trị từ −2 đến 8. Theo từ điển dữ liệu, −1 là "trả đúng hạn" và 1–8 là "số tháng trễ". Còn hai giá trị −2 (2 759 người) và 0 (14 737 người, nhiều nhất!) lại *không* được mô tả. Các nghiên cứu sau này thường hiểu:

- −2: không phát sinh chi tiêu trong tháng;
- 0: có dùng tín dụng quay vòng (trả mức tối thiểu, phần còn lại chuyển sang tháng sau).

Ta giữ nguyên các giá trị này, vì thứ tự $-2 < -1 < 0 < 1 < \dots$ vẫn hợp lý: số càng lớn thì tình trạng trả nợ càng xấu.

1.3.7 Bước 11. Thống kê mô tả và phân bố của các biến số

Mục đích: biết mỗi biến số nằm trong khoảng nào, có giá trị âm hay giá trị cực lớn bất thường không.

Ô lệnh 10 · Thống kê mô tả cho bốn biến số tiêu biểu `code/ch01/s10_mo_ta.py`

```
num_cols = ["LIMIT_BAL", "AGE", "BILL_AMT1", "PAY_AMT1"]
desc = df[num_cols].describe().T            # .T: xoay bảng để mỗi biến là một dòng
print(desc[["mean", "std", "min", "50%", "max"]].round(0))
```

Kết quả

```
               mean       std       min       50%        max
LIMIT_BAL  167484.0  129748.0   10000.0  140000.0  1000000.0
AGE            35.0       9.0      21.0      34.0       79.0
BILL_AMT1   51223.0   73636.0 -165580.0   22382.0   964511.0
PAY_AMT1     5664.0   16563.0       0.0    2100.0   873552.0
```

Đọc kết quả. Năm cột lần lượt là: trung bình (`mean`), độ lệch chuẩn (`std`), nhỏ nhất (`min`), trung vị (`50\%`, giá trị đứng giữa khi xếp từ nhỏ đến lớn) và lớn nhất (`max`).

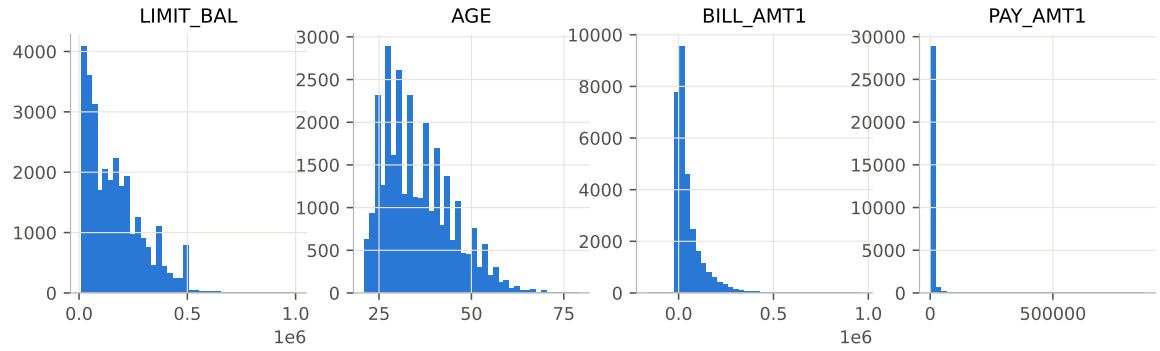
- `AGE`: từ 21 đến 79 tuổi, trung bình 35. Hợp lý.
- `BILL_AMT1` có giá trị *âm* (−165 580). Dư nợ âm nghĩa là khách hàng đã trả *thừa*, ngân hàng đang "nợ" lại khách. Đây không phải lỗi.

- `PAY_AMT1`: trung bình 5 664 nhưng trung vị chỉ 2 100, và lớn nhất lên tới 873 552. Khi **trung bình lớn hơn nhiều so với trung vị**, nghĩa là có một số ít giá trị rất lớn kéo trung bình lên. Ta gọi đây là phân bố **lệch phải** (có "đuôi dài" bên phải).

Biểu đồ phân bố (*histogram*) giúp thấy rõ điều này:

Ô lệnh 11 · Vẽ phân bố (histogram) của bốn biến số `code/ch01/s11_histogram.py`

```
fig, axes = plt.subplots(1, 4, figsize=(10, 2.4))
for ax, col in zip(axes, num_cols):
    ax.hist(df[col], bins=40, color="#2a78d6")
    ax.set_title(col)
    ax.locator_params(axis="x", nbins=3)     # chỉ hiện khoảng 3 vạch số cho đỡ rối
plt.savefig("figures/ch01_s11_hist.pdf", bbox_inches="tight")
```



Hình 1.3: Phân bố của bốn biến số. `BILL_AMT1` và `PAY_AMT1` lệch phải rất mạnh: hầu hết khách hàng dồn ở gần 0, chỉ một số ít có giá trị rất lớn.

Đọc kết quả. Mỗi cột trong histogram cho biết có bao nhiêu khách hàng rơi vào một khoảng giá trị. Ở `PAY_AMT1`, gần như toàn bộ khách hàng dồn vào cột đầu tiên, còn phần lớn trục hoành gần như trống. Ta ghi nhớ đặc điểm "đuôi dài" này: nó ảnh hưởng đến những mô hình nhạy với thang đo như hồi quy logistic (Bước 15).

1.3.8 Bước 12. Tìm tín hiệu đầu tiên

Mục đích: trước khi dùng mô hình phức tạp, kiểm tra bằng mắt xem có đặc trưng nào liên quan rõ đến vỡ nợ không.

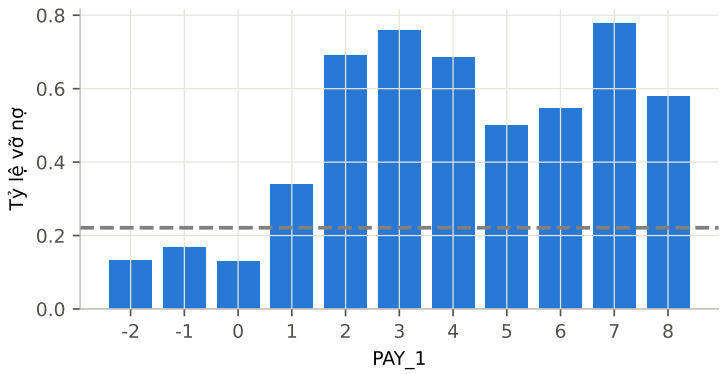
Ô lệnh 12 · Tỷ lệ vỡ nợ theo trạng thái trả nợ tháng 9 `code/ch01/s12_ty_le_theo_pay.py`

```
rate = df.groupby("PAY_1")["default"].agg(["count", "mean"])
rate.columns = ["Số khách hàng", "Tỷ lệ vỡ nợ"]
print(rate)

fig, ax = plt.subplots(figsize=(5.5, 2.6))
ax.bar(rate.index.astype(str), rate["Tỷ lệ vỡ nợ"], color="#2a78d6")
ax.axhline(df["default"].mean(), color="gray", ls="--")   # đường tỷ lệ chung 22%
ax.set_xlabel("PAY_1")
ax.set_ylabel("Tỷ lệ vỡ nợ")
plt.savefig("figures/ch01_s12_rate.pdf", bbox_inches="tight")
```

Kết quả

```
       Số khách hàng  Tỷ lệ vỡ nợ
PAY_1
-2              2759        0.132
-1              5686        0.168
 0             14737        0.128
 1              3688        0.339
 2              2667        0.691
 3               322        0.758
 4                76        0.684
 5                26        0.500
 6                11        0.545
 7                 9        0.778
 8                19        0.579
```



Hình 1.4: Tỷ lệ vỡ nợ theo `PAY_1`. Đường nét đứt là tỷ lệ chung 22,1%.

Đọc kết quả. Lệnh `groupby("PAY_1")` chia khách hàng thành từng nhóm theo giá trị `PAY_1`; `agg(["count", "mean"])` tính số người và giá trị trung bình của `default` trong mỗi nhóm. Vì `default` chỉ nhận 0 hoặc 1, **trung bình của nó chính là tỷ lệ vỡ nợ**. Ví dụ, một nhóm gồm 4 người có nhãn 1, 0, 0, 1 thì trung bình là $2/4 = 0{,}5$, tức 50% vỡ nợ.

Kết quả rất rõ ràng:

- Khách hàng trả đúng hạn hoặc không phát sinh chi tiêu (`PAY_1` $\leq$ 0): tỷ lệ vỡ nợ chỉ 13–17%.
- Trễ 1 tháng: 33,9%.
- Trễ 2 tháng: 69,1%, gấp hơn 3 lần tỷ lệ chung.

Ở các nhóm trễ từ 4 tháng trở lên, tỷ lệ dao động thất thường (50%, 54,5%, 77,8%...) vì mỗi nhóm chỉ có vài chục người hoặc ít hơn. Nhóm `PAY_1`$=$ 7 chỉ có 9 người; một người thay đổi nhãn là tỷ lệ đã đổi hơn 10%. **Đừng tin tỷ lệ tính trên nhóm quá nhỏ.**

Bài học của Giai đoạn 2

Chỉ với một bảng đếm đơn giản, ta đã biết `PAY_1` là đặc trưng rất mạnh. Mô hình học máy làm việc tương tự, nhưng tự động, trên *tất cả* 23 đặc trưng cùng lúc, và tìm được cả những kết hợp phức tạp mà mắt người khó thấy.

1.4 Giai đoạn 3 – Chia dữ liệu thành tập huấn luyện và tập kiểm tra

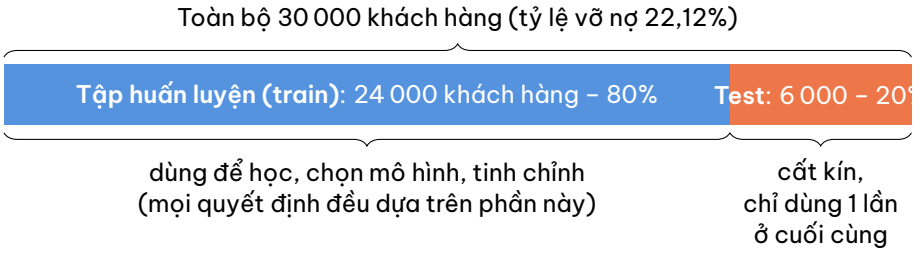
1.4.1 Bước 13. Chia train/test

Mục đích: giữ lại một phần dữ liệu mà mô hình *không bao giờ được nhìn thấy* khi học, để cuối cùng kiểm tra xem mô hình có thật sự dự đoán tốt trên khách hàng mới không.

Vì sao phải chia?

Hãy nghĩ đến một kỳ thi. Nếu sinh viên được xem trước đề thi và học thuộc đáp án, điểm thi rất cao nhưng không phản ánh năng lực thật. Mô hình học máy cũng vậy: nếu đánh giá mô hình trên chính dữ liệu nó đã học, kết quả sẽ đẹp một cách giả tạo. Vì vậy ta chia dữ liệu thành hai phần:

- **Tập huấn luyện** (*train*): mô hình được "học" từ đây. Giống sách bài tập.
- **Tập kiểm tra** (*test*): cất đi, chỉ dùng một lần ở cuối cùng. Giống đề thi cuối kỳ.



Hình 1.5: Chia dữ liệu theo tỷ lệ 80/20, giữ nguyên tỷ lệ vỡ nợ ở cả hai phần.

Ô lệnh 13 · Tách đặc trưng X và nhãn y, rồi chia train/test theo tỷ lệ 80/20 `code/ch01/s13_chia.py`

```
from sklearn.model_selection import train_test_split

y = df["default"]
X = df.drop(columns="default").astype(float)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)

print("X_train:", X_train.shape, "   X_test:", X_test.shape)
print(f"Tỷ lệ vỡ nợ -- toàn bộ: {y.mean():.4f} | train: {y_train.mean():.4f}"
      f" | test: {y_test.mean():.4f}")
```

Kết quả

```
X_train: (24000, 23)    X_test: (6000, 23)
Tỷ lệ vỡ nợ -- toàn bộ: 0.2212 | train: 0.2212 | test: 0.2212
```

Đọc kết quả. Ô lệnh làm hai việc:

1. **Tách `X` và `y`.** `X` là bảng 23 cột đặc trưng (đầu vào của mô hình), `y` là cột nhãn (đáp án). Theo quy ước, chữ hoa `X` chỉ một *bảng* (nhiều cột), chữ thường `y` chỉ một *cột*. Phần

`.astype(float)` đổi các cột sang kiểu số thực, cho thống nhất.

2. **Chia** bằng `train_test_split`. Hàm trả về bốn phần theo đúng thứ tự: `X_train`, `X_test`, `y_train`, `y_test`. Ba tham số cần chú ý:
   - `test_size=0.2`: 20% dữ liệu, tức 6 000 khách hàng, dành cho test.
   - `stratify=y`: **chia phân tầng**, tức giữ nguyên tỷ lệ vỡ nợ ở cả hai phần. Kết quả cho thấy cả ba con số đều là 0,2212.
   - `random_state=SEED`: cố định cách chia, để lần sau chạy lại vẫn ra đúng 6 000 khách hàng test này.

Thử tự làm

Bỏ tham số `stratify=y` rồi chạy lại. Tỷ lệ vỡ nợ ở tập train và tập test bây giờ là bao nhiêu? Chúng vẫn gần 0,22, nhưng không còn bằng nhau tuyệt đối. Với dữ liệu nhỏ hơn hoặc mất cân bằng nặng hơn (ví dụ lớp dương chỉ 1%), chênh lệch có thể lớn đến mức kết quả đánh giá bị sai lệch đáng kể.

Nguyên tắc vàng

Từ giờ trở đi, **không dùng `X_test`, `y_test` để ra bất kỳ quyết định nào**: không dùng để chọn mô hình, không dùng để chỉnh tham số, không dùng để chọn đặc trưng. Tập test chỉ được dùng *một lần* ở Bước 23. Vi phạm nguyên tắc này là lỗi phổ biến nhất trong khóa luận và bài báo của sinh viên.

1.5 Giai đoạn 4 – Mô hình đầu tiên và cách chấm điểm

1.5.1 Bước 14. Mô hình "ngây thơ" và cái bẫy Accuracy

Mục đích: có một mốc so sánh tối thiểu. Mô hình nào không vượt qua được mốc này thì vô dụng.

Ô lệnh 14 · Mô hình "ngây thơ": lúc nào cũng đoán lớp xuất hiện nhiều nhất — code/ch01/s14_dummy.py

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

dummy = DummyClassifier(strategy="most_frequent")
dummy.fit(X_train, y_train)
pred_dummy = dummy.predict(X_test)

print("Các dự đoán:", pd.Series(pred_dummy).value_counts().to_dict())
print("Accuracy   :", round(accuracy_score(y_test, pred_dummy), 4))
```

Kết quả

```
Các dự đoán: {0: 6000}
Accuracy   : 0.7788
```

Đọc kết quả. `DummyClassifier(strategy="most_frequent")` là một "mô hình" không học gì cả: nó chỉ nhớ lớp nào xuất hiện nhiều nhất trong tập train (lớp 0) và luôn trả lời lớp đó. Kết quả: dự đoán 6 000 khách hàng đều "không vỡ nợ".

Độ chính xác (*Accuracy*) là tỷ lệ dự đoán đúng:

$$\text{Accuracy} = \frac{\text{số dự đoán đúng}}{\text{tổng số dự đoán}} = \frac{4\,673}{6\,000} = 0{,}7788.$$

(Có $6\,000 - 1\,327 = 4\,673$ khách hàng thực sự không vỡ nợ trong tập test, và mô hình ngây thơ đoán đúng tất cả họ.)

Một mô hình *không phát hiện được bất kỳ ai vỡ nợ* vẫn đạt Accuracy gần 78%. Đây là **cái bẫy Accuracy** khi dữ liệu mất cân bằng: con số trông cao, nhưng mô hình hoàn toàn vô dụng với ngân hàng. Đáp án Câu 1 cũng chính là con số này.

1.5.2 Bước 15. Mô hình thật đầu tiên: hồi quy logistic

Mục đích: huấn luyện một mô hình thật sự học từ dữ liệu.

Hồi quy logistic hoạt động thế nào? Mô hình tính một "điểm rủi ro" z cho mỗi khách hàng bằng cách cộng các đặc trưng, mỗi đặc trưng nhân với một **trọng số** w:

$$z = w_0 + w_1 \cdot \text{LIMIT_BAL} + w_2 \cdot \text{SEX} + \cdots + w_{23} \cdot \text{PAY_AMT6}.$$

Sau đó, điểm z (có thể là số bất kỳ, âm hay dương) được "nén" vào khoảng $(0, 1)$ bằng hàm **sigmoid**, để trở thành một xác suất:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Ví dụ: $z = 0$ cho $\hat{p} = 0{,}5$; $z = 2$ cho $\hat{p} = 1/(1 + e^{-2}) \approx 0{,}88$; $z = -2$ cho $\hat{p} \approx 0{,}12$. "Huấn luyện" mô hình nghĩa là tìm các trọng số w sao cho $\hat{p}$ gần với nhãn thật nhất có thể trên tập train. Cuối cùng, mô hình dự đoán "vỡ nợ" nếu $\hat{p} \geq 0{,}5$.

Vì sao cần chuẩn hóa? `LIMIT_BAL` có giá trị hàng trăm nghìn, còn `SEX` chỉ là 1 hoặc 2. Thang đo chênh lệch quá lớn khiến việc tìm trọng số khó khăn và chậm. **Chuẩn hóa** (*standardization*) đưa mọi đặc trưng về cùng thang đo, bằng cách trừ trung bình rồi chia cho độ lệch chuẩn:

$$x_{\text{mới}} = \frac{x - \text{trung bình}}{\text{độ lệch chuẩn}}.$$

Ví dụ: với `LIMIT_BAL` (trung bình 167 484, độ lệch chuẩn 129 748 theo Bước 11), khách hàng có hạn mức 20 000 được chuẩn hóa thành $(20\,000 - 167\,484)/129\,748 \approx -1{,}14$, nghĩa là "thấp hơn trung bình 1,14 lần độ lệch chuẩn".

Ô lệnh 15 · Hồi quy logistic, có chuẩn hóa dữ liệu trước khi học code/ch01/s15_logreg.py

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
logreg.fit(X_train, y_train)
pred_lr = logreg.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, pred_lr), 4))
print("Số khách hàng bị dự đoán là vỡ nợ:", pred_lr.sum(), "/", len(pred_lr))
print("Số khách hàng thực sự vỡ nợ      :", y_test.sum())
```

Kết quả

```
Accuracy: 0.8083
Số khách hàng bị dự đoán là vỡ nợ: 465 / 6000
Số khách hàng thực sự vỡ nợ      : 1327
```

Đọc kết quả.

- Hàm `make_pipeline` ghép `StandardScaler()` và `LogisticRegression()` thành một **đường ống** (*pipeline*): dữ liệu đi qua bước chuẩn hóa *rồi mới* vào mô hình. Khi gọi `fit`, cả hai bước đều được "học" trên tập train. Khi gọi `predict`, dữ liệu test được chuẩn hóa bằng *chính* trung bình và độ lệch chuẩn của tập train. Bước 21 sẽ giải thích vì sao điều này quan trọng.
- `max_iter=2000`: cho phép thuật toán chạy tối đa 2 000 vòng lặp để tìm trọng số. Giá trị mặc định (100) đôi khi không đủ.
- Accuracy đạt 0,8083, chỉ hơn mô hình ngây thơ (0,7788) khoảng 3 điểm phần trăm.
- Quan trọng hơn: mô hình chỉ dự đoán 465 người vỡ nợ, trong khi thực tế có 1 327 người. Nó quá "rụt rè" khi gắn nhãn vỡ nợ.

Accuracy không cho ta biết mô hình sai *ở đâu*. Ta cần một công cụ chi tiết hơn.

1.5.3 Bước 16. Ma trận nhầm lẫn

Mục đích: đếm riêng từng loại đúng và từng loại sai của mô hình.

Ô lệnh 16 · Ma trận nhầm lẫn của hồi quy logistic code/ch01/s16_confusion.py

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, pred_lr)
print(pd.DataFrame(cm, index=["Thực tế 0", "Thực tế 1"],
                   columns=["Dự đoán 0", "Dự đoán 1"]))

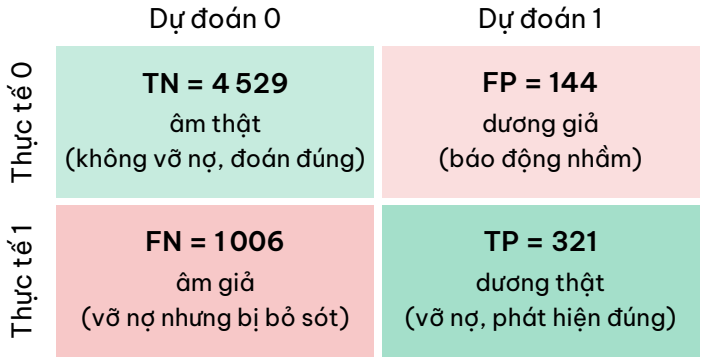
TN, FP, FN, TP = cm.ravel()       # "trải phẳng" ma trận 2×2 thành 4 con số
print(f"\nTN = {TN}, FP = {FP}, FN = {FN}, TP = {TP}")
```

Kết quả

```
           Dự đoán 0  Dự đoán 1
Thực tế 0       4529        144
Thực tế 1       1006        321

TN = 4529, FP = 144, FN = 1006, TP = 321
```

Mỗi dự đoán rơi vào đúng một trong bốn ô (Hình 1.6):



Hình 1.6: Ma trận nhầm lẫn của hồi quy logistic trên tập test (ngưỡng 0,5). Ô xanh: đúng; ô đỏ: sai.

Đọc kết quả. Cách nhớ tên bốn ô: chữ thứ hai (**P**ositive/**N**egative) là *điều mô hình dự đoán*; chữ thứ nhất (**T**rue/**F**alse) cho biết dự đoán đó *đúng hay sai*. Ví dụ, FN (*False Negative*) là "dự đoán âm (không vỡ nợ), và dự đoán đó sai".

Kiểm tra lại:

- Tổng bốn ô: $4\,529 + 144 + 1\,006 + 321 = 6\,000$, đúng bằng số khách hàng test.
- Hàng dưới cộng lại: $1\,006 + 321 = 1\,327$, đúng bằng số người thực sự vỡ nợ.
- Cột phải cộng lại: $144 + 321 = 465$, đúng bằng số người bị dự đoán vỡ nợ ở Bước 15.

Điều đáng lo nhất là ô FN: **1 006 người vỡ nợ bị bỏ sót**, nhiều gấp 3 lần số người được phát hiện đúng (321). Với ngân hàng, mỗi người bị bỏ sót có thể là một khoản nợ xấu.

1.5.4 Bước 17. Precision, Recall và F1 – tự tay tính

Mục đích: tóm tắt ma trận nhầm lẫn thành vài con số dễ so sánh, và hiểu chính xác mỗi con số đo điều gì.

Ba chỉ số quan trọng nhất, mỗi chỉ số trả lời một câu hỏi khác nhau:

Precision, Recall, F1

Precision (độ chuẩn xác): *Trong những người bị mô hình báo là vỡ nợ, bao nhiêu phần trăm thực sự vỡ nợ?*

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{321}{321 + 144} = \frac{321}{465} = 0{,}6903.$$

Recall (độ phủ): *Trong những người thực sự vỡ nợ, mô hình phát hiện được bao nhiêu phần trăm?*

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{321}{321 + 1\,006} = \frac{321}{1\,327} = 0{,}2419.$$

F1: trung bình điều hòa của Precision và Recall, một con số duy nhất cân bằng cả hai:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 0{,}6903 \times 0{,}2419}{0{,}6903 + 0{,}2419} = \frac{0{,}3340}{0{,}9322} = 0{,}3583.$$

Bây giờ để Python tính lại và đối chiếu:

Ô lệnh 17 · Tính Precision, Recall, F1 bằng tay rồi đối chiếu với scikit-learn `code/ch01/s17_tinh_tay.py`

```
from sklearn.metrics import precision_score, recall_score, f1_score

precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * precision * recall / (precision + recall)

print(f"Tính tay    : Precision = {precision:.4f}, Recall = {recall:.4f}, F1 =
↪  {f1:.4f}")
print(f"scikit-learn: Precision = {precision_score(y_test, pred_lr):.4f}, "
      f"Recall = {recall_score(y_test, pred_lr):.4f}, F1 = {f1_score(y_test,
      ↪  pred_lr):.4f}")
```

Kết quả

```
Tính tay    : Precision = 0.6903, Recall = 0.2419, F1 = 0.3583
scikit-learn: Precision = 0.6903, Recall = 0.2419, F1 = 0.3583
```

Đọc kết quả. Kết quả tính tay khớp hoàn toàn với scikit-learn. Diễn giải bằng lời:

- **Precision** $= 69\%$: khi mô hình "báo động", nó đúng khoảng 7 trên 10 lần. Khá tốt.
- **Recall** $= 24\%$: mô hình chỉ phát hiện được khoảng 1 trong 4 người vỡ nợ. Rất kém.
- **F1** $= 0{,}36$: thấp, vì F1 bị kéo xuống bởi chỉ số yếu hơn (Recall).

Vì sao F1 dùng trung bình điều hòa, không dùng trung bình cộng?

Trung bình cộng của 0,69 và 0,24 là 0,47, trông "tạm được". Trung bình điều hòa phạt nặng khi một trong hai chỉ số thấp. Một mô hình có Precision $= 1{,}0$ nhưng Recall $= 0{,}01$ (chỉ bắt được 1 người nhưng bắt đúng) có trung bình cộng 0,505, nhưng F1 chỉ là 0,02. F1 cao chỉ khi *cả hai* chỉ số đều cao.

Tự kiểm tra

Câu 2. Một bệnh viện dùng mô hình sàng lọc ung thư. Họ nên ưu tiên Precision cao hay Recall cao? Vì sao?

1.5.5 Bước 18. Xác suất dự đoán và ngưỡng quyết định

Mục đích: hiểu rằng mô hình thật ra đưa ra *xác suất*, còn nhãn 0/1 chỉ là kết quả của việc so xác suất với một *ngưỡng*.

Ô lệnh 18 · Xác suất vỡ nợ mà mô hình gán cho 8 khách hàng đầu tiên của tập test

code/ch01/s18_xac_suat.py

```
proba_lr = logreg.predict_proba(X_test)[:, 1]   # cột 1 = xác suất thuộc lớp 1 (vỡ nợ)

view = pd.DataFrame({"Xác suất vỡ nợ": proba_lr[:8],
                     "Dự đoán (ngưỡng 0,5)": pred_lr[:8],
                     "Thực tế": y_test.values[:8]})
print(view)
```

Kết quả

```
   Xác suất vỡ nợ  Dự đoán (ngưỡng 0,5)  Thực tế
0           0.108                     0        0
1           0.118                     0        0
2           0.220                     0        0
3           0.096                     0        1
4           0.028                     0        0
5           0.202                     0        0
6           0.131                     0        0
7           0.153                     0        0
```

Đọc kết quả. `predict_proba` trả về một bảng hai cột: cột 0 là xác suất thuộc lớp 0, cột 1 là xác suất thuộc lớp 1, và hai cột cộng lại bằng 1. Ta lấy cột 1 bằng `[:, 1]`.

Hãy nhìn khách hàng ở dòng 3: mô hình cho xác suất vỡ nợ chỉ 0,096, nhưng thực tế người này *đã vỡ nợ*. Ngược lại, khách hàng ở dòng 2 có xác suất 0,220, *cao nhất* trong 8 người, nhưng không vỡ nợ. Không mô hình nào hoàn hảo.

Điều quan trọng hơn: cả 8 xác suất đều dưới 0,5. Ngưỡng 0,5 chỉ là *mặc định*, không phải quy luật. Ngân hàng hoàn toàn có thể quyết định: "cứ ai có xác suất vỡ nợ từ 0,3 trở lên thì đưa vào danh sách theo dõi". Hãy xem điều gì xảy ra khi ta hạ ngưỡng:

Ô lệnh 19 · Cùng một mô hình, đổi ngưỡng quyết định thì kết quả thay đổi thế nào?

code/ch01/s19_nguong.py

```
rows = []
for t in [0.5, 0.4, 0.3, 0.2]:
    pred_t = (proba_lr >= t).astype(int)          # 1 nếu xác suất ≥ ngưỡng t
    rows.append({"Ngưỡng": t,
                 "Số bị gắn cờ": pred_t.sum(),
                 "Precision": precision_score(y_test, pred_t),
                 "Recall": recall_score(y_test, pred_t),
                 "F1": f1_score(y_test, pred_t)})
print(pd.DataFrame(rows).to_string(index=False))
```

Kết quả

```
 Ngưỡng  Số bị gắn cờ  Precision  Recall    F1
    0.5           465      0.690   0.242 0.358
    0.4           789      0.636   0.378 0.474
    0.3          1104      0.553   0.460 0.502
    0.2          2818      0.319   0.678 0.434
```

Đọc kết quả. Đây là **cùng một mô hình**, chỉ thay đổi ngưỡng t:

- Hạ ngưỡng từ 0,5 xuống 0,3: số người bị "gắn cờ" tăng từ 465 lên 1 104. Recall tăng từ 0,24 lên 0,46, nhưng Precision giảm từ 0,69 xuống 0,55.
- Hạ tiếp xuống 0,2: Recall lên 0,68, nhưng Precision chỉ còn 0,32. Hai phần ba số người bị gắn cờ là báo động nhầm.
- F1 cao nhất (0,502) ở ngưỡng 0,3 trong bốn ngưỡng đã thử.

Precision và Recall luôn "kéo co" với nhau

Hạ ngưỡng thì bắt được nhiều người vỡ nợ hơn (Recall tăng) nhưng cũng báo động nhầm nhiều hơn (Precision giảm). Nâng ngưỡng thì ngược lại. Không có ngưỡng nào "đúng" cho mọi trường hợp: ngưỡng tốt nhất phụ thuộc vào *chi phí* của từng loại sai lầm. Bỏ sót một khoản nợ xấu đắt hơn hay gọi điện nhắc nhầm một khách hàng tốt đắt hơn? Chương 5 sẽ cho cách tính ngưỡng tối ưu từ chi phí.

1.5.6 Bước 19. ROC-AUC và PR-AUC – chấm điểm trên mọi ngưỡng

Mục đích: có một con số đánh giá mô hình *không phụ thuộc* vào ngưỡng ta chọn, để so sánh các mô hình một cách công bằng.

Ý tưởng: thay vì chọn một ngưỡng, ta thử *tất cả* các ngưỡng từ 1 về 0 và vẽ lại kết quả thành đường cong.

- **Đường ROC**: trục hoành là tỷ lệ báo động nhầm FPR $= FP/(FP+TN)$, trục tung là Recall. **ROC-AUC** là diện tích dưới đường này.
- **Đường Precision–Recall (PR)**: trục hoành là Recall, trục tung là Precision. **PR-AUC** là diện tích dưới đường này (scikit-learn tính bằng hàm `average_precision_score`).

Ô lệnh 20 · ROC-AUC và PR-AUC: đánh giá mô hình trên MỌI ngưỡng cùng lúc `code/ch01/s20_auc.py`

```
from sklearn.metrics import (roc_auc_score, average_precision_score,
                             roc_curve, precision_recall_curve)

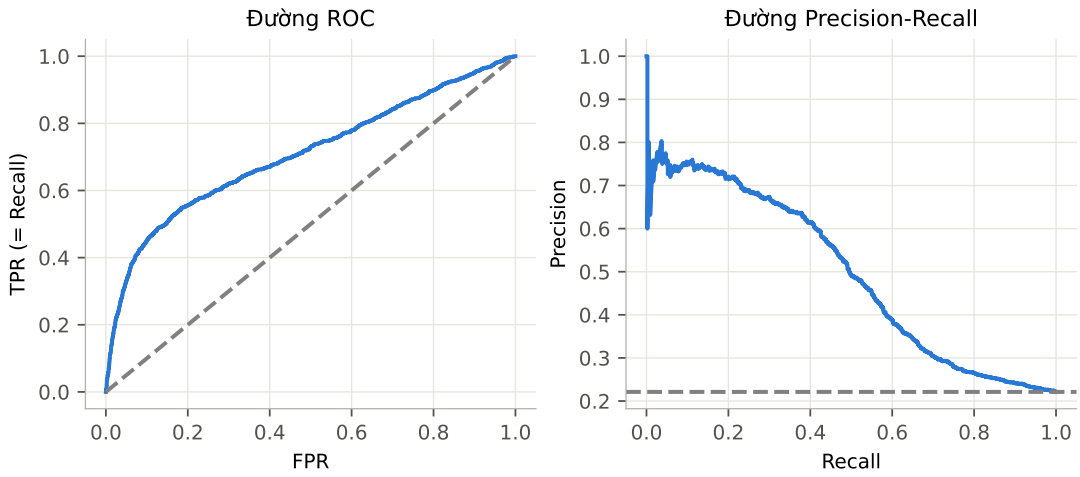
print("ROC-AUC:", round(roc_auc_score(y_test, proba_lr), 4), "  (đoán ngẫu nhiên:
↪  0,5)")
print("PR-AUC :", round(average_precision_score(y_test, proba_lr), 4),
      f"  (đoán ngẫu nhiên: {y_test.mean():.3f})")

fpr, tpr, _ = roc_curve(y_test, proba_lr)
prec, rec, _ = precision_recall_curve(y_test, proba_lr)
fig, ax = plt.subplots(1, 2, figsize=(8, 3))
ax[0].plot(fpr, tpr)
```

```
ax[0].plot([0, 1], [0, 1], "--", color="gray")
ax[0].set_xlabel("FPR"); ax[0].set_ylabel("TPR (= Recall)"); ax[0].set_title("Đường
↪  ROC")
ax[1].plot(rec, prec)
ax[1].axhline(y_test.mean(), ls="--", color="gray")
ax[1].set_xlabel("Recall"); ax[1].set_ylabel("Precision"); ax[1].set_title("Đường
↪  Precision-Recall")
plt.savefig("figures/ch01_s20_curves.pdf", bbox_inches="tight")
```

Kết quả

```
ROC-AUC: 0.7079   (đoán ngẫu nhiên: 0,5)
PR-AUC : 0.4942   (đoán ngẫu nhiên: 0.221)
```



Hình 1.7: Đường ROC (trái) và đường Precision–Recall (phải) của hồi quy logistic trên tập test. Đường nét đứt là mức của một mô hình đoán ngẫu nhiên.

Đọc kết quả.

- **ROC-AUC** $= 0{,}708$**.** Chỉ số này có một cách hiểu rất trực quan: *chọn ngẫu nhiên một người vỡ nợ và một người không vỡ nợ, xác suất để mô hình cho người vỡ nợ điểm rủi ro cao hơn là* $70{,}8\%$. Đoán ngẫu nhiên được 50%; mô hình hoàn hảo được 100%.
- **PR-AUC** $= 0{,}494$**.** Mốc so sánh của PR-AUC *không phải* $0{,}5$, mà là **tỷ lệ lớp dương** $(0{,}221)$: một mô hình đoán ngẫu nhiên có Precision bằng đúng tỷ lệ người vỡ nợ, ở mọi mức Recall (đường nét đứt nằm ngang ở hình phải). Vậy $0{,}494$ là gấp hơn 2 lần mức ngẫu nhiên.

Nên báo cáo chỉ số nào?

Với dữ liệu mất cân bằng, hãy báo cáo **PR-AUC** và **ROC-AUC** (không phụ thuộc ngưỡng), kèm **F1, Precision, Recall** tại ngưỡng đã chọn. PR-AUC phân biệt các mô hình rõ hơn ROC-AUC khi lớp dương hiếm [20]. Tránh chỉ báo cáo Accuracy.

Tự kiểm tra

Câu 3. Mô hình A có ROC-AUC $= 0{,}80$, mô hình B có ROC-AUC $= 0{,}75$. Có chắc mô hình A cho F1 cao hơn ở ngưỡng $0{,}5$ không?

1.6 Giai đoạn 5 – Đánh giá chéo

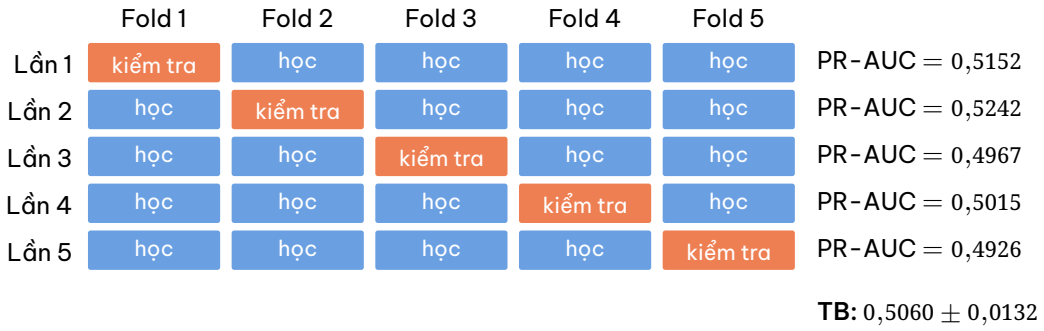
1.6.1 Bước 20. Vì sao một lần chia là chưa đủ? Đánh giá chéo k-fold

Mục đích: có một ước lượng hiệu năng *ổn định* hơn, và *không* cần đụng đến tập test.

Đến giờ ta đã đánh giá mô hình trên tập test. Nhưng ở Giai đoạn 6, ta muốn *so sánh* nhiều mô hình để chọn mô hình tốt nhất. Nếu dùng tập test để chọn, ta vi phạm nguyên tắc vàng ở Bước 13. Ngoài ra, kết quả trên *một* lần chia có thể may hoặc rủi tùy vào những khách hàng nào tình cờ rơi vào tập test.

Giải pháp là **đánh giá chéo k-fold** (*k-fold cross-validation*), thực hiện *chỉ trên tập train*:

1. Chia tập train (24 000 người) thành $k = 5$ phần bằng nhau, gọi là 5 **fold**, mỗi fold 4 800 người.
2. Lần lượt lấy từng fold làm "đề kiểm tra nhỏ", học trên 4 fold còn lại (19 200 người).
3. Được 5 điểm số. Lấy trung bình và độ lệch chuẩn.



Hình 1.8: Đánh giá chéo 5-fold trên tập train. Mỗi lần, một fold (cam) dùng để kiểm tra, bốn fold còn lại (xanh) dùng để học. Điểm số là kết quả thật của Ô lệnh 21.

Để thấy rõ từng bước, ta viết vòng lặp bằng tay:

Ô lệnh 21 · Đánh giá chéo 5-fold, viết vòng lặp "bằng tay" để thấy rõ từng bước

code/ch01/s21_cv_tay.py

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
scores = []
for k, (idx_tr, idx_va) in enumerate(skf.split(X_train, y_train), start=1):
    model = clone(logreg)                       # bản sao "mới tinh", chưa học gì
    model.fit(X_train.iloc[idx_tr], y_train.iloc[idx_tr])
    p = model.predict_proba(X_train.iloc[idx_va])[:, 1]
    s = average_precision_score(y_train.iloc[idx_va], p)
    scores.append(s)
    print(f"Fold {k}: học {len(idx_tr)} mẫu, kiểm tra {len(idx_va)} mẫu -> PR-AUC =
    ↪  {s:.4f}")

print(f"\nTrung bình ± độ lệch chuẩn: {np.mean(scores):.4f} ± {np.std(scores,
↪  ddof=1):.4f}")
```

Kết quả

```
Fold 1: học 19200 mẫu, kiểm tra 4800 mẫu -> PR-AUC = 0.5152
Fold 2: học 19200 mẫu, kiểm tra 4800 mẫu -> PR-AUC = 0.5242
Fold 3: học 19200 mẫu, kiểm tra 4800 mẫu -> PR-AUC = 0.4967
Fold 4: học 19200 mẫu, kiểm tra 4800 mẫu -> PR-AUC = 0.5015
Fold 5: học 19200 mẫu, kiểm tra 4800 mẫu -> PR-AUC = 0.4926

Trung bình ± độ lệch chuẩn: 0.5060 ± 0.0132
```

Đọc kết quả.

- `StratifiedKFold(n_splits=5, shuffle=True, ...)`: chia thành 5 fold, xáo trộn trước khi chia, và phân tầng (mỗi fold đều có khoảng 22% người vỡ nợ).
- `skf.split(X_train, y_train)` lần lượt trả về hai danh sách *chỉ số dòng*: `idx_tr` (dòng dùng để học) và `idx_va` (dòng dùng để kiểm tra).
- `clone(logreg)` tạo một bản sao mới, **chưa học gì**, cho mỗi lần. Nếu dùng lại cùng một mô hình, nó sẽ "nhớ" những gì đã học ở lần trước.

Tính trung bình bằng tay:

$$\frac{0{,}5152 + 0{,}5242 + 0{,}4967 + 0{,}5015 + 0{,}4926}{5} = \frac{2{,}5302}{5} = 0{,}5060.$$

Độ lệch chuẩn 0,0132 cho biết điểm số dao động bao nhiêu giữa các fold. Năm điểm nằm trong khoảng 0,49 đến 0,52: chỉ vì *khách hàng khác nhau* rơi vào phần kiểm tra mà điểm đã dao động khoảng $\pm 0{,}015$. Đây là thông tin rất quan trọng: nếu hai mô hình chỉ chênh nhau 0,005, chênh lệch đó nhỏ hơn mức dao động tự nhiên, và có thể chỉ là may rủi.

1.6.2 Bước 21. Viết gọn bằng `cross_validate` và vì sao cần pipeline

Mục đích: làm lại Bước 20 bằng một dòng lệnh, tính nhiều chỉ số cùng lúc.

Ô lệnh 22 · Viết gọn bằng cross_validate và tính nhiều chỉ số một lúc `code/ch01/s22_cv_nhanh.py`

```python
from sklearn.model_selection import cross_validate

scoring = {"ROC-AUC": "roc_auc", "PR-AUC": "average_precision",
           "F1": "f1", "Recall": "recall"}
res = cross_validate(logreg, X_train, y_train, cv=skf, scoring=scoring)

for name in scoring:
    s = res["test_" + name]                    # mảng 5 điểm số, mỗi fold một điểm
    print(f"{name:8s}: {s.mean():.4f} ± {s.std(ddof=1):.4f}")
```

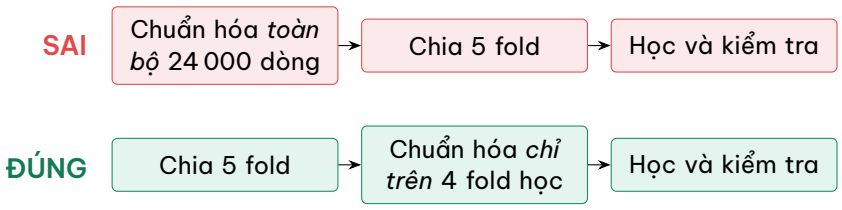
Kết quả

```
ROC-AUC : 0.7260 ± 0.0122
PR-AUC  : 0.5060 ± 0.0132
F1      : 0.3662 ± 0.0168
Recall  : 0.2464 ± 0.0135
```

Đọc kết quả. Dòng PR-AUC ($0{,}5060 \pm 0{,}0132$) trùng khớp hoàn toàn với vòng lặp bằng tay ở Bước 20: `cross_validate` làm đúng những việc đó. Tham số `scoring` là một từ điển: khóa là tên

ta tự đặt, giá trị là tên chỉ số theo quy ước của scikit-learn (`"roc_auc"`, `"average_precision"`, `"f1"`, `"recall"`).

Vì sao chuẩn hóa phải nằm *trong* pipeline? Hãy so sánh hai cách làm:



Ở cách **sai**, trung bình và độ lệch chuẩn được tính trên toàn bộ dữ liệu, *kể cả* fold dùng để kiểm tra. Nghĩa là mô hình đã "nhìn trộm" một chút thông tin từ đề thi. Hiện tượng này gọi là **rò rỉ dữ liệu** (*data leakage*) [12].

Với phép chuẩn hóa, lượng thông tin rò rỉ rất nhỏ nên kết quả gần như không đổi. Nhưng với những kỹ thuật mạnh hơn, rò rỉ có thể làm kết quả sai lệch rất nghiêm trọng: ở Chương 2, ta sẽ thấy PR-AUC bị thổi phồng từ 0,54 lên 0,93 chỉ vì đặt kỹ thuật SMOTE sai chỗ. Vì vậy, hãy tập thói quen ngay từ đầu: **mọi bước có "học" từ dữ liệu đều phải nằm trong pipeline**.

1.7 Giai đoạn 6 – So sánh nhiều mô hình

1.7.1 Bước 22. So sánh bốn mô hình bằng cùng một quy trình

Mục đích: xem các mô hình mạnh hơn có cải thiện đáng kể so với hồi quy logistic không.

Ta thêm ba mô hình thuộc họ **cây quyết định**. Ý tưởng của chúng, nói ngắn gọn:

- **Cây quyết định** đặt một chuỗi câu hỏi có/không. Ví dụ: "`PAY_1` có $\geq$ 2 không?"; nếu có, hỏi tiếp "`LIMIT_BAL` có $<$ 50 000 không?"; và cứ thế cho đến khi đưa ra dự đoán. Nó bắt được các quan hệ phi tuyến và các "kết hợp" giữa đặc trưng, điều mà hồi quy logistic không làm được.
- **Random Forest** [1]: trồng 200 cây, mỗi cây học trên một mẫu ngẫu nhiên của dữ liệu, rồi lấy trung bình dự đoán của cả "rừng".
- **XGBoost** [5] và **LightGBM** [13]: trồng cây *nối tiếp nhau*, mỗi cây mới tập trung sửa lỗi của các cây trước. Kỹ thuật này gọi là *boosting*.

Các tham số trong ngoặc (số cây, độ sâu, tốc độ học, ...) gọi là **siêu tham số**. Ở đây ta dùng những giá trị hợp lý thường gặp; Phần IV của giáo trình sẽ hướng dẫn cách tinh chỉnh chúng.

Ô lệnh 23 · So sánh bốn mô hình bằng cùng một quy trình đánh giá chéo

`code/ch01/s23_bon_mo_hinh.py`

```
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

models = {
    "LogReg": logreg,
```

```
    "RandomForest": RandomForestClassifier(n_estimators=200, min_samples_leaf=5,
                                           n_jobs=2, random_state=SEED),
    "XGBoost": XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=4,
                             subsample=0.8, colsample_bytree=0.8,
                             eval_metric="logloss", n_jobs=2, random_state=SEED),
    "LightGBM": LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31,
                               subsample=0.8, subsample_freq=1, colsample_bytree=0.8,
                               n_jobs=2, random_state=SEED, verbose=-1),
}
table = {}
for name, m in models.items():
    r = cross_validate(m, X_train, y_train, cv=skf, scoring=scoring)
    table[name] = {k: f"{r['test_' + k].mean():.3f} ± {r['test_' + k].std(ddof=1):.3f}"
                   for k in scoring}
print(pd.DataFrame(table).T)
```

Kết quả

	ROC-AUC	PR-AUC	F1	Recall
LogReg	0.726 ± 0.012	0.506 ± 0.013	0.366 ± 0.017	0.246 ± 0.014
RandomForest	0.780 ± 0.005	0.556 ± 0.009	0.472 ± 0.010	0.363 ± 0.008
XGBoost	0.783 ± 0.007	0.558 ± 0.009	0.476 ± 0.010	0.369 ± 0.008
LightGBM	0.780 ± 0.008	0.558 ± 0.009	0.477 ± 0.008	0.371 ± 0.008

Đọc kết quả. (Ô lệnh này mất khoảng nửa phút.)

- Ba mô hình cây **vượt trội** so với hồi quy logistic: ROC-AUC khoảng 0,78 so với 0,726, PR-AUC khoảng 0,556–0,558 so với 0,506. Điều này phù hợp với nhận định chung rằng mô hình cây rất mạnh trên dữ liệu bảng [8].
- Ba mô hình cây **gần như ngang nhau**: ROC-AUC chênh nhau chỉ 0,003, nhỏ hơn độ lệch chuẩn giữa các fold (0,005–0,008).
- Recall của mọi mô hình vẫn thấp (dưới 0,38) ở ngưỡng 0,5. Toàn bộ Phần II của giáo trình sẽ tìm cách cải thiện điều này.

Theo kết quả CV, ta chọn **XGBoost** làm mô hình chính cho các chương sau, vì nó có ROC-AUC và PR-AUC cao nhất (dù chỉ cao hơn rất ít).

1.7.2 Bước 23. Đánh giá một lần duy nhất trên tập test

Mục đích: kiểm tra xem kết luận từ CV có đúng trên dữ liệu hoàn toàn mới không.

Đây là lần *đầu tiên và duy nhất* ta dùng tập test cho việc so sánh. Mọi lựa chọn đã được quyết định xong ở Bước 22.

Ô lệnh 24 · Đánh giá MỘT LẦN trên tập test (sau khi đã so sánh xong bằng CV)

code/ch01/s24_test.py

```
rows = {}
for name, m in models.items():
    m.fit(X_train, y_train)                    # học lại trên toàn bộ tập train
    p = m.predict_proba(X_test)[:, 1]
    rows[name] = {"ROC-AUC": roc_auc_score(y_test, p),
```

```
                  "PR-AUC": average_precision_score(y_test, p),
                  "F1": f1_score(y_test, (p ⩾ 0.5).astype(int)),
                  "Recall": recall_score(y_test, (p ⩾ 0.5).astype(int))}
print(pd.DataFrame(rows).T)
```

Kết quả

```
              ROC-AUC  PR-AUC     F1  Recall
LogReg          0.708   0.494  0.358   0.242
RandomForest    0.768   0.550  0.458   0.353
XGBoost         0.778   0.556  0.467   0.361
LightGBM        0.776   0.550  0.467   0.363
```

Đọc kết quả. So sánh với kết quả CV ở Bước 22:

XGBoost	5-fold CV (tập train)	Tập test
ROC-AUC	0,783 ± 0,007	0,778
PR-AUC	0,558 ± 0,009	0,556

Kết quả trên tập test nằm trong khoảng dao động của CV. Đây là dấu hiệu tốt: quy trình đánh giá của ta đáng tin cậy, không có rò rỉ dữ liệu, và mô hình không "học thuộc" tập train. Thứ hạng của bốn mô hình trên test cũng giống với CV: XGBoost dẫn đầu, LogReg cuối cùng.

1.7.3 Bước 24. Chênh lệch có phải do may rủi? Kiểm định thống kê

Mục đích: trả lời câu hỏi mà mọi người phản biện bài báo đều hỏi: "mô hình A tốt hơn B thật, hay chỉ do may mắn?"

Ý tưởng: lặp lại đánh giá chéo 5-fold **3 lần**, mỗi lần chia ngẫu nhiên theo một cách khác, được 15 cặp điểm số. Nếu mô hình A thật sự tốt hơn B, A phải thắng B ở *phần lớn* trong 15 lần. **Kiểm định Wilcoxon** [6] tính xác suất p để thấy một mức thắng áp đảo như vậy *nếu hai mô hình thật ra ngang nhau.* Nếu p rất nhỏ (thường dưới 0,05), ta kết luận chênh lệch *có ý nghĩa thống kê.*

Ô lệnh 25 · Chênh lệch giữa hai mô hình có phải do may rủi? Kiểm định Wilcoxon

code/ch01/s25_wilcoxon.py

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from scipy.stats import wilcoxon

rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=SEED)  # 15 lần
↪ chia
auc = {name: cross_val_score(models[name], X_train, y_train, cv=rcv, scoring="roc_auc")
       for name in ["LogReg", "XGBoost", "LightGBM"]}
for name, s in auc.items():
    print(f"{name:8s}: ROC-AUC = {s.mean():.4f} ± {s.std(ddof=1):.4f}")
print()
for a, b in [("XGBoost", "LogReg"), ("XGBoost", "LightGBM")]:
    d = auc[a] - auc[b]
    p = wilcoxon(auc[a], auc[b]).pvalue
    print(f"{a} - {b}: chênh lệch TB = {d.mean():+.4f}, thắng {np.sum(d > 0)}/15 lần,"
          f" p = {p:.1e}")
```

Kết quả

```
LogReg  : ROC-AUC = 0.7261 ± 0.0099
XGBoost : ROC-AUC = 0.7831 ± 0.0070
LightGBM: ROC-AUC = 0.7798 ± 0.0072

XGBoost - LogReg: chênh lệch TB = +0.0570, thắng 15/15 lần, p = 6.1e-05
XGBoost - LightGBM: chênh lệch TB = +0.0034, thắng 14/15 lần, p = 1.8e-04
```

Đọc kết quả.

- **XGBoost với LogReg**: XGBoost thắng *cả 15 lần*, trung bình hơn 0,057 ROC-AUC. $p = 6{,}1 \times 10^{-5}$ là giá trị nhỏ nhất có thể có với 15 cặp. Kết luận chắc chắn: XGBoost tốt hơn.
- **XGBoost với LightGBM**: XGBoost thắng 14/15 lần, $p = 1{,}8 \times 10^{-4}$, cũng "có ý nghĩa thống kê". Nhưng chênh lệch chỉ là 0,0034 ROC-AUC, một mức gần như không có ý nghĩa trong thực tế.

"Có ý nghĩa thống kê" khác với "quan trọng"

Hai điều cần nhớ:

1. p nhỏ chỉ cho biết chênh lệch *có thật*, không cho biết chênh lệch *lớn*. Chênh 0,003 ROC-AUC hầu như không thay đổi quyết định cho vay nào.
2. 15 lần chia dùng chung dữ liệu nên *không độc lập* hoàn toàn với nhau, khiến p thường nhỏ hơn thực tế [18].

Vì vậy, khi viết bài, hãy luôn báo cáo *độ lớn* của chênh lệch bên cạnh p. Kết luận trung thực cho cặp thứ hai là: *XGBoost và LightGBM tương đương nhau về mặt thực tiễn.*

1.8 Giai đoạn 7 – Làm lại với bài toán hồi quy

Bộ dữ liệu thứ hai là **California Housing** [19], lấy từ điều tra dân số Hoa Kỳ năm 1990. Mỗi dòng là một *khu dân cư*, và ta cần dự đoán **giá nhà trung vị** của khu đó, một con số liên tục. Bạn sẽ thấy quy trình gần như giống hệt bài toán phân loại; chỉ có *chỉ số đánh giá* là khác.

1.8.1 Bước 25. Đọc dữ liệu và phát hiện mức chặn trên

Mục đích: làm quen với bộ dữ liệu thứ hai và phát hiện một đặc điểm quan trọng của biến mục tiêu.

Ô lệnh 26 · Bộ dữ liệu thứ hai: California Housing (có sẵn trong scikit-learn)

code/ch01/s26_housing.py

```python
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)   # lần đầu sẽ tự tải về máy
Xh, yh = housing.data, housing.target

print("Kích thước:", Xh.shape)
print(Xh.head(3).round(2).T)        # .T: xoay bảng cho gọn
print()
```

```
print(yh.describe().round(3))
```

Kết quả

```
Kích thước: (20640, 8)
                0        1       2
MedInc        8.33     8.30    7.26
HouseAge     41.00    21.00   52.00
AveRooms      6.98     6.24    8.29
AveBedrms     1.02     0.97    1.07
Population  322.00  2401.00  496.00
AveOccup      2.56     2.11    2.80
Latitude     37.88    37.86   37.85
Longitude  -122.23  -122.22 -122.24

count    20640.000
mean         2.069
std          1.154
min          0.150
25%          1.196
50%          1.797
75%          2.647
max          5.000
Name: MedHouseVal, dtype: float64
```

Đọc kết quả. Bộ dữ liệu có sẵn trong scikit-learn; lần đầu chạy, hàm `fetch_california_housing` sẽ tự tải về máy (cần có Internet). Có 20 640 khu dân cư và 8 đặc trưng:

- `MedInc`: thu nhập trung vị của khu (đơn vị: chục nghìn USD);
- `HouseAge`: tuổi nhà trung vị;
- `AveRooms`, `AveBedrms`: số phòng và số phòng ngủ trung bình mỗi hộ;
- `Population`: dân số của khu; `AveOccup`: số người trung bình mỗi hộ;
- `Latitude`, `Longitude`: vĩ độ và kinh độ.

Biến mục tiêu `MedHouseVal` tính theo đơn vị *trăm nghìn USD*: giá trị 2,069 nghĩa là khoảng 206 900 USD. Điều đáng chú ý là giá trị lớn nhất đúng bằng 5,000, một con số tròn đáng ngờ. Hãy kiểm tra:

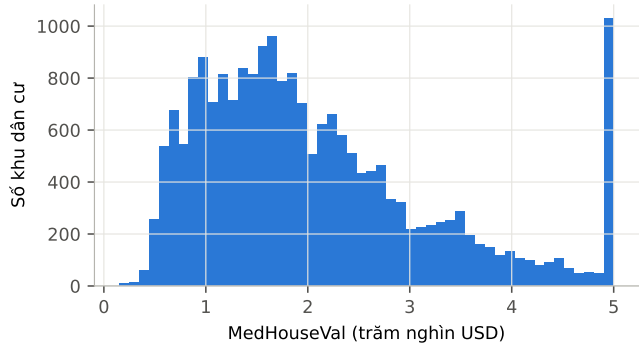
Ô lệnh 27 · Có bao nhiêu khu dân cư có giá bị "chặn trên"? `code/ch01/s27_muc_chan.py`

```
n_cap = (yh >= 5.0).sum()
print(f"Số khu có giá >= 5,0: {n_cap} ({n_cap / len(yh):.1%})")
print("Giá trị lớn nhất    :", yh.max())

fig, ax = plt.subplots(figsize=(5, 2.6))
ax.hist(yh, bins=50, color="#2a78d6")
ax.set_xlabel("MedHouseVal (trăm nghìn USD)")
ax.set_ylabel("Số khu dân cư")
plt.savefig("figures/ch01_s27_hist.pdf", bbox_inches="tight")
```

Kết quả

```
Số khu có giá ⩾ 5,0: 992 (4.8%)
Giá trị lớn nhất    : 5.00001
```



Hình 1.9: Phân bố giá nhà. Cột cao bất thường ở mép phải là 992 khu bị "chặn" ở mức 5,0.

Đọc kết quả. Có 992 khu (4,8%) có giá đúng bằng 5,00001. Khi thu thập dữ liệu, mọi khu có giá trên 500 000 USD đều bị ghi là 500 001. Ta gọi đây là **dữ liệu bị chặn trên** (*censored*). Hệ quả: mô hình không thể biết một khu "đắt" thực ra đắt đến mức nào, và sai số ở nhóm giá cao sẽ lớn bất thường. Hãy ghi nhớ đặc điểm này; nó sẽ xuất hiện lại ở Chương 19 and 26.

1.8.2 Bước 26. Chỉ số đánh giá cho hồi quy – tự tay tính

Mục đích: hiểu ba chỉ số đo sai số của bài toán hồi quy qua một ví dụ nhỏ.

Với hồi quy, không có "đúng/sai" như phân loại, mà chỉ có *sai lệch bao nhiêu*. Gọi y_i là giá thật, $\hat{y}_i$ là giá dự đoán, $e_i = y_i - \hat{y}_i$ là sai số của căn nhà thứ i, và n là số căn nhà:

$$\text{MAE} = \frac{1}{n}\sum_i |e_i| \qquad \text{(sai số tuyệt đối trung bình)}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_i e_i^2} \qquad \text{(căn của sai số bình phương trung bình)}$$

$$R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2} \qquad \text{(tỷ lệ biến động được mô hình giải thích)}$$

Ô lệnh 28 · Tính MAE, RMSE và R² bằng tay trên ví dụ 4 căn nhà `code/ch01/s28_tinh_tay_hoi_quy.py`

```
y_true = np.array([2.0, 3.0, 1.5, 4.0])     # giá thật
y_pred = np.array([2.5, 2.5, 1.5, 3.0])     # giá mô hình dự đoán
err = y_true - y_pred

print("Sai số:", err)
print("MAE  =", np.mean(np.abs(err)))
print("RMSE =", np.sqrt(np.mean(err ** 2)).round(4))
print("R²   =", (1 - np.sum(err ** 2) / np.sum((y_true - y_true.mean()) **
↪  2)).round(4))
```

Kết quả

```
Sai số: [-0.5  0.5  0.   1. ]
MAE  = 0.5
RMSE = 0.6124
R²   = 0.5932
```

Đọc kết quả. Tính lại từng bước với 4 căn nhà, sai số $e = (-0{,}5;\ 0{,}5;\ 0;\ 1{,}0)$:

- MAE $= (0{,}5 + 0{,}5 + 0 + 1{,}0)/4 = 2{,}0/4 = 0{,}5$.
- RMSE $= \sqrt{(0{,}25 + 0{,}25 + 0 + 1{,}0)/4} = \sqrt{0{,}375} = 0{,}6124$. RMSE lớn hơn MAE vì việc *bình phương* làm sai số lớn (1,0) bị phạt nặng hơn. Hai sai số nhỏ 0,5 chỉ đóng góp 0,25 mỗi cái.
- R^2: giá trung bình $\bar{y} = 2{,}625$. Mẫu số $= (2 - 2{,}625)^2 + (3 - 2{,}625)^2 + (1{,}5 - 2{,}625)^2 + (4 - 2{,}625)^2 = 3{,}6875$. Vậy $R^2 = 1 - 1{,}5/3{,}6875 = 0{,}5932$: mô hình giải thích được khoảng 59% sự chênh lệch giá giữa các căn nhà.

Mốc so sánh: $R^2 = 0$ nghĩa là mô hình chỉ tốt bằng việc luôn đoán giá trung bình; $R^2 = 1$ là hoàn hảo.

1.8.3 Bước 27. So sánh mô hình hồi quy và đánh giá trên test

Mục đích: áp dụng đúng quy trình của Giai đoạn 3, 5, 6 cho bài toán hồi quy.

Ô lệnh 29 · Chia dữ liệu và so sánh ba mô hình hồi quy bằng 5-fold CV `code/ch01/s29_hoi_quy_cv.py`

```python
from sklearn.model_selection import KFold
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from lightgbm import LGBMRegressor

Xh_train, Xh_test, yh_train, yh_test = train_test_split(
    Xh, yh, test_size=0.2, random_state=SEED)
kf = KFold(n_splits=5, shuffle=True, random_state=SEED)

reg_models = {
    "Dummy (trung bình)": DummyRegressor(strategy="mean"),
    "LinearReg": make_pipeline(StandardScaler(), LinearRegression()),
    "LightGBM": LGBMRegressor(n_estimators=500, learning_rate=0.05, num_leaves=63,
                              subsample=0.8, subsample_freq=1, colsample_bytree=0.8,
                              n_jobs=2, random_state=SEED, verbose=-1),
}
reg_scoring = {"RMSE": "neg_root_mean_squared_error",
               "MAE": "neg_mean_absolute_error", "R2": "r2"}
tab = {}
for name, m in reg_models.items():
    r = cross_validate(m, Xh_train, yh_train, cv=kf, scoring=reg_scoring)
    tab[name] = {"RMSE": -r["test_RMSE"].mean(),   # đổi dấu: scikit-learn trả về số âm
                 "MAE": -r["test_MAE"].mean(),
                 "R2": r["test_R2"].mean()}
print(pd.DataFrame(tab).T.round(3))
```

Kết quả

```
                     RMSE    MAE     R2
Dummy (trung bình)  1.156  0.914 -0.000
LinearReg           0.721  0.529  0.611
LightGBM            0.445  0.289  0.852
```

Đọc kết quả. Ba điểm khác biệt nhỏ so với bài toán phân loại:

- Dùng `KFold` thay vì `StratifiedKFold`, vì không có "lớp" để phân tầng.
- Tên chỉ số bắt đầu bằng `neg_` (`"neg_root_mean_squared_error"`). scikit-learn quy ước "điểm càng cao càng tốt", nên nó trả về RMSE *mang dấu âm*. Ta đổi dấu lại bằng dấu trừ phía trước.
- Mô hình ngây thơ `DummyRegressor` luôn đoán giá trung bình, nên $R^2 \approx 0$, đúng như định nghĩa.

LightGBM giảm RMSE từ 1,156 (ngây thơ) và 0,721 (hồi quy tuyến tính) xuống 0,445, với $R^2 = 0{,}852$. Hồi quy tuyến tính kém hơn nhiều vì giá nhà phụ thuộc vào *vị trí* theo cách phi tuyến: khu ven biển gần San Francisco và Los Angeles đắt hơn hẳn khu nội địa.

Ô lệnh 30 · Đánh giá LightGBM trên tập test và xem vài dự đoán cụ thể `code/ch01/s30_hoi_quy_test.py`

```python
from sklearn.metrics import root_mean_squared_error, r2_score

lgb_reg = reg_models["LightGBM"].fit(Xh_train, yh_train)
pred_h = lgb_reg.predict(Xh_test)
print(f"Test RMSE = {root_mean_squared_error(yh_test, pred_h):.3f},"
      f"  R2 = {r2_score(yh_test, pred_h):.3f}")
print()
print(pd.DataFrame({"Giá thật": yh_test.values[:6], "Dự đoán": pred_h[:6],
                    "Sai số": yh_test.values[:6] - pred_h[:6]}).round(3))
```

Kết quả

```
Test RMSE = 0.429,  R2 = 0.859

   Giá thật  Dự đoán  Sai số
0     0.477    0.544  -0.067
1     0.458    0.863  -0.405
2     5.000    5.189  -0.189
3     2.186    2.453  -0.267
4     2.780    2.530   0.250
5     1.587    1.569   0.018
```

Đọc kết quả. Trên tập test, LightGBM đạt RMSE $= 0{,}429$ (khoảng 43 000 USD) và $R^2 = 0{,}859$, nhất quán với kết quả CV. Nhìn vào 6 căn nhà cụ thể:

- Có căn dự đoán rất sát (sai 0,018, tức chưa tới 2 000 USD).
- Có căn sai khá nhiều: căn thứ hai có giá thật 0,458 nhưng dự đoán 0,863, gần gấp đôi.
- Căn thứ ba có giá thật 5,000, tức thuộc nhóm bị chặn trên. Mô hình dự đoán 5,189, *vượt* mức chặn. Giá thật của khu này có lẽ còn cao hơn nữa, nhưng ta không bao giờ biết được.

Một mô hình chỉ đưa ra *một con số* mà không nói mình chắc chắn đến đâu là chưa đủ cho

những quyết định quan trọng. Đây là động lực của Phần VII (độ tin cậy của dự đoán).

1.9 Tổng kết: giao thức chuẩn của giáo trình

Toàn bộ 27 bước có thể gói gọn thành một **giao thức thực nghiệm** (Algorithm 1). Mọi chương sau đều tuân theo giao thức này; kỹ thuật mới của mỗi chương được "cắm" vào bước 4.

Thuật toán 1: Giao thức thực nghiệm chuẩn của giáo trình

Đầu vào: Dữ liệu $\mathcal{D}$; danh sách cấu hình $\mathcal{C}$ (mô hình + kỹ thuật); chỉ số M; số fold k
Đầu ra: Bảng kết quả CV (trung bình $\pm$ độ lệch chuẩn) và kết quả trên tập test

Chia $\mathcal{D}$ thành $\mathcal{D}_{\text{train}}$ (80%) và $\mathcal{D}_{\text{test}}$ (20%), phân tầng theo nhãn, `random_state`$= 42$; // Bước 13
Cất $\mathcal{D}_{\text{test}}$ đi, **không** dùng nó cho đến bước 9;
for *mỗi cấu hình* $c \in \mathcal{C}$ **do**
    Đóng gói tiền xử lý, kỹ thuật và mô hình thành **một** pipeline P_c ; // Bước 15, 21
    for $j = 1$ **to** k **do**
        Huấn luyện P_c trên các fold học; *mọi* phép `fit` chỉ thấy dữ liệu này;
        Tính $M_j(c)$ trên fold kiểm tra ; // Bước 20
Chọn cấu hình $c^\star$ theo trung bình CV; nếu cần, kiểm định thống kê ; // Bước 22, 24
Huấn luyện $P_{c^\star}$ trên toàn bộ $\mathcal{D}_{\text{train}}$, đánh giá **một lần** trên $\mathcal{D}_{\text{test}}$; // Bước 23

Module dùng chung cho các chương sau

Để không phải chép lại các bước nạp dữ liệu, chia dữ liệu, cấu hình mô hình ở mọi chương, giáo trình gom chúng vào file `code/common.py`. Ví dụ, toàn bộ Bước 5–9 và Bước 13 được gói vào hai hàm `load_credit()` và `credit_split()`:

Trích `code/common.py`

```python
def credit_split(test_size: float = 0.2):
    """Chia 80/20 có phân tầng (stratified), giống Bước 13 của Chương 1."""
    X, y = load_credit()
    return train_test_split(X, y, test_size=test_size, stratify=y,
                            random_state=RANDOM_STATE)
```

Từ Chương 2 trở đi, mỗi chương chỉ cần một dòng sau là có đúng dữ liệu đã chia như chương này:

```
X_train, X_test, y_train, y_test = credit_split()
```

Các hàm `xgb_clf()`, `lgbm_clf()` trả về mô hình với đúng cấu hình ở Bước 22. Nhờ vậy, kết quả giữa các chương so sánh trực tiếp được với nhau.

Bảng kết quả mốc cho cả giáo trình

Hai bảng dưới đây là "mốc" mà mọi kỹ thuật ở các chương sau sẽ được so sánh với.

Bảng 1.2: Kết quả 5-fold CV trên tập train của Credit Default (trung bình ± độ lệch chuẩn), giống Ô lệnh 23.

Mô hình	ROC-AUC	PR-AUC	F1	Recall
LogReg	0.726 ± 0.012	0.506 ± 0.013	0.366 ± 0.017	0.246 ± 0.014
RandomForest	0.780 ± 0.005	0.556 ± 0.009	0.472 ± 0.010	0.363 ± 0.008
XGBoost	0.783 ± 0.007	0.558 ± 0.009	0.476 ± 0.010	0.369 ± 0.008
LightGBM	0.780 ± 0.008	0.558 ± 0.009	0.477 ± 0.008	0.371 ± 0.008

Bảng 1.3: Kết quả hồi quy trên California Housing (thêm Random Forest và XGBoost để đối chiếu). Trên: 5-fold CV. Dưới: tập test.

Mô hình	RMSE	MAE	R2
LinearReg	0.721 ± 0.016	0.529 ± 0.010	0.611 ± 0.014
RandomForest	0.510 ± 0.013	0.333 ± 0.006	0.806 ± 0.007
XGBoost	0.452 ± 0.011	0.297 ± 0.007	0.847 ± 0.005
LightGBM	0.445 ± 0.013	0.289 ± 0.007	0.852 ± 0.006

Mô hình	RMSE	MAE	R2
LinearReg	0.746	0.533	0.576
RandomForest	0.504	0.326	0.806
XGBoost	0.444	0.290	0.850
LightGBM	**0.429**	**0.280**	**0.859**

1.10 Những lỗi thường gặp

Mười lỗi người mới hay mắc

1. **Chỉ báo cáo Accuracy** cho dữ liệu mất cân bằng. Mô hình ngây thơ vẫn đạt 77,9% (Bước 14).
2. **Dùng tập test để chọn mô hình hoặc chỉnh tham số.** Hãy dùng CV trên tập train (Bước 20–22).
3. **Chuẩn hóa hoặc xử lý dữ liệu trước khi chia.** Hãy đặt mọi bước vào pipeline (Bước 21).
4. **Quên `stratify=y`** khi chia dữ liệu mất cân bằng (Bước 13).
5. **Quên `random_state`**, khiến mỗi lần chạy ra một kết quả khác và không ai tái lập được.
6. **Chỉ báo cáo một con số**, không có độ lệch chuẩn (Bước 20).
7. **Nhầm "có ý nghĩa thống kê" với "quan trọng"** (Bước 24).
8. **Tin vào tỷ lệ tính trên nhóm quá nhỏ** (Bước 12).
9. **Bỏ qua các giá trị lạ**, như mã 0, 5, 6 của `EDUCATION` (Bước 9) hay mức chặn 5,0 của giá nhà (Bước 25).
10. **Đặt notebook sai thư mục**, gây lỗi `FileNotFoundError` (Bước 3).

1.11 Khi viết bài báo khoa học

Gợi ý cho bài báo

Mục *Experimental Setup* của một bài báo cần đủ chi tiết để người khác làm lại được đúng kết quả của bạn. Dựa vào chương này, mục đó nên có:

- **Dữ liệu**: nguồn kèm trích dẫn gốc, kích thước, tỷ lệ lớp, và mọi bước làm sạch ("EDUCATION codes 0, 5, 6 were merged into category 4").
- **Cách chia**: 80/20, phân tầng, seed; số fold CV và số lần lặp.
- **Mô hình**: tên, thư viện, và bảng siêu tham số (thường đặt ở phụ lục).
- **Chỉ số đánh giá kèm lý do chọn**: "We report PR-AUC because the positive class accounts for only 22% of samples [20]".
- **Mô hình mốc**: ít nhất một mô hình đơn giản như hồi quy logistic, và nếu được thì cả mô hình ngây thơ.
- **Kiểm định thống kê**, kèm độ lớn của chênh lệch.
- **Phiên bản thư viện** (Ô lệnh 1) và đường dẫn đến mã nguồn.

Người phản biện thường hỏi gì?

- *"Why not compare with a simple baseline?"* Mỗi bài báo nên có ít nhất một mô hình tuyến tính làm mốc.
- *"Are the improvements statistically significant?"* Hãy chuẩn bị bảng trung bình $\pm$ độ lệch chuẩn và kết quả kiểm định.
- *"Was the test set used for model selection?"* Hãy mô tả rõ giao thức như Algorithm 1.

1.12 Bài tập

Mức 1 – Làm quen.

1. Đếm số khách hàng nam và nữ trong tập train. Tỷ lệ vỡ nợ của hai nhóm là bao nhiêu? Dùng `groupby` giống Bước 12.
2. Vẽ histogram của `AGE` riêng cho nhóm vỡ nợ và nhóm không vỡ nợ, chồng lên nhau (gợi ý: tham số `alpha=0.5` trong `plt.hist`). Hai phân bố có khác nhau nhiều không?
3. Tính tay Accuracy của hồi quy logistic từ ma trận nhầm lẫn ở Bước 16, rồi so với kết quả của Bước 15.

Mức 2 – Vận dụng.

4. Lặp lại Bước 19 (ngưỡng) cho XGBoost. Ở ngưỡng nào F1 của XGBoost cao nhất? Chú ý: chọn ngưỡng bằng CV, không bằng tập test.
5. Chạy lại Bước 20 với 10 fold thay vì 5. Độ lệch chuẩn thay đổi thế nào? Vì sao?

6. Đổi `SEED` thành 5 giá trị khác nhau, chạy lại Bước 13 và Bước 23 cho XGBoost. ROC-AUC trên tập test dao động bao nhiêu? So với độ lệch chuẩn CV thì sao?

Mức 3 – Nâng cao.

7. Thêm đặc trưng mới `PAY_MEAN` là trung bình của `PAY_1`–`PAY_6`. Đặc trưng này có giúp hồi quy logistic không? Còn XGBoost thì sao? Giải thích sự khác biệt.

8. Bỏ các dòng có `MedHouseVal` $\geq 5{,}0$ khỏi tập test của California Housing rồi tính lại RMSE. Giải thích vì sao kết quả thay đổi.

9. Đọc về kiểm định t hiệu chỉnh của Nadeau and Bengio [18], cài đặt bằng Python, và áp dụng cho cặp XGBoost–LightGBM. Kết luận có thay đổi so với Wilcoxon không?

1.13 Đáp án phần Tự kiểm tra

Câu 1.

Khoảng $77{,}9\%$, đúng bằng tỷ lệ khách hàng không vỡ nợ. Bước 14 xác nhận điều này: mô hình ngây thơ đạt Accuracy 0,7788 trên tập test.

Câu 2.

Nên ưu tiên **Recall cao**. Bỏ sót một người mắc bệnh (FN) có thể khiến họ không được điều trị kịp thời, hậu quả rất nặng. Báo động nhầm (FP) chỉ khiến người đó phải làm thêm xét nghiệm. Đây là lý do các xét nghiệm sàng lọc thường được thiết kế để có Recall rất cao, chấp nhận Precision thấp.

Câu 3.

Không chắc. ROC-AUC đánh giá khả năng *xếp hạng* trên mọi ngưỡng, còn F1 chỉ đo tại *một* ngưỡng. Một mô hình có thể xếp hạng tốt nhưng có xác suất bị lệch (ví dụ, mọi xác suất đều dưới 0,5), nên ở ngưỡng 0,5 nó dự đoán toàn lớp 0 và F1 bằng 0. Chương 5 và Chương 8 sẽ có ví dụ cụ thể về hiện tượng này.

Thuật ngữ của chương

Tiếng Việt	Tiếng Anh	Nghĩa ngắn gọn
Đặc trưng	feature	Một cột thông tin đầu vào của mô hình
Nhãn, biến mục tiêu	label, target	Cột đáp án mà mô hình cần dự đoán
Phân loại / Hồi quy	classification / regression	Dự đoán lớp rời rạc / dự đoán số liên tục
Mất cân bằng	imbalanced	Một lớp ít hơn hẳn lớp kia
Tập huấn luyện / kiểm tra	train / test set	Phần dữ liệu để học / để chấm điểm cuối cùng
Chia phân tầng	stratified split	Chia mà giữ nguyên tỷ lệ các lớp
Ma trận nhầm lẫn	confusion matrix	Bảng đếm TN, FP, FN, TP
Độ chuẩn xác / Độ phủ	precision / recall	Báo đúng bao nhiêu / bắt được bao nhiêu
Ngưỡng quyết định	decision threshold	Mức xác suất để gán nhãn 1
Đánh giá chéo	cross-validation	Chia train thành *k* fold, lần lượt kiểm tra
Đường ống	pipeline	Chuỗi bước xử lý và mô hình gói làm một
Rò rỉ dữ liệu	data leakage	Thông tin từ dữ liệu kiểm tra lọt vào quá trình học
Siêu tham số	hyperparameter	Tham số do người dùng đặt, không học từ dữ liệu
Mô hình mốc	baseline	Mô hình đơn giản để so sánh

Tài liệu đọc thêm

Demšar [6] là tài liệu kinh điển về so sánh các bộ phân loại bằng kiểm định thống kê. Saito and Rehmsmeier [20] giải thích vì sao nên dùng PR-AUC cho dữ liệu mất cân bằng. Kaufman et al. [12] phân loại các dạng rò rỉ dữ liệu. Cawley and Talbot [2] phân tích hiện tượng quá khớp trong khâu chọn mô hình.

PHẦN II

Xử lý mất cân bằng dữ liệu

CHƯƠNG 2

SMOTE – Sinh mẫu tổng hợp cho lớp thiểu số

Chương này giúp bạn làm được gì?

Sau khi làm theo hết các bước trong chương, bạn sẽ:

- hiểu vì sao mô hình "ngại" dự đoán lớp thiểu số, và vì sao nhân bản dữ liệu chưa phải cách tốt;
- **tự tay viết** thuật toán SMOTE trong khoảng 10 dòng Python, và kiểm tra lại bằng thư viện `imbalanced-learn`;
- áp dụng SMOTE đúng cách trên dữ liệu thật, *bên trong* pipeline, kèm chuẩn hóa;
- tự đo được mức sai lệch khổng lồ khi đặt SMOTE sai chỗ (rò rỉ dữ liệu), và giải thích vì sao;
- so sánh SMOTE với một giải pháp đơn giản hơn nhiều là dời ngưỡng quyết định, và biết khi nào nên dùng cách nào;
- xử lý dữ liệu có biến phân loại bằng SMOTE-NC.

2.1 Trước khi bắt đầu

Bạn cần gì?

Chương này dùng lại mọi thứ đã làm ở Chương 1: cùng môi trường, cùng thư mục, cùng cách chia dữ liệu. Bạn cần cài thêm đúng một thư viện:

Cửa sổ lệnh (đã kích hoạt môi trường ảo)

```
pip install imbalanced-learn
```

`imbalanced-learn` (gọi tắt là `imblearn`) là thư viện chuyên xử lý dữ liệu mất cân bằng [15]. Nó được viết để dùng chung với scikit-learn.

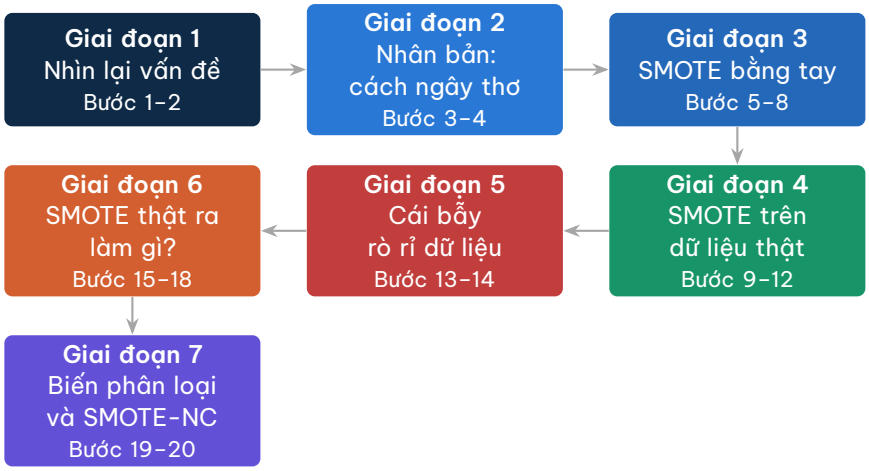
Vấn đề cần giải quyết

Ở Chương 1, mô hình XGBoost đạt ROC-AUC khá tốt (0,78), nhưng ở ngưỡng mặc định 0,5 nó chỉ phát hiện được khoảng **1/3 số khách hàng vỡ nợ**. Với ngân hàng, bỏ sót 2/3 số người sắp vỡ nợ là không chấp nhận được.

Một nguyên nhân là **mất cân bằng**: trong tập train, người không vỡ nợ đông gấp 3,5 lần người vỡ nợ. Khi học, mô hình cố gắng giảm *tổng* số lỗi. Vì lớp đa số chiếm 78% dữ liệu, cách

"an toàn" nhất để ít lỗi là nghiêng về phía dự đoán "không vỡ nợ".

Ý tưởng của chương này: **làm cho hai lớp cân bằng hơn bằng cách tạo thêm mẫu cho lớp thiểu số**. Kỹ thuật nổi tiếng nhất để làm điều đó là SMOTE (*Synthetic Minority Over-sampling TEchnique*) của Chawla et al. [4].



Hình 2.1: Lộ trình của Chương 2: 7 giai đoạn, 20 bước.

2.2 Giai đoạn 1 – Nhìn lại vấn đề

2.2.1 Bước 1. Nạp dữ liệu đã chia ở Chương 1

Mục đích: có đúng tập train và tập test như Chương 1, để mọi so sánh trong chương này công bằng.

Ô lệnh 1 · Nạp thư viện và dữ liệu đã chia ở Chương 1 code/ch02/s01_chuan_bi.py

```
import sys
sys.path.append("code")              # để Python tìm thấy file code/common.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from common import credit_split, xgb_clf

pd.set_option("display.width", 80)
pd.set_option("display.precision", 3)
SEED = 42

X_train, X_test, y_train, y_test = credit_split()     # đúng cách chia của Bước 13,
↪  Chương 1
print("Train:", X_train.shape, "  Test:", X_test.shape)
print("Số khách hàng trong tập train theo nhãn:", y_train.value_counts().to_dict())
```

Kết quả

```
Train: (24000, 23)   Test: (6000, 23)
Số khách hàng trong tập train theo nhãn: {0: 18691, 1: 5309}
```

Đọc kết quả. Thay vì lặp lại toàn bộ Bước 5–13 của Chương 1, ta gọi hàm `credit_split()` trong file `code/common.py`. Hàm này làm đúng các bước đọc, làm sạch và chia dữ liệu như Chương 1 (xem mục "Module dùng chung" ở cuối Chương 1).

- Dòng `sys.path.append("code")` báo cho Python biết: "hãy tìm thêm module trong thư mục `code/`". Thiếu dòng này, lệnh `from common import ...` sẽ báo lỗi `ModuleNotFoundError`.
- Tập train có **18 691** người không vỡ nợ và **5 309** người vỡ nợ. Hãy nhớ hai con số này; ta sẽ dùng lại ở Bước 10.

2.2.2 Bước 2. Mô hình hiện tại bỏ sót bao nhiêu người?

Mục đích: có một con số cụ thể về "vấn đề" mà chương này muốn giải quyết.

Ô lệnh 2 · Nhắc lại: XGBoost ở Chương 1 bỏ sót bao nhiêu người vỡ nợ? `code/ch02/s02_nhac_lai.py`

```
from sklearn.metrics import (confusion_matrix, precision_score, recall_score,
                             f1_score, average_precision_score)

xgb = xgb_clf().fit(X_train, y_train)
proba_base = xgb.predict_proba(X_test)[:, 1]
pred_base = (proba_base ⩾ 0.5).astype(int)

TN, FP, FN, TP = confusion_matrix(y_test, pred_base).ravel()
print(f"Thực sự vỡ nợ: {TP + FN} người -> phát hiện được {TP}, bỏ sót {FN}")
print(f"Precision = {precision_score(y_test, pred_base):.3f}, "
      f"Recall = {recall_score(y_test, pred_base):.3f}, F1 = {f1_score(y_test,
      ↪  pred_base):.3f}")
```

Kết quả

```
Thực sự vỡ nợ: 1327 người -> phát hiện được 479, bỏ sót 848
Precision = 0.662, Recall = 0.361, F1 = 0.467
```

Đọc kết quả. Hàm `xgb_clf()` trả về mô hình XGBoost với đúng cấu hình của Bước 22, Chương 1. Trên tập test có 1 327 người vỡ nợ, mô hình chỉ phát hiện 479 người (Recall $= 479/1\,327 = 0{,}361$) và **bỏ sót 848 người**. Đây là con số ta muốn giảm.

Tự kiểm tra

Câu 1. Nếu ta hạ ngưỡng quyết định từ 0,5 xuống 0,3 (như Bước 18 của Chương 1), số người bị bỏ sót sẽ tăng hay giảm? Precision sẽ thay đổi thế nào?

2.3 Giai đoạn 2 – Cách ngây thơ: nhân bản dữ liệu

2.3.1 Bước 3. Một bộ dữ liệu "đồ chơi" để nhìn bằng mắt

Mục đích: có một bộ dữ liệu chỉ 2 đặc trưng để *vẽ* được lên mặt phẳng, nhờ đó thấy tận mắt các kỹ thuật hoạt động thế nào.

Dữ liệu thật có 23 đặc trưng, không thể vẽ được. Vì vậy, trong Giai đoạn 2 và 3, ta tạm dùng một bộ dữ liệu nhân tạo chỉ có 2 đặc trưng.

Ô lệnh 3 · Một bộ dữ liệu "đồ chơi" 2 chiều để nhìn thấy SMOTE bằng mắt

code/ch02/s03_du_lieu_do_choi.py

```
from sklearn.datasets import make_classification

X_toy, y_toy = make_classification(n_samples=600, n_features=2, n_redundant=0,
                                   n_clusters_per_class=1, weights=[0.9, 0.1],
                                   class_sep=1.0, random_state=3)
print("Số điểm mỗi lớp [lớp 0, lớp 1]:", np.bincount(y_toy))
print("5 điểm đầu tiên:")
print(np.round(X_toy[:5], 3))
```

Kết quả

```
Số điểm mỗi lớp [lớp 0, lớp 1]: [534  66]
5 điểm đầu tiên:
[[0.697 0.909]
 [0.945 1.27 ]
 [1.18  0.165]
 [0.93  0.784]
 [0.999 1.466]]
```

Đọc kết quả. Hàm `make_classification` của scikit-learn tạo dữ liệu phân loại ngẫu nhiên theo yêu cầu:

- `n_samples=600`, `n_features=2`: 600 điểm, mỗi điểm có 2 tọa độ.
- `weights=[0.9, 0.1]`: khoảng 90% thuộc lớp 0, 10% thuộc lớp 1. Kết quả: 534 điểm lớp 0, 66 điểm lớp 1.
- `class_sep=1.0`: hai lớp tách nhau ở mức vừa phải, có một phần chồng lấn.

Mỗi dòng trong mảng in ra là một điểm (x, y) trên mặt phẳng. Hàm `np.bincount` đếm số lần xuất hiện của từng nhãn 0, 1.

2.3.2 Bước 4. Nhân bản ngẫu nhiên và điểm yếu của nó

Mục đích: thử cách cân bằng đơn giản nhất và thấy vấn đề của nó.

Cách dễ nghĩ nhất: chọn ngẫu nhiên các điểm lớp 1 và *chép lại* chúng cho đến khi hai lớp bằng nhau. Kỹ thuật này gọi là **nhân bản ngẫu nhiên** (*random oversampling*).

Ô lệnh 4 · Cách đơn giản nhất: nhân bản ngẫu nhiên các điểm thiểu số code/ch02/s04_nhan_ban.py

```
from imblearn.over_sampling import RandomOverSampler

X_ros, y_ros = RandomOverSampler(random_state=SEED).fit_resample(X_toy, y_toy)
```

```
print("Sau khi nhân bản [lớp 0, lớp 1]:", np.bincount(y_ros))

minority = X_ros[y_ros == 1]
print("Số điểm lớp 1:", len(minority),
      "| trong đó số điểm KHÁC NHAU:", len(np.unique(minority, axis=0)))
```

Kết quả

```
Sau khi nhân bản [lớp 0, lớp 1]: [534 534]
Số điểm lớp 1: 534 | trong đó số điểm KHÁC NHAU: 66
```

Đọc kết quả. Sau khi nhân bản, lớp 1 có 534 điểm, bằng lớp 0. Nhưng dòng thứ hai cho thấy vấn đề: trong 534 điểm đó, chỉ có **66 điểm khác nhau**, đúng bằng số điểm ban đầu. Trung bình mỗi điểm bị chép khoảng 8 lần. Hàm `np.unique(..., axis=0)` đếm số *dòng* khác nhau trong mảng.

Vì sao nhân bản chưa tốt?

Hãy tưởng tượng học sinh ôn thi bằng cách làm đi làm lại *đúng một* đề. Lần thứ 8, các em đã thuộc lòng đáp án nhưng chưa chắc làm được đề mới. Mô hình cũng vậy: khi thấy một điểm lặp lại 8 lần, nó học rất kỹ *đúng vị trí đó*, và vùng dự đoán "lớp 1" co lại thành những "ốc đảo" nhỏ quanh từng điểm. Hiện tượng này là **quá khớp** (*overfitting*): học thuộc dữ liệu cũ thay vì học quy luật chung.

2.4 Giai đoạn 3 – SMOTE từng bước bằng tay

Ý tưởng của SMOTE: thay vì *chép* điểm cũ, hãy **tạo điểm mới nằm giữa hai điểm thiểu số gần nhau**. Thuật toán chỉ có hai bước:

1. Với một điểm thiểu số, tìm k **láng giềng gần nhất** của nó, cũng thuộc lớp thiểu số.
2. Chọn ngẫu nhiên một láng giềng, rồi đặt điểm mới ở một vị trí ngẫu nhiên trên **đoạn thẳng** nối hai điểm.

Ta sẽ làm từng bước bằng tay.

2.4.1 Bước 5. Tìm láng giềng gần nhất

Mục đích: thực hiện bước 1 của SMOTE cho một điểm cụ thể.

Ô lệnh 5 · Bước 1 của SMOTE: tìm 5 láng giềng gần nhất (cùng lớp thiểu số)

code/ch02/s05_lang_gieng.py

```
from sklearn.neighbors import NearestNeighbors

X_min = X_toy[y_toy == 1]                          # 66 điểm thiểu số
nn = NearestNeighbors(n_neighbors=6).fit(X_min)    # 6 = chính nó + 5 láng giềng
dist, idx = nn.kneighbors(X_min[[0]])              # hỏi láng giềng của điểm số 0

print("Điểm x_0          :", X_min[0].round(3))
print("Chỉ số 5 láng giềng:", idx[0][1:])           # bỏ phần tử đầu (chính nó)
```

```
print("Khoảng cách        :", dist[0][1:].round(3))
```

Kết quả

```
Điểm x_0          : [-2.687  0.334]
Chỉ số 5 láng giềng: [40 41 51  3 34]
Khoảng cách        : [0.368 0.632 0.906 0.93  0.965]
```

Đọc kết quả.

- `X_min = X_toy[y_toy == 1]` lọc ra 66 điểm thiểu số. Biểu thức `y_toy == 1` tạo một mảng True/False, và `X_toy[...]` chỉ giữ các dòng ứng với True.
- `NearestNeighbors(n_neighbors=6)` tìm 6 điểm gần nhất. Ta cần 6 vì điểm gần nhất với x_0 luôn là *chính nó* (khoảng cách 0); 5 điểm còn lại mới là láng giềng thật.
- Láng giềng gần nhất của x_0 là điểm số 40, cách x_0 một khoảng 0,368. Láng giềng thứ năm là điểm số 34, cách 0,965.

"Khoảng cách" ở đây là **khoảng cách Euclid** quen thuộc: với hai điểm (a_1, a_2) và (b_1, b_2),

$$d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}.$$

2.4.2 Bước 6. Sinh một điểm mới bằng tay

Mục đích: thực hiện bước 2 của SMOTE và hiểu chính xác công thức.

Công thức SMOTE để sinh điểm mới $\mathbf{x}_{\text{mới}}$ từ điểm $\mathbf{x}_i$ và láng giềng $\mathbf{x}_{\text{lg}}$ là:

$$\mathbf{x}_{\text{mới}} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{\text{lg}} - \mathbf{x}_i), \qquad \lambda \text{ là số ngẫu nhiên trong khoảng } (0, 1). \tag{2.1}$$

Ô lệnh 6 · Bước 2 của SMOTE: sinh MỘT điểm mới bằng tay `code/ch02/s06_sinh_mot_diem.py`

```
rng = np.random.default_rng(SEED)
j = rng.choice(idx[0][1:])          # chọn ngẫu nhiên 1 trong 5 láng giềng
lam = rng.uniform(0, 1)             # hệ số lambda ngẫu nhiên trong khoảng (0, 1)

x_i, x_nb = X_min[0], X_min[j]
x_new = x_i + lam * (x_nb - x_i)    # công thức SMOTE

print(f"Láng giềng được chọn: điểm số {j}, lambda = {lam:.3f}")
print("x_i   =", x_i.round(3))
print("x_nb  =", x_nb.round(3))
print("x_new =", x_new.round(3))
```

Kết quả

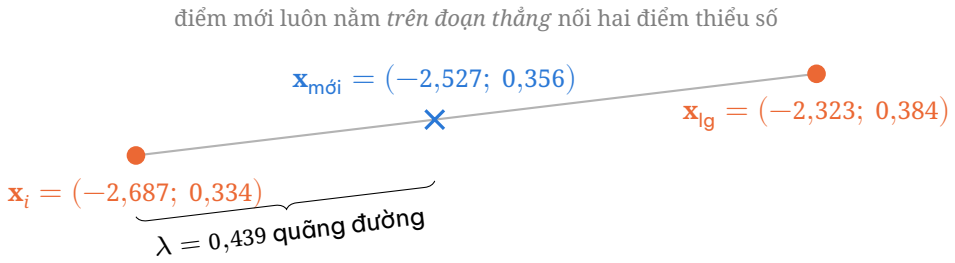
```
Láng giềng được chọn: điểm số 40, lambda = 0.439
x_i   = [-2.687  0.334]
x_nb  = [-2.323  0.384]
x_new = [-2.527  0.356]
```

Đọc kết quả. Hãy tự tính lại từng tọa độ với $\lambda = 0{,}439$:

$$\begin{aligned}\text{Tọa độ 1:}\quad & -2{,}687 + 0{,}439 \times \big(-2{,}323 - (-2{,}687)\big) \\ & = -2{,}687 + 0{,}439 \times 0{,}364 = -2{,}687 + 0{,}160 = -2{,}527,\end{aligned}$$

$$\begin{aligned}\text{Tọa độ 2:}\quad & 0{,}334 + 0{,}439 \times (0{,}384 - 0{,}334) \\ & = 0{,}334 + 0{,}439 \times 0{,}050 = 0{,}334 + 0{,}022 = 0{,}356.\end{aligned}$$

Hai kết quả khớp với máy tính. Hình 2.2 minh họa ý nghĩa hình học của công thức.



Hình 2.2: Sinh một điểm tổng hợp bằng (2.1). Với $\lambda = 0{,}439$, điểm mới nằm ở 43,9% quãng đường từ $\mathbf{x}_i$ đến láng giềng $\mathbf{x}_{\text{lg}}$.

Điểm mới "trông giống" lớp thiểu số

Vì λ nằm giữa 0 và 1, điểm mới luôn nằm *giữa* hai điểm thiểu số gần nhau. Nếu hai khách hàng vỡ nợ có hạn mức 50 triệu và 60 triệu, khách hàng "tổng hợp" sẽ có hạn mức khoảng 50 đến 60 triệu. Dữ liệu mới không phải bản sao, nhưng cũng không "bịa" ra những giá trị vô lý nằm ngoài vùng của lớp thiểu số.

2.4.3 Bước 7. Tự viết hàm SMOTE hoàn chỉnh

Mục đích: ghép Bước 5 và Bước 6 thành một hàm, lặp lại cho đến khi đủ số điểm cần thiết.

Ô lệnh 7 · Tự viết hàm SMOTE hoàn chỉnh (khoảng 10 dòng) code/ch02/s07_smote_tu_viet.py

```python
def smote_tay(X_min, G, k=5, seed=SEED):
    """Sinh G điểm tổng hợp từ tập thiểu số X_min (phiên bản tối giản để học)."""
    rng = np.random.default_rng(seed)
    _, idx = NearestNeighbors(n_neighbors=k + 1).fit(X_min).kneighbors(X_min)
    new = []
    for _ in range(G):
        i = rng.integers(len(X_min))          # chọn ngẫu nhiên một điểm thiểu số
        j = rng.choice(idx[i][1:])            # chọn một trong k láng giềng của nó
        lam = rng.uniform(0, 1)
        new.append(X_min[i] + lam * (X_min[j] - X_min[i]))
    return np.array(new)
```

```
G = (y_toy == 0).sum() - (y_toy == 1).sum()   # số điểm cần sinh để hai lớp bằng nhau
X_syn = smote_tay(X_min, G)
print("Cần sinh:", G, "điểm | Kích thước kết quả:", X_syn.shape)
```

Kết quả

```
Cần sinh: 468 điểm | Kích thước kết quả: (468, 2)
```

Đọc kết quả. Đọc hàm `smote_tay` từng dòng:

1. `NearestNeighbors(...).fit(X_min).kneighbors(X_min)`: tìm $k+1$ láng giềng cho *mọi* điểm thiểu số cùng lúc. Kết quả `idx` là bảng 66 dòng, mỗi dòng chứa chỉ số các láng giềng của một điểm.
2. Vòng `for` chạy G lần, mỗi lần sinh một điểm:
   - `rng.integers(len(X_min))` chọn ngẫu nhiên một điểm thiểu số i;
   - `rng.choice(idx[i][1:])` chọn ngẫu nhiên một láng giềng j của nó (bỏ phần tử đầu là chính nó);
   - dòng `new.append(...)` chính là (2.1).
3. Số điểm cần sinh: $G = 534 - 66 = 468$, để sau khi thêm vào, lớp 1 có $66 + 468 = 534$ điểm, bằng lớp 0.

Toàn bộ thuật toán được tóm tắt trong Algorithm 2.

Thuật toán 2: SMOTE [4]

Đầu vào: Tập thiểu số $S_{\min}$ (n_1 điểm, đã chuẩn hóa); số điểm cần sinh G; số láng giềng k
Đầu ra: Tập điểm tổng hợp S_{syn} gồm G điểm

for *mỗi* $\mathbf{x}_i \in S_{\min}$ **do**
  Tìm k láng giềng gần nhất của $\mathbf{x}_i$ trong $S_{\min}$; // Bước 5
$S_{\text{syn}} \leftarrow \emptyset$;
for $g = 1$ **to** G **do**
  Chọn ngẫu nhiên $\mathbf{x}_i \in S_{\min}$ và một láng giềng $\mathbf{x}_{\text{lg}}$ của nó;
  Lấy λ ngẫu nhiên trong $(0, 1)$;
  Thêm $\mathbf{x}_i + \lambda(\mathbf{x}_{\text{lg}} - \mathbf{x}_i)$ vào S_{syn} ; // Bước 6
return S_{syn}

2.4.4 Bước 8. Dùng SMOTE của thư viện và nhìn kết quả

Mục đích: dùng phiên bản đã được kiểm thử kỹ của `imblearn`, và *nhìn thấy* các điểm tổng hợp.

Ô lệnh 8 · Dùng SMOTE có sẵn trong imbalanced-learn và vẽ kết quả

code/ch02/s08_smote_imblearn.py

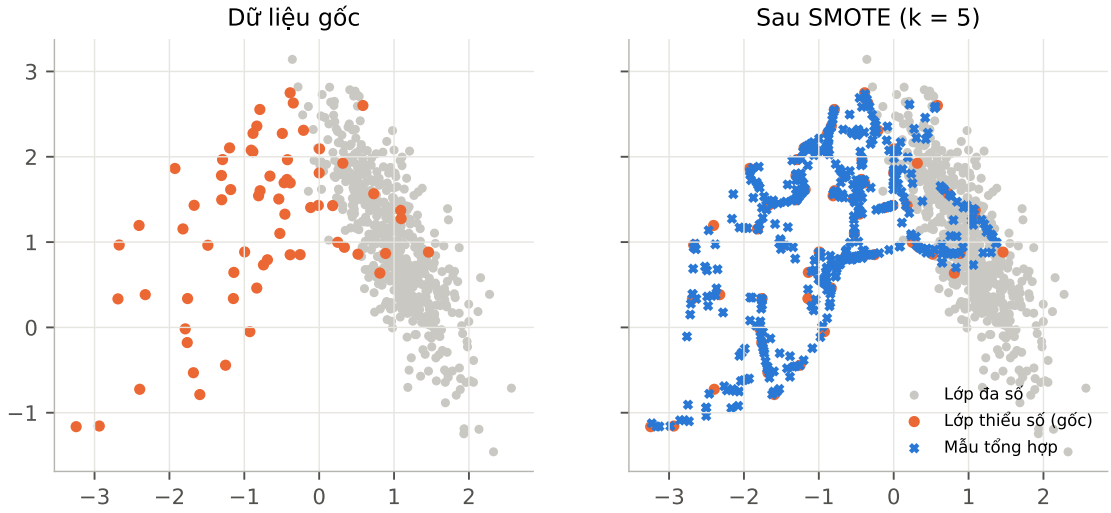
```
from imblearn.over_sampling import SMOTE

X_sm, y_sm = SMOTE(k_neighbors=5, random_state=SEED).fit_resample(X_toy, y_toy)
print("Sau SMOTE [lớp 0, lớp 1]:", np.bincount(y_sm))
syn = X_sm[len(X_toy):]                          # imblearn đặt các điểm mới ở cuối mảng
```

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3.3), sharex=True, sharey=True)
for ax, title in zip(axes, ["Dữ liệu gốc", "Sau SMOTE (k = 5)"]):
    ax.scatter(*X_toy[y_toy == 0].T, s=8, color="#c9c8c2", label="Lớp đa số")
    ax.scatter(*X_toy[y_toy == 1].T, s=14, color="#eb6834", label="Lớp thiểu số (gốc)")
    ax.set_title(title)
axes[1].scatter(*syn.T, s=10, marker="x", color="#2a78d6", label="Mẫu tổng hợp")
axes[1].legend(loc="lower right", fontsize=7)
plt.savefig("figures/ch02_s08_toy.pdf", bbox_inches="tight")
```

Kết quả

```
Sau SMOTE [lớp 0, lớp 1]: [534 534]
```



Hình 2.3: Dữ liệu đồ chơi trước và sau SMOTE. Các dấu × màu xanh là 468 điểm tổng hợp; chúng nằm trên các đoạn thẳng nối những điểm thiểu số gần nhau.

Đọc kết quả. Cách dùng `SMOTE` giống mọi công cụ của `imblearn`: tạo đối tượng, rồi gọi phương thức `fit_resample` với hai tham số `X` và `y`. Hàm trả về `X` và `y` *mới*, gồm toàn bộ dữ liệu cũ cộng với các điểm tổng hợp được nối vào cuối.

Quan sát Hình 2.3:

- Ở bên trái của hình phải, các điểm tổng hợp **lấp đầy khoảng trống** giữa những điểm thiểu số gốc. Đây là điều ta mong muốn.
- Ở vùng giữa, nơi lớp cam "lấn" vào vùng xám, SMOTE cũng sinh rất nhiều điểm. Những điểm này nằm *giữa các điểm của lớp đa số*, nên có thể làm mô hình nhầm lẫn. Đây là nhược điểm của SMOTE, và Chương 3 and 4 sẽ tìm cách khắc phục.

Thử tự làm

Trong Ô lệnh 8, đổi `k_neighbors=5` thành `k_neighbors=1`, rồi thành `k_neighbors=30`. Vẽ lại hình. Với $k = 1$, các điểm tổng hợp nằm sát các điểm gốc hơn hay xa hơn? Với $k = 30$ thì sao?

2.5 Giai đoạn 4 – SMOTE trên dữ liệu thật

2.5.1 Bước 9. Vì sao phải chuẩn hóa trước khi dùng SMOTE?

Mục đích: hiểu vì sao bước chuẩn hóa ảnh hưởng trực tiếp đến chất lượng các điểm tổng hợp.

SMOTE dựa hoàn toàn vào "láng giềng gần nhất", tức dựa vào khoảng cách Euclid. Mà khoảng cách Euclid lại bị chi phối bởi đặc trưng có *số lớn*. Hãy kiểm tra trên dữ liệu thật:

Ô lệnh 9 · Vì sao phải chuẩn hóa trước SMOTE? So sánh "láng giềng gần nhất"

code/ch02/s09_chuan_hoa.py

```
from sklearn.preprocessing import StandardScaler

X_min_c = X_train[y_train == 1]                    # 5 309 khách hàng vỡ nợ (tập train)
scaler = StandardScaler().fit(X_train)
nb_raw = NearestNeighbors(n_neighbors=2).fit(X_min_c).kneighbors(X_min_c)[1][:, 1]
Z = scaler.transform(X_min_c)
nb_std = NearestNeighbors(n_neighbors=2).fit(Z).kneighbors(Z)[1][:, 1]

i = 2                                              # một ví dụ cụ thể
cols = ["LIMIT_BAL", "AGE", "PAY_1", "PAY_2", "BILL_AMT1", "PAY_AMT1"]
print(pd.DataFrame({"Khách hàng": X_min_c.iloc[i][cols],
                    "LG chưa chuẩn hóa": X_min_c.iloc[nb_raw[i]][cols],
                    "LG đã chuẩn hóa": X_min_c.iloc[nb_std[i]][cols]}))

same = lambda nb: (X_min_c["PAY_1"].values == X_min_c["PAY_1"].values[nb]).mean()
print(f"\nTỷ lệ láng giềng có CÙNG PAY_1 -- chưa chuẩn hóa: {same(nb_raw):.1%},"
      f" đã chuẩn hóa: {same(nb_std):.1%}")
```

Kết quả

```
           Khách hàng  LG chưa chuẩn hóa  LG đã chuẩn hóa
LIMIT_BAL     50000.0            50000.0          30000.0
AGE              34.0               22.0             31.0
PAY_1             3.0                0.0              3.0
PAY_2             3.0                0.0              3.0
BILL_AMT1     47904.0            45458.0          25320.0
PAY_AMT1       2000.0             2051.0              0.0

Tỷ lệ láng giềng có CÙNG PAY_1 -- chưa chuẩn hóa: 39.5%, đã chuẩn hóa: 84.9%
```

Đọc kết quả. Bảng so sánh một khách hàng vỡ nợ (cột 1) với láng giềng gần nhất của họ theo hai cách tính:

- **Chưa chuẩn hóa** (cột 2): láng giềng có `LIMIT_BAL`, `BILL_AMT1`, `PAY_AMT1` rất gần (đều là số hàng chục nghìn). Nhưng `PAY_1` và `PAY_2` của họ là 0 (trả bình thường), trong khi khách hàng gốc **trễ 3 tháng**. Đây là hai người có hành vi trả nợ rất khác nhau.
- **Đã chuẩn hóa** (cột 3): láng giềng cũng trễ đúng 3 tháng ở cả `PAY_1` lẫn `PAY_2`.

Lý do: chênh lệch `BILL_AMT1` giữa hai người là $47\,904 - 45\,458 = 2\,446$, lớn gấp hàng trăm lần chênh lệch `PAY_1` ($3 - 0 = 3$). Khi chưa chuẩn hóa, khoảng cách Euclid gần như "không thấy" biến `PAY_1`.

Dòng cuối cho thấy đây không phải trường hợp cá biệt. Chưa chuẩn hóa, chỉ 39,5% khách hàng có láng giềng cùng trạng thái PAY_1. Đã chuẩn hóa, con số này là 84,9%. Mà PAY_1 lại chính là đặc trưng quan trọng nhất (Bước 12, Chương 1). **Không chuẩn hóa thì SMOTE sẽ "trộn" những khách hàng chẳng giống nhau.**

2.5.2 Bước 10. SMOTE thay đổi tập train như thế nào?

Mục đích: nhìn rõ số lượng trước và sau SMOTE, và chắc chắn rằng tập test không bị động đến.

Ô lệnh 10 · SMOTE thay đổi tập train như thế nào? code/ch02/s10_ben_trong.py

```
X_res, y_res = SMOTE(random_state=SEED).fit_resample(scaler.transform(X_train),
↪  y_train)

print("Trước SMOTE:", y_train.value_counts().to_dict())
print("Sau SMOTE  :", pd.Series(y_res).value_counts().to_dict())
print("Số mẫu tổng hợp được thêm vào:", len(y_res) - len(y_train))
print("Tập test vẫn giữ nguyên       :", y_test.value_counts().to_dict())
```

Kết quả

```
Trước SMOTE: {0: 18691, 1: 5309}
Sau SMOTE  : {0: 18691, 1: 18691}
Số mẫu tổng hợp được thêm vào: 13382
Tập test vẫn giữ nguyên       : {0: 4673, 1: 1327}
```

Đọc kết quả.

- Trước SMOTE: 18 691 người không vỡ nợ, 5 309 người vỡ nợ.
- SMOTE sinh thêm 18 691 − 5 309 = 13 382 "khách hàng vỡ nợ tổng hợp". Sau SMOTE, hai lớp bằng nhau: mỗi lớp 18 691 người. Tập train mới có 37 382 dòng.
- **Tập test giữ nguyên** 4 673 / 1 327. Đây là điều bắt buộc: mô hình phải được chấm điểm trên *khách hàng thật*, với tỷ lệ thật.

SMOTE chỉ dành cho dữ liệu huấn luyện

Các điểm tổng hợp là "dữ liệu giả". Chúng chỉ được dùng để *dạy* mô hình, không bao giờ được xuất hiện trong dữ liệu dùng để *chấm điểm*. Giai đoạn 5 sẽ cho thấy chuyện gì xảy ra nếu vi phạm điều này.

2.5.3 Bước 11. Đánh giá chéo có và không có SMOTE

Mục đích: so sánh công bằng bốn cấu hình bằng đúng quy trình của Chương 1.

Ô lệnh 11 · So sánh có/không SMOTE bằng đánh giá chéo 5-fold code/ch02/s11_so_sanh_cv.py

```
from imblearn.pipeline import Pipeline          # CHÚ Ý: Pipeline của imblearn, không
↪  phải sklearn
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.linear_model import LogisticRegression
```

```python
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
scoring = {"ROC-AUC": "roc_auc", "PR-AUC": "average_precision",
           "F1": "f1", "Recall": "recall", "Precision": "precision"}

def make(model, smote):
    """Ghép chuẩn hóa -> (SMOTE) -> mô hình thành một pipeline."""
    steps = [("scale", StandardScaler())]
    if smote:
        steps.append(("smote", SMOTE(random_state=SEED)))
    return Pipeline(steps + [("model", model)])

configs = {"LogReg": make(LogisticRegression(max_iter=2000), False),
           "LogReg + SMOTE": make(LogisticRegression(max_iter=2000), True),
           "XGBoost": make(xgb_clf(), False),
           "XGBoost + SMOTE": make(xgb_clf(), True)}
cv_table = {}
for name, pipe in configs.items():
    r = cross_validate(pipe, X_train, y_train, cv=skf, scoring=scoring)
    cv_table[name] = {k: r["test_" + k].mean() for k in scoring}
print(pd.DataFrame(cv_table).T)
```

Kết quả

```
                ROC-AUC  PR-AUC     F1  Recall  Precision
LogReg            0.726   0.506  0.366   0.246      0.714
LogReg + SMOTE    0.728   0.503  0.478   0.652      0.377
XGBoost           0.783   0.558  0.476   0.369      0.671
XGBoost + SMOTE   0.768   0.539  0.512   0.459      0.580
```

Đọc kết quả. Trước hết, về code:

- Hàm `make` ghép ba bước thành một pipeline: **chuẩn hóa** → **SMOTE** (nếu `smote=True`) → **mô hình**.
- Ta dùng `Pipeline` của `imblearn`, *không phải* của scikit-learn. Pipeline của `imblearn` hiểu rằng bước SMOTE chỉ chạy khi *huấn luyện* (`fit`), và tự động bỏ qua khi *dự đoán* (`predict`). Nhờ vậy, ở mỗi fold, SMOTE chỉ được áp dụng lên 4 fold học; fold kiểm tra luôn là khách hàng thật.

Về kết quả (mỗi số là trung bình của 5 fold):

XGBoost	Recall	Precision	F1	PR-AUC
Không SMOTE	0,369	0,671	0,476	0,558
Có SMOTE	0,459	0,580	0,512	0,539
Thay đổi	+0,090	−0,091	+0,036	−0,019

- **Recall tăng** khoảng 9 điểm phần trăm: mô hình bắt được nhiều người vỡ nợ hơn. Với LogReg, Recall còn tăng mạnh hơn, từ 0,246 lên 0,652.
- **Precision giảm** gần đúng bằng mức đó: mô hình báo động nhầm nhiều hơn.
- **F1 tăng nhẹ**, vì Recall đang rất thấp nên cải thiện nó có lợi hơn.
- **PR-AUC và ROC-AUC không tăng, thậm chí giảm nhẹ** với XGBoost.

Điểm cuối cùng rất quan trọng. Nhớ lại Chương 1: ROC-AUC và PR-AUC đo khả năng *xếp hạng* khách hàng theo rủi ro, không phụ thuộc ngưỡng. Vậy SMOTE **không giúp mô hình phân biệt người vỡ nợ với người không vỡ nợ tốt hơn**. Nó chỉ làm mô hình "mạnh tay" hơn khi gắn nhãn vỡ nợ. Giai đoạn 6 sẽ đào sâu nhận xét này.

2.5.4 Bước 12. Kiểm tra trên tập test

Mục đích: xác nhận kết luận của CV trên dữ liệu hoàn toàn mới.

Ô lệnh 12 · Kiểm tra trên tập test (ngưỡng 0,5) code/ch02/s12_test.py

```
test_table = {}
for name, pipe in configs.items():
    pipe.fit(X_train, y_train)
    p = pipe.predict_proba(X_test)[:, 1]
    pred = (p ⩾ 0.5).astype(int)
    test_table[name] = {"PR-AUC": average_precision_score(y_test, p),
                        "F1": f1_score(y_test, pred),
                        "Recall": recall_score(y_test, pred),
                        "Precision": precision_score(y_test, pred),
                        "Bị gắn cờ": pred.sum()}
test_df = pd.DataFrame(test_table).T
test_df["Bị gắn cờ"] = test_df["Bị gắn cờ"].astype(int)
print(test_df)
```

Kết quả

	PR-AUC	F1	Recall	Precision	Bị gắn cờ
LogReg	0.494	0.358	0.242	0.690	465
LogReg + SMOTE	0.492	0.458	0.627	0.361	2303
XGBoost	0.556	0.467	0.361	0.662	724
XGBoost + SMOTE	0.547	0.507	0.453	0.576	1044

Đọc kết quả. Kết quả trên tập test có cùng xu hướng với CV. Cột *Bị gắn cờ* cho biết số khách hàng bị dự đoán là vỡ nợ:

- XGBoost gắn cờ 724 người; thêm SMOTE thì gắn cờ 1 044 người. Recall tăng từ 0,361 lên 0,453.
- LogReg + SMOTE gắn cờ tới 2 303 người, gấp 5 lần LogReg gốc. Recall lên 0,627 nhưng Precision chỉ còn 0,361: cứ 10 người bị gắn cờ thì gần 2/3 là báo động nhầm.

Tự kiểm tra

Câu 2. Một bạn sinh viên báo cáo: "Dùng SMOTE giúp F1 của XGBoost tăng từ 0,467 lên 0,507, vậy SMOTE làm mô hình tốt hơn." Kết luận này có đầy đủ không? Bạn sẽ hỏi thêm điều gì?

2.6 Giai đoạn 5 – Cái bẫy rò rỉ dữ liệu

2.6.1 Bước 13. Làm sai thử một lần

Mục đích: tận mắt thấy kết quả bị "thổi phồng" thế nào khi SMOTE được chạy *trước* khi chia fold. Đây là lỗi rất hay gặp trong khóa luận và bài báo.

Cách làm sai trông rất tự nhiên: "Dữ liệu bị mất cân bằng, vậy trước hết hãy cân bằng nó bằng SMOTE, rồi đánh giá mô hình bằng 5-fold CV."

Ô lệnh 13 · Làm SAI: chạy SMOTE trên toàn bộ tập train RỒI MỚI chia fold code/ch02/s13_ro_ri.py

```
X_leak, y_leak = SMOTE(random_state=SEED).fit_resample(scaler.transform(X_train),
↪  y_train)
r_wrong = cross_validate(xgb_clf(), X_leak, y_leak, cv=skf, scoring=scoring)
wrong = {k: r_wrong["test_" + k].mean() for k in scoring}

print(pd.DataFrame({"ĐÚNG (SMOTE trong pipeline)": cv_table["XGBoost + SMOTE"],
                    "SAI (SMOTE trước khi chia)": wrong}).round(3))
```

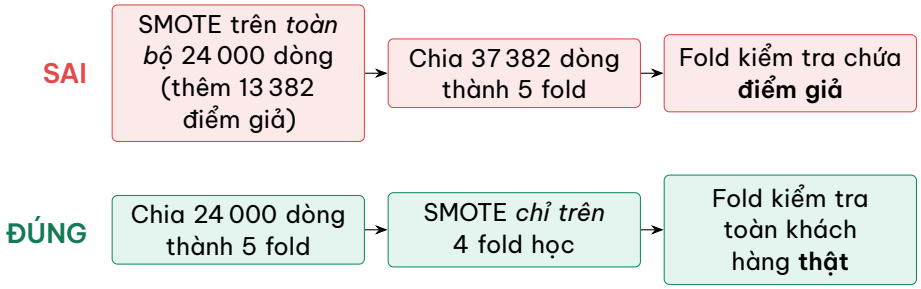
Kết quả

	ĐÚNG (SMOTE trong pipeline)	SAI (SMOTE trước khi chia)
ROC-AUC	0.768	0.913
PR-AUC	0.539	0.926
F1	0.512	0.832
Recall	0.459	0.781
Precision	0.580	0.890

Đọc kết quả. Cùng mô hình, cùng dữ liệu, cùng SMOTE. Chỉ khác *thứ tự* hai bước, mà:

- ROC-AUC tăng từ 0,768 lên 0,913;
- **PR-AUC tăng từ** 0,539 **lên** 0,926, gần gấp đôi;
- F1 tăng từ 0,512 lên 0,832.

Một sinh viên làm sai sẽ hào hứng báo cáo "mô hình đạt F1 = 0,83", trong khi trên khách hàng thật (tập test, Bước 12) F1 chỉ là 0,507.



Hình 2.4: Hai thứ tự thực hiện. Ở cách sai, các điểm giả vừa nằm trong phần học vừa nằm trong phần kiểm tra.

2.6.2 Bước 14. Vì sao làm sai lại cho điểm cao như vậy?

Mục đích: hiểu *cơ chế* của rò rỉ, không chỉ nhớ quy tắc.

Mỗi điểm tổng hợp được tạo ra từ hai khách hàng vỡ nợ thật, gọi là hai điểm "cha mẹ" của nó. Ở cách làm sai, một điểm tổng hợp có thể nằm trong fold kiểm tra, trong khi cha mẹ của nó

nằm trong fold học. Mô hình chỉ cần nhận ra "điểm này nằm ngay cạnh một người vỡ nợ mình đã thấy" là đoán đúng. Hãy đo xem các điểm tổng hợp nằm gần dữ liệu thật đến mức nào:

Ô lệnh 14 · Vì sao làm sai lại cho điểm cao như vậy? Đo khoảng cách code/ch02/s14_vi_sao_ro_ri.py

```
real_min = Z                                  # 5 309 khách hàng vỡ nợ thật (đã chuẩn hóa)
synth = X_leak[len(X_train):]                   # 13 382 điểm tổng hợp (nằm ở cuối mảng)
nn_real = NearestNeighbors(n_neighbors=2).fit(real_min)

d_real = nn_real.kneighbors(real_min)[0][:, 1]   # điểm thật -> điểm thật KHÁC gần nhất
d_syn = nn_real.kneighbors(synth)[0][:, 0]       # điểm tổng hợp -> điểm thật gần nhất
print(f"Trung vị khoảng cách: điểm thật     -> điểm thật khác gần nhất:
 ↪  {np.median(d_real):.3f}")
print(f"Trung vị khoảng cách: điểm tổng hợp -> điểm thật gần nhất     :
 ↪  {np.median(d_syn):.3f}")
```

Kết quả

```
Trung vị khoảng cách: điểm thật     -> điểm thật khác gần nhất: 1.058
Trung vị khoảng cách: điểm tổng hợp -> điểm thật gần nhất     : 0.302
```

Đọc kết quả.

- Với một khách hàng vỡ nợ *thật*, người vỡ nợ thật *khác* gần nhất cách họ khoảng 1,06 (trung vị, đơn vị là độ lệch chuẩn sau khi chuẩn hóa).
- Với một điểm *tổng hợp*, khách hàng thật gần nhất chỉ cách khoảng 0,30, tức **gần gấp 3,5 lần**.

Nói cách khác: khi kiểm tra trên điểm tổng hợp, mô hình được hỏi về những "khách hàng" gần như trùng với người nó đã học. Đó là bài kiểm tra dễ hơn rất nhiều so với dự đoán một khách hàng thật hoàn toàn mới. Hiện tượng này là một dạng **rò rỉ dữ liệu** [12].

Dấu hiệu nhận biết bài báo bị rò rỉ do SMOTE

- Phần phương pháp viết kiểu: *"we first balanced the dataset using SMOTE, then performed 10-fold cross-validation"*.
- Kết quả cao bất thường so với các công trình khác trên cùng bộ dữ liệu.
- Tập test cũng có tỷ lệ lớp 50:50, trong khi dữ liệu gốc mất cân bằng.

Đây là một trong những lý do phổ biến nhất khiến bài báo bị từ chối.

2.7 Giai đoạn 6 – SMOTE thật ra làm gì?

Ở Bước 11, ta đã nhận thấy: SMOTE làm tăng Recall, giảm Precision, nhưng không làm tăng khả năng xếp hạng (PR-AUC). Ba bước tiếp theo sẽ giải thích hiện tượng này.

2.7.1 Bước 15. Ảnh hưởng của số láng giềng k

Mục đích: biết tham số chính của SMOTE thay đổi kết quả ra sao.

Ô lệnh 15 · Ảnh hưởng của số láng giềng k (đánh giá chéo, mất khoảng 1 phút)

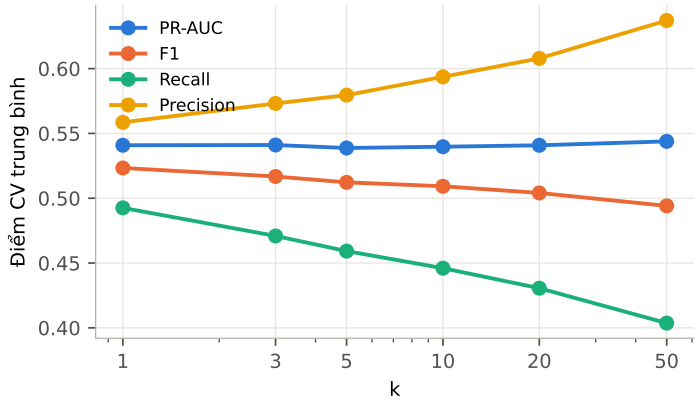
code/ch02/s15_tham_so_k.py

```
rows = []
for k in [1, 3, 5, 10, 20, 50]:
    pipe = Pipeline([("scale", StandardScaler()),
                     ("smote", SMOTE(k_neighbors=k, random_state=SEED)),
                     ("model", xgb_clf())])
    r = cross_validate(pipe, X_train, y_train, cv=skf, scoring=scoring)
    rows.append({"k": k, **{m: r["test_" + m].mean()
                            for m in ["PR-AUC", "F1", "Recall", "Precision"]}})
k_table = pd.DataFrame(rows).set_index("k")
print(k_table)

ax = k_table[["PR-AUC", "F1", "Recall", "Precision"]].plot(marker="o", figsize=(5,
↪  2.8), logx=True)
ax.set_xticks(k_table.index); ax.set_xticklabels(k_table.index)
ax.set_ylabel("Điểm CV trung bình")
plt.savefig("figures/ch02_s15_k.pdf", bbox_inches="tight")
```

Kết quả

k	PR-AUC	F1	Recall	Precision
1	0.541	0.523	0.493	0.559
3	0.541	0.517	0.471	0.573
5	0.539	0.512	0.459	0.580
10	0.540	0.509	0.446	0.594
20	0.541	0.504	0.431	0.608
50	0.544	0.494	0.404	0.637



Hình 2.5: Điểm CV của XGBoost + SMOTE theo số láng giềng *k* (trục hoành theo thang log).

Đọc kết quả. Khi *k* tăng từ 1 lên 50:

- Recall *giảm* từ 0,493 xuống 0,404, Precision *tăng* từ 0,559 lên 0,637.
- PR-AUC gần như đứng yên, quanh 0,54.

Một lần nữa: *k* thay đổi *sự đánh đổi* giữa Recall và Precision, nhưng không thay đổi khả năng xếp hạng. Điều này gợi ý rằng SMOTE hoạt động giống như việc **dời ngưỡng quyết định**. Ta

thử kiểm chứng.

2.7.2 Bước 16. Một đối thủ đơn giản: dời ngưỡng

Mục đích: tìm ngưỡng tốt nhất cho mô hình gốc (không SMOTE), *mà không đụng đến tập test*.

Ở Chương 1 (Bước 18), ta đã thấy hạ ngưỡng làm tăng Recall. Câu hỏi là nên hạ xuống bao nhiêu. Ta không được dùng tập test để chọn ngưỡng. Vì vậy, ta dùng **dự đoán out-of-fold**: chạy 5-fold CV trên tập train, và với mỗi khách hàng, lấy xác suất do mô hình *chưa từng thấy họ* dự đoán. Sau đó thử mọi ngưỡng từ 0,05 đến 0,95 và chọn ngưỡng cho F1 cao nhất.

Ô lệnh 16 · Một đối thủ đơn giản: giữ nguyên mô hình, chỉ dời ngưỡng `code/ch02/s16_doi_nguong.py`

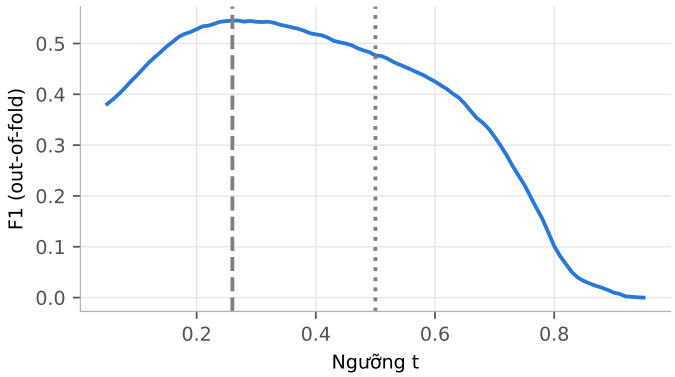
```
from sklearn.model_selection import cross_val_predict

# Dự đoán "out-of-fold": mỗi khách hàng train được dự đoán bởi mô hình CHƯA từng thấy họ
oof = cross_val_predict(xgb_clf(), X_train, y_train, cv=skf, method="predict_proba")[:,
↪  1]
thresholds = np.round(np.arange(0.05, 0.951, 0.01), 2)
f1s = [f1_score(y_train, (oof ⩾ t).astype(int)) for t in thresholds]
best_t = thresholds[np.argmax(f1s)]
print(f"Ngưỡng cho F1 cao nhất (tính trên tập train): t* = {best_t:.2f}, F1 =
↪  {max(f1s):.3f}")

fig, ax = plt.subplots(figsize=(5, 2.6))
ax.plot(thresholds, f1s)
ax.axvline(best_t, ls="--", color="gray"); ax.axvline(0.5, ls=":", color="gray")
ax.set_xlabel("Ngưỡng t"); ax.set_ylabel("F1 (out-of-fold)")
plt.savefig("figures/ch02_s16_threshold.pdf", bbox_inches="tight")
```

Kết quả

```
Ngưỡng cho F1 cao nhất (tính trên tập train): t* = 0.26, F1 = 0.545
```



Hình 2.6: F1 (tính trên dự đoán out-of-fold của tập train) theo ngưỡng t. Đường nét đứt: ngưỡng tốt nhất $t^\star = 0{,}26$. Đường chấm: ngưỡng mặc định 0,5.

Đọc kết quả. Hàm `cross_val_predict` trả về, cho *mỗi* khách hàng trong tập train, xác suất dự đoán bởi mô hình được học trên 4 fold còn lại. Đường cong ở hình trên có dạng "ngọn đồi": ngưỡng quá thấp thì báo động nhầm quá nhiều, ngưỡng quá cao thì bỏ sót quá nhiều. Đỉnh đồi

ở $t^\star = 0{,}26$ với F1 $= 0{,}545$, cao hơn hẳn mức F1 ở ngưỡng 0,5.

2.7.3 Bước 17. SMOTE hay dời ngưỡng?

Mục đích: so sánh trực tiếp hai giải pháp trên tập test.

Ô lệnh 17 · So sánh trên tập test: SMOTE hay dời ngưỡng? code/ch02/s17_so_sanh_cuoi.py

```
p_smote = configs["XGBoost + SMOTE"].predict_proba(X_test)[:, 1]
cases = {"XGBoost, t = 0,5": (proba_base, 0.5),
         "XGBoost + SMOTE, t = 0,5": (p_smote, 0.5),
         f"XGBoost, t = {best_t:.2f}".replace(".", ","): (proba_base, best_t)}
cmp = {}
for name, (p, t) in cases.items():
    pred = (p ⩾ t).astype(int)
    cmp[name] = {"F1": f1_score(y_test, pred), "Recall": recall_score(y_test, pred),
                 "Precision": precision_score(y_test, pred),
                 "PR-AUC": average_precision_score(y_test, p)}
print(pd.DataFrame(cmp).T.round(3))
```

Kết quả

	F1	Recall	Precision	PR-AUC
XGBoost, t = 0,5	0.467	0.361	0.662	0.556
XGBoost + SMOTE, t = 0,5	0.507	0.453	0.576	0.547
XGBoost, t = 0,26	0.543	0.584	0.508	0.556

Đọc kết quả. Kết quả rất rõ ràng:

- **Dời ngưỡng thắng SMOTE** trên chính chỉ số mà SMOTE muốn cải thiện: F1 0,543 so với 0,507, Recall 0,584 so với 0,453.
- Dời ngưỡng **giữ nguyên PR-AUC** (0,556), vì mô hình không hề thay đổi; chỉ có quy tắc ra quyết định thay đổi. SMOTE làm PR-AUC giảm nhẹ (0,547).
- Dời ngưỡng đơn giản hơn nhiều: không sinh dữ liệu giả, không thêm siêu tham số k, không tốn thêm thời gian huấn luyện.

2.7.4 Bước 18. SMOTE làm "phình" xác suất

Mục đích: hiểu *vì sao* SMOTE gần giống dời ngưỡng.

Ô lệnh 18 · SMOTE làm "phình" xác suất dự đoán code/ch02/s18_xac_suat.py

```
print(f"Tỷ lệ vỡ nợ thật trên tập test        : {y_test.mean():.3f}")
print(f"Xác suất vỡ nợ TB -- XGBoost          : {proba_base.mean():.3f}")
print(f"Xác suất vỡ nợ TB -- XGBoost + SMOTE  : {p_smote.mean():.3f}")
```

Kết quả

```
Tỷ lệ vỡ nợ thật trên tập test        : 0.221
Xác suất vỡ nợ TB -- XGBoost          : 0.222
Xác suất vỡ nợ TB -- XGBoost + SMOTE  : 0.325
```

Đọc kết quả. Mô hình gốc cho xác suất vỡ nợ trung bình 0,222, khớp với tỷ lệ thật 0,221. Ta nói mô hình được **hiệu chuẩn** (*calibrated*) tốt. Sau SMOTE, xác suất trung bình tăng lên 0,325, cao hơn thực tế gần 50%.

Giải thích. Khi huấn luyện với SMOTE, mô hình "tưởng" rằng một nửa số khách hàng vỡ nợ (tỷ lệ 50% sau cân bằng), thay vì 22% như thực tế. Vì vậy nó ước lượng mọi xác suất cao hơn. Một cách xấp xỉ, gọi $\text{odds}(\mathbf{x}) = \mathbb{P}(y=1 \mid \mathbf{x})/\mathbb{P}(y=0 \mid \mathbf{x})$ là *tỷ số cược*. Khi tỷ lệ lớp dương trong dữ liệu huấn luyện đổi từ π sang π', tỷ số cược mà mô hình học được bị nhân lên:

$$\text{odds}'(\mathbf{x}) \approx \text{odds}(\mathbf{x}) \cdot \frac{\pi'/(1-\pi')}{\pi/(1-\pi)}. \tag{2.2}$$

Với $\pi = 0{,}221$ và $\pi' = 0{,}5$, hệ số nhân là $\dfrac{0{,}5/0{,}5}{0{,}221/0{,}779} = \dfrac{1}{0{,}284} \approx 3{,}5$.

Ví dụ: một khách hàng có xác suất vỡ nợ thật 0,2, tức odds $= 0{,}2/0{,}8 = 0{,}25$. Sau SMOTE, odds xấp xỉ $0{,}25 \times 3{,}5 = 0{,}88$, tương ứng xác suất $0{,}88/1{,}88 \approx 0{,}47$. Người này vẫn chưa vượt ngưỡng 0,5, nhưng đã rất gần. Khi *mọi* xác suất đều bị đẩy lên như vậy, việc giữ ngưỡng 0,5 tương đương với việc *hạ ngưỡng* trên mô hình gốc. Đó là lý do SMOTE trông giống dời ngưỡng.

Thực tế, mức "phình" quan sát được ($0{,}222 \to 0{,}325$) nhỏ hơn mức lý thuyết, vì công thức trên giả định các điểm tổng hợp có cùng phân bố với điểm thật, điều không hoàn toàn đúng. Nhưng xu hướng là như nhau.

Vậy SMOTE còn hữu ích không?

Có, trong một số trường hợp:

1. hệ thống triển khai không cho phép chỉnh ngưỡng, hoặc mô hình không đưa ra xác suất;
2. lớp thiểu số *rất* nhỏ (dưới khoảng 5%), khiến mô hình gần như không học được gì về nó;
3. mô hình nhạy với phân bố lớp như kNN, SVM, hoặc cây quyết định đơn lẻ.

Trong bài báo, hãy luôn so sánh SMOTE với phương án dời ngưỡng để chứng minh SMOTE thực sự có ích [21].

2.8 Giai đoạn 7 – SMOTE và biến phân loại

2.8.1 Bước 19. SMOTE "phá hỏng" biến phân loại

Mục đích: phát hiện một vấn đề thường bị bỏ qua: (2.1) nội suy trên *mọi* cột, kể cả cột chỉ nên nhận số nguyên.

Ô lệnh 19 · SMOTE "phá hỏng" các biến phân loại code/ch02/s19_bien_phan_loai.py

```
X_res_raw, _ = SMOTE(random_state=SEED).fit_resample(X_train, y_train)  # dữ liệu gốc
↪ cho dễ đọc
syn_raw = X_res_raw.iloc[len(X_train):]
print("Tỷ lệ mẫu tổng hợp có EDUCATION không phải số nguyên:",
      f"{(syn_raw['EDUCATION'] % 1 != 0).mean():.1%}")
print(syn_raw[["SEX", "EDUCATION", "MARRIAGE", "PAY_1"]].head(5).round(2))
```

Kết quả

```
Tỷ lệ mẫu tổng hợp có EDUCATION không phải số nguyên: 57.1%
        SEX  EDUCATION  MARRIAGE  PAY_1
24000  1.11       1.89       2.0   0.00
24001  2.00       2.00       2.0   1.11
24002  1.75       1.25       1.0   1.49
24003  2.00       2.91       1.0   0.00
24004  2.00       3.00       1.1  -1.00
```

Đọc kết quả. Có 57,1% khách hàng tổng hợp có `EDUCATION` *không phải số nguyên*. Ví dụ, khách hàng tổng hợp đầu tiên có `SEX` = 1,11 và `EDUCATION` = 1,89. Một khách hàng có giới tính "1,11" và học vấn "1,89" là vô nghĩa. Nguyên nhân: nếu cha mẹ có `EDUCATION` lần lượt là 1 và 2, điểm nằm giữa sẽ có giá trị như 1,3 hay 1,89.

Với mô hình cây như XGBoost, các giá trị lẻ này thường ít gây hại vì cây chỉ hỏi "`EDUCATION` < 1,5?". Nhưng chúng làm dữ liệu huấn luyện mất ý nghĩa, và có thể gây hại với các mô hình khác.

2.8.2 Bước 20. Dùng SMOTE-NC cho dữ liệu hỗn hợp

Mục đích: xử lý đúng các biến phân loại khi sinh mẫu.

SMOTE-NC (*SMOTE for Nominal and Continuous*) là phiên bản của SMOTE cho dữ liệu có cả biến số lẫn biến phân loại. Nó vẫn nội suy các biến số như cũ, nhưng với biến phân loại, nó lấy giá trị **xuất hiện nhiều nhất** trong các láng giềng.

Ô lệnh 20 · SMOTE-NC: phiên bản dành cho dữ liệu có biến phân loại code/ch02/s20_smotenc.py

```
from imblearn.over_sampling import SMOTENC

cat_cols = ["SEX", "EDUCATION", "MARRIAGE"]
smnc = SMOTENC(categorical_features=[X_train.columns.get_loc(c) for c in cat_cols],
               random_state=SEED)
X_nc, y_nc = smnc.fit_resample(X_train, y_train)
syn_nc = X_nc.iloc[len(X_train):]
print("Tỷ lệ mẫu tổng hợp có EDUCATION không phải số nguyên:",
      f"{(syn_nc['EDUCATION'] % 1 != 0).mean():.1%}")
print(syn_nc[cat_cols + ["PAY_1"]].head(5).round(2))
```

Kết quả

```
Tỷ lệ mẫu tổng hợp có EDUCATION không phải số nguyên: 0.0%
       SEX  EDUCATION  MARRIAGE  PAY_1
24000  1.0        2.0       2.0   0.11
```

```
24001  2.0        2.0       2.0   1.11
24002  1.0        1.0       1.0   1.49
24003  2.0        3.0       1.0   0.00
24004  2.0        3.0       1.0  -0.90
```

Đọc kết quả.

- Tham số `categorical_features` của SMOTE-NC nhận danh sách *vị trí* (số thứ tự) của các cột phân loại. Lệnh `columns.get_loc` trả về vị trí của một cột; ví dụ, vị trí của `SEX` là 1.
- Kết quả: 0% giá trị lẻ. `SEX`, `EDUCATION`, `MARRIAGE` đều là số nguyên hợp lệ.
- `PAY_1` vẫn có giá trị lẻ (ví dụ 1,11), vì ta chưa khai báo nó là biến phân loại. Đây là một *lựa chọn*: `PAY_1` có thứ tự ($-2 < -1 < 0 < 1 < \dots$), nên có thể coi là biến số. Nếu muốn giữ nó nguyên, hãy thêm vào danh sách `cat_cols`.

Thử tự làm

Thay SMOTE bằng SMOTE-NC trong hàm `make` của Ô lệnh 11 (lưu ý: SMOTE-NC cần biết vị trí cột, nên hãy đặt nó *trước* bước chuẩn hóa, hoặc chuẩn hóa chỉ các cột số). F1 và PR-AUC có thay đổi đáng kể không?

2.9 Tổng kết

Ưu điểm của SMOTE	Hạn chế của SMOTE
Đơn giản, nhanh, có sẵn trong `imblearn`	Không cải thiện khả năng xếp hạng (PR-AUC, ROC-AUC)
Không chép trùng như nhân bản, nên giảm quá khớp	Trên dữ liệu này, không thắng được phương án dời ngưỡng
Mở rộng vùng quyết định của lớp thiểu số	Sinh điểm cả ở vùng chồng lấn giữa hai lớp
Là nền tảng của hàng trăm biến thể	Làm "phình" xác suất dự đoán (mất hiệu chuẩn)
	Cần chuẩn hóa; cần SMOTE-NC khi có biến phân loại

Ba điều cần nhớ nhất

1. SMOTE phải nằm **bên trong pipeline của `imblearn`**. Đặt sai chỗ làm PR-AUC bị thổi phồng từ 0,54 lên 0,93.
2. SMOTE chủ yếu **dịch chuyển sự đánh đổi** giữa Precision và Recall, giống như hạ ngưỡng quyết định.
3. Luôn so sánh SMOTE với phương án **dời ngưỡng** trước khi kết luận SMOTE có ích.

2.10 Những lỗi thường gặp

Lỗi thường gặp

1. **Chạy SMOTE trước khi chia train/test hoặc trước khi chia fold** (Bước 13).
2. **Dùng `sklearn.pipeline.Pipeline`** thay vì `imblearn.pipeline.Pipeline`. Pipeline của scikit-learn không biết cách xử lý bước lấy mẫu.
3. **Không chuẩn hóa trước SMOTE** khi các đặc trưng có thang đo rất khác nhau (Bước 9).
4. **Chỉ báo cáo F1 hoặc Accuracy tại ngưỡng** 0,5, không báo cáo PR-AUC (Bước 11).
5. **Không so sánh với phương án dời ngưỡng** (Bước 17).
6. **Dùng xác suất của mô hình có SMOTE như xác suất thật**, ví dụ để tính dự phòng rủi ro (Bước 18).
7. **Dùng SMOTE cho biến phân loại** mà không dùng SMOTE-NC (Bước 19).

2.11 Khi viết bài báo khoa học

Gợi ý cho bài báo

Ba hướng đóng góp hợp lý cho một bài báo nhập môn có dùng SMOTE:

- **Nghiên cứu so sánh có kiểm soát**: SMOTE và các biến thể (Chương 3 and 4), gán trọng số lớp (Chương 5) và dời ngưỡng, trên nhiều mô hình và nhiều mức mất cân bằng, kèm kiểm định thống kê [6].
- **Tác động của SMOTE lên lời giải thích mô hình**: kết quả SHAP thay đổi thế nào khi mô hình học trên dữ liệu có SMOTE?
- **Áp dụng vào dữ liệu thực ở Việt Nam**: điểm chính là mô tả kỹ quy trình để chứng minh không có rò rỉ.

Mẫu câu cho phần phương pháp: *"To prevent information leakage, SMOTE [4] was applied exclusively within the training folds using an `imblearn` pipeline; validation and test folds retained the original class distribution."*

Người phản biện thường hỏi gì?

- *"Was oversampling applied before or after the train/test split?"*
- *"Did you compare against threshold moving or cost-sensitive learning?"*
- *"How were categorical features handled during interpolation?"*

2.12 Bài tập

Mức 1 – Làm quen.

1. Dùng hàm `smote_tay` của Ô lệnh 7 với `k=1`, `k=5`, `k=20`, rồi vẽ kết quả lên cùng một hình với dữ liệu đồ chơi. Mô tả sự khác biệt.

2. Kiểm tra lại: điểm mới ở Ô lệnh 6 có thực sự nằm trên đoạn thẳng nối $\mathbf{x}_i$ và $\mathbf{x}_{\text{lg}}$ không? (Gợi ý: tính $(\mathbf{x}_{\text{mới}} - \mathbf{x}_i)/(\mathbf{x}_{\text{lg}} - \mathbf{x}_i)$ cho từng tọa độ; cả hai phải bằng λ.)

3. Thay SMOTE bằng `RandomOverSampler` trong hàm `make` của Ô lệnh 11. So sánh F1 và PR-AUC.

Mức 2 – Vận dụng.

4. Thay tham số `sampling_strategy` của SMOTE lần lượt bằng 0,5, 0,75 và 1,0 (tỷ lệ lớp thiểu số so với lớp đa số sau khi sinh). Vẽ F1 và PR-AUC theo tham số này.

5. Bỏ bước `StandardScaler` khỏi pipeline. Kết quả của LogReg và XGBoost thay đổi thế nào? Giải thích dựa trên Bước 9.

6. Lặp lại Bước 16 nhưng tìm ngưỡng cho Recall cao nhất với điều kiện Precision $\geq 0{,}5$.

Mức 3 – Nâng cao.

7. Tạo phiên bản Credit Default có tỷ lệ vỡ nợ chỉ 5% (bỏ ngẫu nhiên bớt người vỡ nợ *trong tập train*). Lặp lại so sánh SMOTE với dời ngưỡng. Kết luận có thay đổi không?

8. Kiểm chứng (2.2): lấy xác suất $\hat{p}$ của mô hình gốc, áp dụng công thức để tính xác suất "dự kiến sau SMOTE", rồi so sánh với xác suất thật của mô hình có SMOTE (vẽ biểu đồ phân tán).

2.13 Đáp án phần Tự kiểm tra

Câu 1.

Hạ ngưỡng thì mô hình gắn cờ nhiều người hơn, nên **số người bị bỏ sót giảm** (Recall tăng). Đổi lại, số người bị báo động nhầm tăng, nên **Precision giảm**. Bước 18 của Chương 1 cho ví dụ cụ thể với hồi quy logistic.

Câu 2.

Chưa đầy đủ. Cần hỏi thêm: (1) PR-AUC và ROC-AUC có tăng không, hay chỉ có F1 tại ngưỡng 0,5 tăng? (Bước 11: PR-AUC giảm nhẹ.) (2) Precision thay đổi thế nào? (3) Nếu chỉ dời ngưỡng của mô hình gốc thì F1 đạt bao nhiêu? (Bước 17: 0,543, cao hơn SMOTE.) (4) SMOTE được đặt trong hay ngoài pipeline?

Thuật ngữ của chương

Tiếng Việt	Tiếng Anh	Nghĩa ngắn gọn
Nhân bản ngẫu nhiên	random oversampling	Chép lại ngẫu nhiên các điểm thiểu số
Mẫu tổng hợp	synthetic sample	Điểm dữ liệu "giả" do thuật toán tạo ra
Láng giềng gần nhất	nearest neighbour	Điểm có khoảng cách nhỏ nhất đến điểm đang xét
Nội suy	interpolation	Tạo giá trị nằm giữa hai giá trị đã biết
Quá khớp	overfitting	Học thuộc dữ liệu huấn luyện, kém trên dữ liệu mới
Dự đoán out-of-fold	out-of-fold prediction	Dự đoán cho mỗi mẫu bởi mô hình chưa từng thấy mẫu đó
Dời ngưỡng	threshold moving	Đổi ngưỡng quyết định thay vì đổi mô hình
Hiệu chuẩn	calibration	Xác suất dự đoán khớp với tần suất thực tế
Tỷ số cược	odds	$\mathbb{P}(y=1)/\mathbb{P}(y=0)$

Tài liệu đọc thêm

Bài báo gốc của Chawla et al. [4]. Tổng quan về học từ dữ liệu mất cân bằng của He and Garcia [10]. Thư viện `imbalanced-learn` [15]. Phân tích phê phán tác động của các kỹ thuật cân bằng lên mô hình dự đoán rủi ro của van den Goorbergh et al. [21].

CHƯƠNG 3

Borderline-SMOTE – Chỉ sinh mẫu ở vùng biên

Chương này giúp bạn làm được gì?

Sau khi làm theo hết các bước trong chương, bạn sẽ:

- phân loại được mỗi điểm thiểu số thành safe, danger hoặc noise bằng cách "hỏi" các láng giềng của nó;
- tự viết Borderline-SMOTE bằng cách sửa *một dòng* trong hàm SMOTE của Chương 2;
- phân biệt Borderline-SMOTE-1 với Borderline-SMOTE-2, và đo được mức "lấn" vào lớp đa số của từng cách;
- dùng tỷ lệ safe/danger/noise để *chẩn đoán* mức độ khó của một bộ dữ liệu;
- tự kiểm chứng xem "tập trung vào vùng biên" có thật sự giúp ích hay không.

3.1 Trước khi bắt đầu

Bạn cần gì?

Chương này dùng tiếp mọi thứ của Chương 2: thư viện `imbalanced-learn`, bộ dữ liệu "đồ chơi" 2 chiều, và hàm SMOTE tự viết. Nếu chưa làm Chương 2, hãy làm trước, đặc biệt là Bước 5–8.

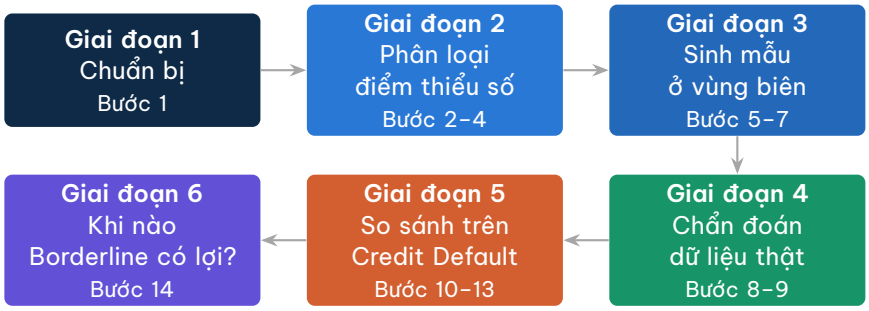
Ý tưởng của chương

SMOTE đối xử với *mọi* điểm thiểu số như nhau: điểm nào cũng có cơ hội làm "hạt giống" để sinh mẫu mới. Nhưng các điểm không quan trọng như nhau:

- Một khách hàng vỡ nợ mà xung quanh *toàn* là người vỡ nợ: mô hình nào cũng nhận ra. Sinh thêm mẫu quanh người này gần như vô ích.
- Một khách hàng vỡ nợ mà xung quanh có cả người vỡ nợ lẫn người trả nợ tốt: đây là **vùng biên**, nơi mô hình hay sai nhất.

Han et al. [9] đề xuất **Borderline-SMOTE**: *chỉ sinh mẫu từ những điểm nằm ở vùng biên*. Giống một giáo viên ôn thi: không luyện thêm những dạng bài học sinh đã làm đúng, mà tập trung vào những câu học sinh hay nhầm.

3.2 Giai đoạn 1 – Chuẩn bị



Hình 3.1: Lộ trình của Chương 3: 6 giai đoạn, 14 bước.

3.2.1 Bước 1. Nạp dữ liệu

Mục đích: có đồng thời dữ liệu thật (Credit Default) và dữ liệu đồ chơi 2 chiều của Chương 2.

Ô lệnh 1 · Nạp thư viện, dữ liệu Credit Default và bộ dữ liệu "đồ chơi" của Chương 2

code/ch03/s01_chuan_bi.py

```
import sys
sys.path.append("code")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.neighbors import NearestNeighbors
from common import credit_split, xgb_clf

pd.set_option("display.width", 80)
pd.set_option("display.precision", 3)
SEED = 42

X_train, X_test, y_train, y_test = credit_split()
X_toy, y_toy = make_classification(n_samples=600, n_features=2, n_redundant=0,
                                   n_clusters_per_class=1, weights=[0.9, 0.1],
                                   class_sep=1.0, random_state=3)
print("Credit -- train:", X_train.shape, "| Đồ chơi:", X_toy.shape, np.bincount(y_toy))
```

Kết quả

```
Credit -- train: (24000, 23) | Đồ chơi: (600, 2) [534  66]
```

Đọc kết quả. Hai bộ dữ liệu giống hệt Chương 2: tập train Credit Default có 24 000 khách hàng và 23 đặc trưng; bộ đồ chơi có 600 điểm, trong đó 534 điểm lớp 0 và 66 điểm lớp 1. Bộ đồ chơi dùng để *nhìn* thuật toán; bộ thật dùng để *đánh giá*.

3.3 Giai đoạn 2 – Phân loại các điểm thiểu số

3.3.1 Bước 2. "Hỏi" các láng giềng

Mục đích: hiểu cách Borderline-SMOTE quyết định một điểm có nằm ở vùng biên hay không.

Với mỗi điểm thiểu số, ta tìm $m = 10$ láng giềng gần nhất trong **toàn bộ** dữ liệu (cả hai lớp), rồi đếm xem có bao nhiêu láng giềng thuộc lớp đa số. Lưu ý điểm khác với SMOTE: ở Chương 2, ta chỉ tìm láng giềng *trong lớp thiểu số*.

Ô lệnh 2 · "Hỏi" 10 láng giềng của một điểm thiểu số xem chúng thuộc lớp nào

code/ch03/s02_hoi_lang_gieng.py

```
m = 10
nn_all = NearestNeighbors(n_neighbors=m + 1).fit(X_toy)    # tìm trong TOÀN BỘ dữ liệu
min_idx = np.where(y_toy == 1)[0]                          # vị trí 66 điểm thiểu số

for i in min_idx[[1, 32, 21]]:                             # xét 3 điểm làm ví dụ
    nb = nn_all.kneighbors(X_toy[[i]], return_distance=False)[0][1:]   # bỏ chính nó
    m_maj = (y_toy[nb] == 0).sum()                         # số láng giềng thuộc lớp đa số
    print(f"Điểm {i:3d}: nhãn láng giềng = {y_toy[nb]} -> {m_maj}/10 thuộc lớp đa số")
```

Kết quả

```
Điểm  23: nhãn láng giềng = [1 1 1 1 1 0 1 0 0 1] -> 3/10 thuộc lớp đa số
Điểm 279: nhãn láng giềng = [1 0 0 1 1 0 0 1 0 1] -> 5/10 thuộc lớp đa số
Điểm 162: nhãn láng giềng = [0 0 0 0 0 0 0 0 0 0] -> 10/10 thuộc lớp đa số
```

Đọc kết quả. Mỗi dòng in ra nhãn của 10 láng giềng của một điểm thiểu số (1 = thiểu số, 0 = đa số):

- **Điểm 23**: chỉ 3/10 láng giềng thuộc lớp đa số. Điểm này nằm trong "lãnh thổ" của lớp thiểu số, nên được coi là **an toàn**.
- **Điểm 279**: 5/10 láng giềng thuộc lớp đa số, tức nửa nọ nửa kia. Điểm này nằm ở **vùng biên**.
- **Điểm 162**: cả 10/10 láng giềng đều thuộc lớp đa số. Điểm này "lạc" giữa lớp đa số; có thể là **nhiễu** hoặc ghi nhãn sai.

Lệnh `kneighbors(..., return_distance=False)` chỉ trả về chỉ số của các láng giềng, không trả về khoảng cách. `[0][1:]` bỏ phần tử đầu tiên vì đó là chính điểm đang xét.

3.3.2 Bước 3. Quy tắc phân loại

Mục đích: áp dụng một quy tắc thống nhất cho mọi điểm thiểu số.

Gọi m' là số láng giềng (trong m) thuộc lớp đa số. Han et al. [9] phân loại như sau:

$$\text{loại của điểm} = \begin{cases} \text{noise (nhiễu)}, & m' = m \quad \text{(mọi láng giềng đều thuộc lớp đa số)}, \\ \text{danger (vùng biên)}, & m/2 \le m' < m, \\ \text{safe (an toàn)}, & m' < m/2. \end{cases} \tag{3.1}$$

Với $m = 10$: $m' = 0$–4 là safe; $m' = 5$–9 là danger; $m' = 10$ là noise.

Ô lệnh 3 · Phân loại mọi điểm thiểu số thành SAFE / DANGER / NOISE

code/ch03/s03_phan_loai_diem.py

```
def minority_types(X, y, m=10):
    """Trả về loại của từng điểm thiểu số theo quy tắc của Han và cộng sự (2005)."""
    nn = NearestNeighbors(n_neighbors=m + 1).fit(X)
    nb = nn.kneighbors(X[y == 1], return_distance=False)[:, 1:]
    m_maj = (y[nb] == 0).sum(axis=1)
    types = np.where(m_maj == m, "NOISE",
                     np.where(m_maj >= m / 2, "DANGER", "SAFE"))
    return types, m_maj

types_toy, mmaj_toy = minority_types(X_toy, y_toy)
print(pd.Series(types_toy).value_counts().to_dict())
```

Kết quả

```
{'SAFE': 47, 'DANGER': 10, 'NOISE': 9}
```

Đọc kết quả. Hàm `minority_types` làm cho *mọi* điểm điều mà Bước 2 làm cho ba điểm. Hai lệnh `np.where` lồng nhau chính là (3.1): "nếu $m' = m$ thì NOISE, nếu không thì (nếu $m' \geq m/2$ thì DANGER, nếu không thì SAFE)".

Trong 66 điểm thiểu số của bộ đồ chơi: 47 điểm an toàn, 10 điểm ở vùng biên, 9 điểm nhiễu. Borderline-SMOTE sẽ chỉ dùng **10 điểm** danger để sinh mẫu.

3.3.3 Bước 4. Nhìn ba loại điểm trên mặt phẳng

Mục đích: kiểm tra bằng mắt xem phép phân loại có hợp lý không.

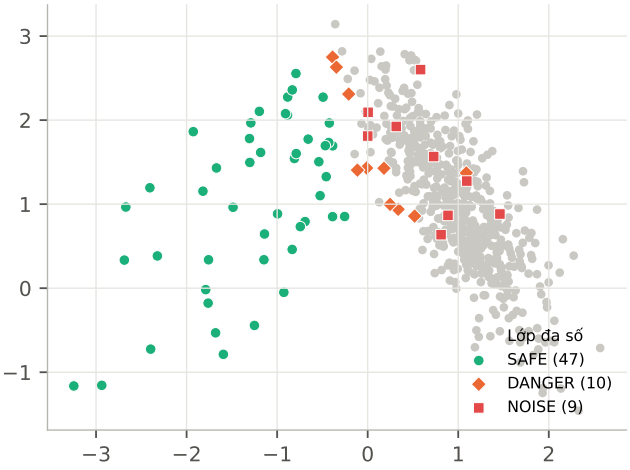
Ô lệnh 4 · Vẽ ba loại điểm thiểu số lên mặt phẳng code/ch03/s04_ve_loai_diem.py

```
X_min = X_toy[y_toy == 1]
style = {"SAFE": ("#1baf7a", "o"), "DANGER": ("#eb6834", "D"), "NOISE": ("#e34948",
↪  "s")}

fig, ax = plt.subplots(figsize=(4.6, 3.4))
ax.scatter(*X_toy[y_toy == 0].T, s=8, color="#c9c8c2", label="Lớp đa số")
for t, (c, mk) in style.items():
    ax.scatter(*X_min[types_toy == t].T, s=22, color=c, marker=mk, edgecolor="white",
               linewidth=0.4, label=f"{t} ({(types_toy == t).sum()})")
ax.legend(fontsize=7, loc="lower right")
plt.savefig("figures/ch03_s04_types.pdf", bbox_inches="tight")
```

Đọc kết quả. Hình cho thấy phép phân loại rất hợp lý:

- Các điểm safe (xanh lá) tập trung ở bên trái, xa đám điểm xám của lớp đa số.
- Các điểm danger (cam) nằm dọc theo "đường ranh giới" với lớp đa số.
- Các điểm noise (đỏ) nằm lọt thỏm giữa đám điểm xám.



Hình 3.2: Ba loại điểm thiểu số trên dữ liệu đồ chơi ($m = 10$). Xanh lá (safe): nằm sâu trong vùng thiểu số. Cam (danger): nằm sát lớp đa số. Đỏ (noise): lạc hẳn vào vùng đa số.

Tự kiểm tra

Câu 1. Nếu tăng m từ 10 lên 30, bạn nghĩ số điểm noise sẽ tăng hay giảm? Vì sao? (Gợi ý: để là noise, *tất cả* m láng giềng phải thuộc lớp đa số.)

3.4 Giai đoạn 3 – Sinh mẫu ở vùng biên

3.4.1 Bước 5. Tự viết Borderline-SMOTE-1

Mục đích: thấy rằng Borderline-SMOTE chỉ khác SMOTE đúng một chỗ.

So sánh với hàm `smote_tay` của Chương 2 (Ô lệnh 7):

Ô lệnh 5 · Tự viết Borderline-SMOTE-1: chỉ sinh mẫu từ các điểm DANGER `code/ch03/s05_tu_viet.py`

```
def borderline_smote_tay(X_min, types, G, k=5, seed=SEED):
    rng = np.random.default_rng(seed)
    _, idx = NearestNeighbors(n_neighbors=k + 1).fit(X_min).kneighbors(X_min)
    danger = np.where(types == "DANGER")[0]          # chỉ những điểm này được làm "hạt
    ↪ giống"
    new = []
    for _ in range(G):
        i = rng.choice(danger)                       # KHÁC SMOTE: chỉ chọn trong DANGER
        j = rng.choice(idx[i][1:])                   # láng giềng vẫn là mọi điểm thiểu số
        lam = rng.uniform(0, 1)
        new.append(X_min[i] + lam * (X_min[j] - X_min[i]))
    return np.array(new)

G = (y_toy == 0).sum() - (y_toy == 1).sum()
X_syn_b1 = borderline_smote_tay(X_min, types_toy, G)
print("Số điểm DANGER làm hạt giống:", (types_toy == "DANGER").sum(), "| số điểm sinh
↪ ra:", len(X_syn_b1))
```

Kết quả

```
Số điểm DANGER làm hạt giống: 10 | số điểm sinh ra: 468
```

Đọc kết quả. Hàm gần như giống hệt `smote_tay`. Khác biệt duy nhất nằm ở dòng chọn điểm hạt giống:

- SMOTE: `i = rng.integers(len(X_min))`, tức chọn ngẫu nhiên trong *tất cả* 66 điểm thiểu số;
- Borderline-SMOTE-1: `i = rng.choice(danger)`, tức chỉ chọn trong *10 điểm* danger.

Phần còn lại giữ nguyên: láng giềng của hạt giống vẫn được chọn trong mọi điểm thiểu số, và vẫn dùng công thức nội suy (2.1). Số điểm cần sinh vẫn là $G = 534 - 66 = 468$, nhưng giờ chúng chỉ xuất phát từ 10 điểm, trung bình mỗi điểm làm hạt giống khoảng 47 lần.

Borderline-SMOTE-2. Biến thể thứ hai khác thêm một chỗ: láng giềng được chọn trong *cả hai lớp*. Nếu láng giềng được chọn thuộc lớp đa số, hệ số λ bị giới hạn trong khoảng $(0;\ 0{,}5)$, để điểm mới vẫn nằm gần điểm thiểu số hơn:

$$\mathbf{x}_{\text{mới}} = \mathbf{x}_i + \lambda(\mathbf{x}_{\text{lg}} - \mathbf{x}_i), \qquad \lambda \text{ trong } \begin{cases} (0;\ 1) & \text{nếu } \mathbf{x}_{\text{lg}} \text{ thuộc lớp thiểu số,} \\ (0;\ 0{,}5) & \text{nếu } \mathbf{x}_{\text{lg}} \text{ thuộc lớp đa số.} \end{cases} \tag{3.2}$$

Nói cách khác, Borderline-2 "dám" đặt điểm mới đi về phía lớp đa số, nhưng không quá nửa đường.

3.4.2 Bước 6. So sánh ba thuật toán bằng hình

Mục đích: nhìn thấy sự khác biệt giữa SMOTE, Borderline-1 và Borderline-2.

Ô lệnh 6 · So sánh SMOTE, Borderline-1 và Borderline-2 (thư viện imblearn)

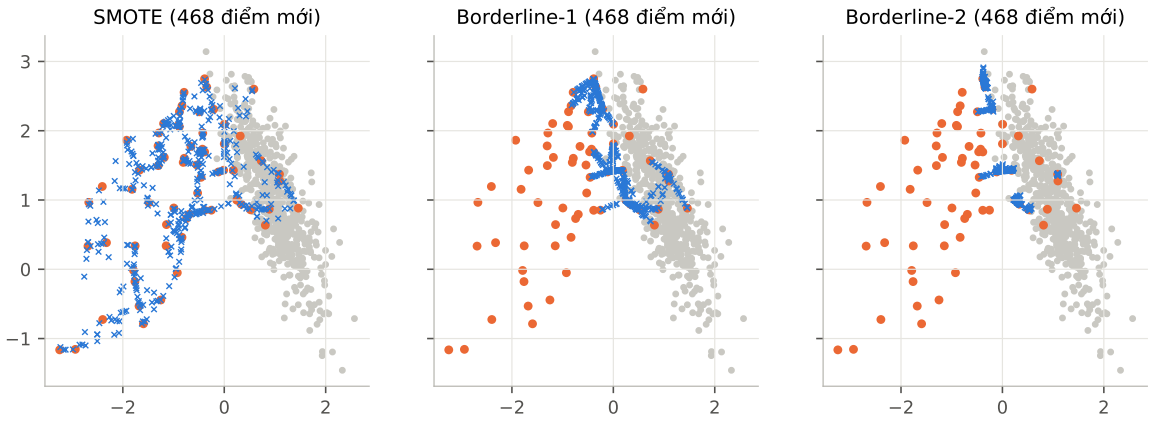
code/ch03/s06_so_sanh_hinh.py

```python
from imblearn.over_sampling import SMOTE, BorderlineSMOTE

samplers = {"SMOTE": SMOTE(random_state=SEED),
            "Borderline-1": BorderlineSMOTE(kind="borderline-1", random_state=SEED),
            "Borderline-2": BorderlineSMOTE(kind="borderline-2", random_state=SEED)}
synth = {}
fig, axes = plt.subplots(1, 3, figsize=(10, 3.2), sharex=True, sharey=True)
for ax, (name, s) in zip(axes, samplers.items()):
    Xr, yr = s.fit_resample(X_toy, y_toy)
    synth[name] = Xr[len(X_toy):]                    # các điểm mới nằm ở cuối mảng
    ax.scatter(*X_toy[y_toy == 0].T, s=6, color="#c9c8c2")
    ax.scatter(*X_min.T, s=12, color="#eb6834")
    ax.scatter(*synth[name].T, s=8, marker="x", color="#2a78d6", linewidth=0.7)
    ax.set_title(f"{name} ({len(synth[name])} điểm mới)")
plt.savefig("figures/ch03_s06_compare.pdf", bbox_inches="tight")
```

Đọc kết quả. Cả ba đều sinh 468 điểm, nhưng ở những nơi rất khác nhau:

- **SMOTE**: rải điểm khắp vùng thiểu số, cả bên trái (vùng an toàn) lẫn vùng giữa.



Hình 3.3: Điểm tổng hợp (dấu × xanh) do ba thuật toán sinh ra trên cùng dữ liệu đồ chơi. Chấm cam: điểm thiểu số gốc; chấm xám: lớp đa số.

- **Borderline-1**: *không có* điểm nào ở vùng an toàn bên trái. Mọi điểm mới dồn thành những dải dày đặc dọc theo ranh giới.
- **Borderline-2**: dồn vào ranh giới còn mạnh hơn, và nhiều điểm đi hẳn vào đám điểm xám.

3.4.3 Bước 7. Đo mức "lấn" vào lớp đa số

Mục đích: biến nhận xét bằng mắt ở Bước 6 thành con số.

Với mỗi điểm tổng hợp, ta tìm 10 láng giềng của nó trong dữ liệu *gốc*, rồi tính tỷ lệ láng giềng thuộc lớp đa số. Tỷ lệ càng cao, điểm mới càng nằm sâu trong vùng của lớp đa số.

Ô lệnh 7 · Đo xem điểm tổng hợp "lấn" vào vùng của lớp đa số đến mức nào `code/ch03/s07_do_lan.py`

```
nn10 = NearestNeighbors(n_neighbors=10).fit(X_toy)       # chỉ dùng dữ liệu GỐC
for name, S_ in synth.items():
    nb = nn10.kneighbors(S_, return_distance=False)
    share_maj = (y_toy[nb] == 0).mean(axis=1)             # tỷ lệ láng giềng thuộc lớp
    ↪ đa số
    print(f"{name:13s}: TB {share_maj.mean():.0%} láng giềng là lớp đa số;"
          f" {np.mean(share_maj > 0.5):.0%} điểm nằm hẳn trong vùng đa số")
```

Kết quả

```
SMOTE        : TB 23% láng giềng là lớp đa số; 23% điểm nằm hẳn trong vùng đa số
Borderline-1 : TB 62% láng giềng là lớp đa số; 72% điểm nằm hẳn trong vùng đa số
Borderline-2 : TB 68% láng giềng là lớp đa số; 87% điểm nằm hẳn trong vùng đa số
```

Đọc kết quả.

	TB tỷ lệ láng giềng là lớp đa số	% điểm nằm hẳn trong vùng đa số
SMOTE	23%	23%
Borderline-1	62%	72%
Borderline-2	68%	87%

"Nằm hẳn trong vùng đa số" nghĩa là hơn một nửa láng giềng thuộc lớp đa số. Với Borderline-2, gần 9/10 điểm mới nằm trong vùng mà lớp đa số chiếm ưu thế.

Đây vừa là **mục đích**, vừa là **rủi ro** của Borderline-SMOTE:

- Mục đích: "đẩy" ranh giới quyết định về phía lớp đa số, để mô hình gắn nhãn thiểu số mạnh tay hơn ở vùng biên.
- Rủi ro: nếu vùng biên thực chất là vùng hai lớp *trộn lẫn* vào nhau, các điểm mới sẽ nằm giữa rất nhiều điểm đa số thật. Mô hình bị "dạy" rằng vùng đó thuộc lớp thiểu số, và sẽ báo động nhầm nhiều hơn.

3.5 Giai đoạn 4 – Chẩn đoán dữ liệu thật

3.5.1 Bước 8. Khách hàng vỡ nợ thuộc loại nào?

Mục đích: dùng phép phân loại safe/danger/noise như một công cụ *chẩn đoán*: trước khi chọn kỹ thuật, hãy hiểu dữ liệu khó đến mức nào.

Ô lệnh 8 · Chẩn đoán dữ liệu thật: khách hàng vỡ nợ thuộc loại nào?

code/ch03/s08_chan_doan_credit.py

```
from sklearn.preprocessing import StandardScaler

Z_train = StandardScaler().fit_transform(X_train)          # chuẩn hóa trước khi tính
↪ khoảng cách
types_c, mmaj_c = minority_types(Z_train, y_train.values)
ratio = pd.Series(types_c).value_counts(normalize=True)
print("Credit Default:", (ratio * 100).round(1).to_dict(), "(%)")
toy_ratio = pd.Series(types_toy).value_counts(normalize=True)
print("Dữ liệu đồ chơi:", (toy_ratio * 100).round(1).to_dict(), "(%)")
```

Kết quả

```
Credit Default: {'DANGER': 61.4, 'SAFE': 29.0, 'NOISE': 9.6} (%)
Dữ liệu đồ chơi: {'SAFE': 71.2, 'DANGER': 15.2, 'NOISE': 13.6} (%)
```

Đọc kết quả. Nhớ chuẩn hóa trước khi tính khoảng cách (Chương 2, Bước 9). So sánh hai bộ dữ liệu:

	safe	danger	noise
Dữ liệu đồ chơi	**71,2**%	15,2%	13,6%
Credit Default	29,0%	**61,4**%	9,6%

Kết quả rất đáng chú ý. Trên Credit Default, **gần 2/3 số khách hàng vỡ nợ nằm ở vùng biên**: ít nhất một nửa số láng giềng gần nhất của họ là người *không* vỡ nợ. Nói cách khác, với 23 đặc trưng hiện có, phần lớn người vỡ nợ "trông giống" người trả nợ tốt.

Phát hiện này giải thích một điều ta đã thấy ở Chương 1: mọi mô hình, từ Random Forest, XGBoost đến LightGBM, đều dừng ở ROC-AUC khoảng 0,78. **Giới hạn nằm ở dữ liệu, không nằm ở thuật toán.**

3.5.2 Bước 9. Nhìn kỹ hơn: phân bố của m'

Mục đích: xem các khách hàng vỡ nợ phân bố thế nào theo mức "bị bao vây".

Ô lệnh 9 · Phân bố số láng giềng thuộc lớp đa số (m') của mọi khách hàng vỡ nợ

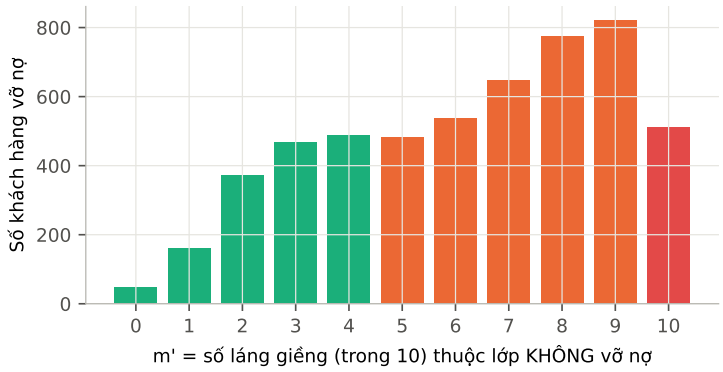
code/ch03/s09_phan_bo_mmaj.py

```
counts = pd.Series(mmaj_c).value_counts().sort_index()
print(counts.to_dict())

colors = ["#1baf7a"] * 5 + ["#eb6834"] * 5 + ["#e34948"]   # SAFE: 0-4, DANGER: 5-9,
↪  NOISE: 10
fig, ax = plt.subplots(figsize=(5.5, 2.6))
ax.bar(counts.index, counts.values, color=[colors[i] for i in counts.index])
ax.set_xticks(range(11))
ax.set_xlabel("m' = số láng giềng (trong 10) thuộc lớp KHÔNG vỡ nợ")
ax.set_ylabel("Số khách hàng vỡ nợ")
plt.savefig("figures/ch03_s09_mmaj.pdf", bbox_inches="tight")
```

Kết quả

```
{0: 49, 1: 162, 2: 371, 3: 469, 4: 488, 5: 481, 6: 536, 7: 646, 8: 775, 9: 820, 10: 512}
```



Hình 3.4: Số khách hàng vỡ nợ theo m', tức số láng giềng (trong 10) thuộc lớp không vỡ nợ. Xanh lá: safe; cam: danger; đỏ: noise.

Đọc kết quả. Biểu đồ *tăng dần* từ trái sang phải: càng nhiều láng giềng là người không vỡ nợ, càng nhiều khách hàng vỡ nợ rơi vào nhóm đó. Chỉ có 49 khách hàng vỡ nợ (khoảng 1%) mà cả 10 láng giềng đều cũng vỡ nợ ($m' = 0$). Trong khi đó, nhóm đông nhất là $m' = 9$ với 820 người: trong 10 láng giềng gần nhất của họ, 9 người không vỡ nợ.

Cộng các cột cam: $481 + 536 + 646 + 775 + 820 = 3\,258$ người, tức $3\,258/5\,309 = 61{,}4\%$, khớp với Bước 8.

Ý nghĩa với Borderline-SMOTE

Khi danger chiếm 61%, "vùng biên" thực chất là *phần lớn* lớp thiểu số. Borderline-SMOTE khi đó không còn "tập trung" vào đâu cả, mà chỉ bỏ qua khoảng 29% điểm an toàn và 10% điểm nhiễu. Ta có thể dự đoán: trên dữ liệu này, Borderline-SMOTE sẽ không khác SMOTE nhiều. Giai đoạn 5 sẽ kiểm chứng dự đoán này.

3.6 Giai đoạn 5 – So sánh trên Credit Default

3.6.1 Bước 10. Đánh giá chéo bốn chiến lược với hai mô hình

Mục đích: so sánh công bằng: không lấy mẫu, SMOTE, Borderline-1, Borderline-2.

Ô lệnh 10 · So sánh 4 chiến lược lấy mẫu với 2 mô hình (đánh giá chéo, 1 phút) code/ch03/s10_cv.py

```
from imblearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.linear_model import LogisticRegression

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
scoring = {"PR-AUC": "average_precision", "F1": "f1", "Recall": "recall", "Precision":
↪  "precision"}
strategies = {"Không lấy mẫu": None, **samplers}

def make(model, sampler):
    steps = [("scale", StandardScaler())]
    if sampler is not None:
        steps.append(("sampler", sampler))
    return Pipeline(steps + [("model", model)])

cv_rows = {}
for mname, mk in {"LogReg": lambda: LogisticRegression(max_iter=2000), "XGBoost":
↪  xgb_clf}.items():
    for sname, s in strategies.items():
        r = cross_validate(make(mk(), s), X_train, y_train, cv=skf, scoring=scoring)
        cv_rows[f"{mname} | {sname}"] = {k: r["test_" + k].mean() for k in scoring}
cv_df = pd.DataFrame(cv_rows).T
print(cv_df)
```

Kết quả

```
                        PR-AUC     F1  Recall  Precision
LogReg | Không lấy mẫu   0.506  0.366   0.246      0.714
LogReg | SMOTE           0.503  0.478   0.652      0.377
LogReg | Borderline-1    0.488  0.457   0.701      0.339
LogReg | Borderline-2    0.491  0.458   0.702      0.340
XGBoost | Không lấy mẫu  0.558  0.476   0.369      0.671
XGBoost | SMOTE          0.539  0.512   0.459      0.580
XGBoost | Borderline-1   0.531  0.518   0.500      0.538
XGBoost | Borderline-2   0.531  0.526   0.530      0.522
```

Đọc kết quả. Code giống hệt Ô lệnh 11 của Chương 2, chỉ thêm hai chiến lược Borderline. Hàm `make` vẫn đặt bước lấy mẫu *bên trong* pipeline của `imblearn`, nên không có rò rỉ dữ liệu.

Với XGBoost, đọc từ trên xuống theo thứ tự "càng tập trung vào biên":

XGBoost	Recall	Precision	F1	PR-AUC
Không lấy mẫu	0,369	0,671	0,476	**0,558**
SMOTE	0,459	0,580	0,512	0,539
Borderline-1	0,500	0,538	0,518	0,531
Borderline-2	**0,530**	0,522	**0,526**	0,531

Xu hướng rất đều đặn: Recall tăng dần, Precision giảm dần, PR-AUC giảm dần. Đúng như Bước 7 dự báo: dồn mẫu vào vùng biên làm mô hình "mạnh tay" hơn với lớp thiểu số, nhưng không giúp mô hình phân biệt hai lớp tốt hơn.

Với LogReg, Borderline-SMOTE còn *kém* SMOTE thường cả về F1 (0,457 so với 0,478) lẫn PR-AUC (0,488 so với 0,503). Mô hình tuyến tính chỉ có một đường ranh giới thẳng; khi các điểm mới dồn vào vùng chồng lấn, đường thẳng đó bị kéo lệch hẳn về phía lớp đa số.

3.6.2 Bước 11. Biểu đồ so sánh

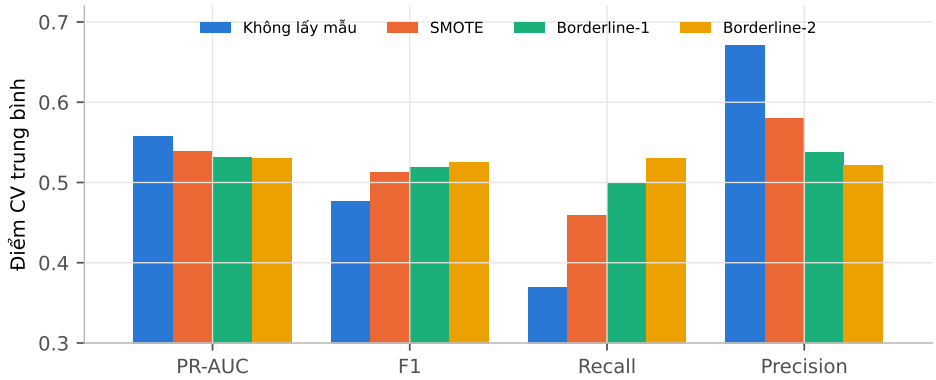
Mục đích: nhìn xu hướng của Bước 10 bằng hình.

Ô lệnh 11 · Biểu đồ cột cho XGBoost: xu hướng khi càng "tập trung vào biên"

code/ch03/s11_bieu_do.py

```
xg = cv_df[cv_df.index.str.startswith("XGBoost")]
xg.index = [i.split("| ")[1] for i in xg.index]

ax = xg.T.plot(kind="bar", figsize=(7, 2.8), width=0.8, rot=0,
               color=["#2a78d6", "#eb6834", "#1baf7a", "#eda100"])
ax.set_ylim(0.3, 0.72); ax.set_ylabel("Điểm CV trung bình")
ax.legend(ncol=4, fontsize=7, loc="upper center")
plt.savefig("figures/ch03_s11_bars.pdf", bbox_inches="tight")
```



Hình 3.5: Điểm CV của XGBoost với bốn chiến lược. Trong mỗi nhóm cột, đi từ trái sang phải là càng "tập trung vào biên": Recall tăng, Precision và PR-AUC giảm.

Đọc kết quả. Lệnh `xg.T.plot(kind="bar")` vẽ một nhóm cột cho mỗi chỉ số, mỗi cột ứng với một chiến lược. Dòng `xg.index = [i.split("| ")[1] ...]` chỉ để rút gọn tên dòng, bỏ phần "XGBoost | " cho chú thích ngắn gọn.

3.6.3 Bước 12. Kiểm tra trên tập test

Mục đích: xác nhận xu hướng trên dữ liệu mới.

Ô lệnh 12 · Kiểm tra trên tập test (XGBoost, ngưỡng 0,5) code/ch03/s12_test.py

```
from sklearn.metrics import average_precision_score, f1_score, recall_score,
↪  precision_score

test_rows, proba = {}, {}
for sname, s in strategies.items():
    pipe = make(xgb_clf(), s).fit(X_train, y_train)
    p = pipe.predict_proba(X_test)[:, 1]
    proba[sname] = p
    pred = (p ⩾ 0.5).astype(int)
    test_rows[sname] = {"PR-AUC": average_precision_score(y_test, p), "F1":
    ↪  f1_score(y_test, pred),
                        "Recall": recall_score(y_test, pred), "Precision":
                        ↪  precision_score(y_test, pred)}
print(pd.DataFrame(test_rows).T)
```

Kết quả

	PR-AUC	F1	Recall	Precision
Không lấy mẫu	0.556	0.467	0.361	0.662
SMOTE	0.547	0.507	0.453	0.576
Borderline-1	0.535	0.502	0.489	0.515
Borderline-2	0.530	0.511	0.515	0.507

Đọc kết quả. Tập test cho cùng xu hướng: Recall tăng từ 0,361 lên 0,515 (Borderline-2), Precision giảm từ 0,662 xuống 0,507, PR-AUC giảm từ 0,556 xuống 0,530. F1 của ba kỹ thuật lấy mẫu gần như ngang nhau, khoảng 0,50–0,51.

3.6.4 Bước 13. Đối chiếu với phương án dời ngưỡng

Mục đích: kiểm tra xem kết luận của Chương 2 có còn đúng không.

Ô lệnh 13 · Đối chiếu với phương án dời ngưỡng của Chương 2 (t* = 0,26)
code/ch03/s13_doi_nguong.py

```
t_star = 0.26                                    # ngưỡng tìm được bằng out-of-fold ở
↪  Chương 2
pred = (proba["Không lấy mẫu"] ⩾ t_star).astype(int)
print(f"XGBoost, t = 0,26 -> F1 = {f1_score(y_test, pred):.3f}, "
      f"Recall = {recall_score(y_test, pred):.3f}, Precision =
      ↪  {precision_score(y_test, pred):.3f}")
```

Kết quả

```
XGBoost, t = 0,26 -> F1 = 0.543, Recall = 0.584, Precision = 0.508
```

Đọc kết quả. Ngưỡng $t^{\star} = 0{,}26$ được tìm ở Chương 2 (Bước 16) bằng dự đoán out-of-fold trên tập train, nên dùng lại ở đây là hợp lệ. Kết quả:

XGBoost, tập test	F1	Recall	Precision
Borderline-2, ngưỡng 0,5	0,511	0,515	0,507
Không lấy mẫu, ngưỡng 0,26	**0,543**	**0,584**	**0,508**

Dời ngưỡng vẫn thắng: Recall và F1 đều cao hơn, trong khi Precision bằng nhau. Kết luận của Chương 2 vẫn đứng vững với Borderline-SMOTE.

Tự kiểm tra

Câu 2. Từ Bước 10 và Bước 13, một bạn kết luận: "Borderline-SMOTE là kỹ thuật kém, không nên dùng." Kết luận này có quá vội không?

3.7 Giai đoạn 6 – Khi nào Borderline-SMOTE có lợi?

3.7.1 Bước 14. Thử với nhiều mức chồng lấn

Mục đích: kiểm tra một giả thuyết thường gặp trong tài liệu: "Borderline-SMOTE tốt hơn SMOTE khi vùng biên rõ ràng và không quá lớn."

Ta tạo 4 bộ dữ liệu nhân tạo, mỗi bộ 4 000 điểm, 10 đặc trưng, tỷ lệ 9 : 1. Chỉ thay đổi tham số `class_sep`: giá trị càng lớn, hai lớp càng tách xa nhau, nên tỷ lệ danger càng nhỏ.

Ô lệnh 14 · Khi nào Borderline-SMOTE thắng SMOTE? Thử với các mức chồng lấn khác nhau

code/ch03/s14_khi_nao_co_loi.py

```
rows = []
for sep in [0.5, 1.0, 1.5, 2.0]:                # class_sep càng lớn, hai lớp càng
↪  tách biệt
    Xs, ys = make_classification(n_samples=4000, n_features=10, n_informative=5,
                                 weights=[0.9, 0.1], class_sep=sep, flip_y=0,
                                 ↪  random_state=0)
    t, _ = minority_types(StandardScaler().fit_transform(Xs), ys)
    row = {"class_sep": sep, "% DANGER": np.mean(t == "DANGER") * 100}
    for sname in ["SMOTE", "Borderline-1"]:
        r = cross_validate(make(xgb_clf(), samplers[sname]), Xs, ys, cv=skf,
        ↪  scoring=scoring)
        row[f"F1 {sname}"] = r["test_F1"].mean()
        row[f"PR-AUC {sname}"] = r["test_PR-AUC"].mean()
    rows.append(row)
print(pd.DataFrame(rows).set_index("class_sep").round(3).to_string())
```

Kết quả

class_sep	% DANGER	F1 SMOTE	PR-AUC SMOTE	F1 Borderline-1	PR-AUC Borderline-1
0.5	60.25	0.662	0.781	0.656	0.778
1.0	39.00	0.831	0.920	0.821	0.917
1.5	20.75	0.938	0.983	0.933	0.983
2.0	10.50	0.976	0.998	0.978	0.998

Đọc kết quả.

- Cột `\% DANGER` giảm từ 60% (chồng lấn mạnh, giống Credit Default) xuống 10,5% (hai lớp tách biệt rõ). Thí nghiệm đã bao phủ đúng khoảng ta muốn kiểm tra.
- *Ở cả bốn* mức, F1 và PR-AUC của SMOTE và Borderline-1 gần như bằng nhau, chênh nhau không quá 0,01. SMOTE nhỉnh hơn ở ba mức đầu; Borderline-1 chỉ nhỉnh hơn ở mức cuối

(0,978 so với 0,976), một chênh lệch rất nhỏ.

Vậy với XGBoost, **ta không quan sát thấy lợi thế rõ ràng nào của Borderline-SMOTE**, kể cả khi vùng biên nhỏ và rõ ràng. Giả thuyết ban đầu không được xác nhận trên các bộ dữ liệu này.

Vì sao kết quả khác với bài báo gốc?

Bài báo của Han et al. [9] thử nghiệm với các mô hình đơn giản hơn nhiều so với XGBoost. Một mô hình mạnh như XGBoost đã tự học được ranh giới phức tạp; việc "nhắc" nó chú ý vào vùng biên bằng mẫu giả không thêm nhiều thông tin mới. Đây là một bài học quan trọng khi đọc tài liệu: *kết quả của một kỹ thuật phụ thuộc vào dữ liệu và vào mô hình đi kèm*. Đừng tin một kỹ thuật "chắc chắn tốt hơn" chỉ vì nó mới hơn hay phức tạp hơn. Hãy tự kiểm chứng trên dữ liệu và mô hình của mình, đúng như ta vừa làm.

3.8 Tổng kết

Ưu điểm	Hạn chế
Tập trung mẫu tổng hợp vào vùng mô hình hay sai	Thêm siêu tham số m, kết quả nhạy với m
Bỏ qua điểm noise, tránh khuếch đại nhiễu	Dồn mẫu vào vùng chồng lấn, làm giảm Precision và PR-AUC
Phép phân loại safe/danger/noise là công cụ chẩn đoán dữ liệu rất hữu ích	Với mô hình mạnh như XGBoost, không thấy lợi thế rõ ràng so với SMOTE
Chỉ khác SMOTE một dòng code	Nếu có ít điểm danger, mẫu mới tập trung vào vài điểm và dễ quá khớp

Ba điều cần nhớ nhất

1. Borderline-SMOTE = SMOTE, nhưng chỉ dùng các điểm danger ($m/2 \leq m' < m$) làm hạt giống.
2. Tỷ lệ safe/danger/noise là một cách mô tả *độ khó* của dữ liệu. Credit Default có 61% danger: hai lớp chồng lấn rất mạnh.
3. Trên dữ liệu và mô hình của giáo trình, Borderline-SMOTE không thắng SMOTE, và cả hai đều không thắng phương án dời ngưỡng.

3.9 Những lỗi thường gặp

Lỗi thường gặp

1. **Không chuẩn hóa trước khi tính láng giềng.** Phép phân loại (3.1) sẽ sai hoàn toàn (xem Chương 2, Bước 9).
2. **Nhầm m với k.** m dùng để *phân loại* điểm (tìm trong cả hai lớp); k dùng để *nội suy* (tìm trong lớp thiểu số). Hai tham số độc lập với nhau.
3. **Kết luận "Borderline-SMOTE tốt hơn SMOTE" chỉ dựa trên F1 tại ngưỡng** 0,5. F1

tăng có thể đi kèm PR-AUC giảm (Bước 10).

4. **Không báo cáo tỷ lệ danger.** Thiếu thông tin này, người đọc không biết kết quả có áp dụng được cho dữ liệu của họ không.
5. **Tin vào kết quả của tài liệu mà không tự kiểm chứng** (Bước 14).

3.10 Khi viết bài báo khoa học

Gợi ý cho bài báo

- **Bảng mô tả dữ liệu mở rộng.** Ngoài kích thước và tỷ lệ lớp, hãy thêm tỷ lệ safe/danger/noise cho mỗi bộ dữ liệu. Đây là thông tin giúp người đọc hiểu *vì sao* các kỹ thuật cho kết quả khác nhau trên các bộ dữ liệu khác nhau.
- **Thí nghiệm theo mức chồng lấn** như Bước 14 là một thiết kế tốt cho bài báo so sánh: thay vì "thử trên một bộ dữ liệu", ta cho thấy kết quả *thay đổi theo đặc điểm dữ liệu*. Một kết quả "không có khác biệt" được thiết kế cẩn thận cũng là kết quả có giá trị.
- **Mẫu câu**: *"Following Han et al. [9], minority instances were categorised as safe, borderline or noisy using $m = 10$ nearest neighbours; 61.4% of defaulters were borderline, indicating substantial class overlap."*

3.11 Bài tập

Mức 1 – Làm quen.

1. Chạy lại Ô lệnh 3 với $m = 5$, $m = 20$, $m = 30$. Số điểm mỗi loại thay đổi thế nào? Câu trả lời có khớp với dự đoán của bạn ở Câu 1 không?
2. Sửa hàm `borderline_smote_tay` để chỉ dùng điểm safe làm hạt giống ("Safe-SMOTE"). Vẽ kết quả và so sánh với Borderline-1.

Mức 2 – Vận dụng.

3. Tính tỷ lệ safe/danger/noise cho Credit Default với $m = 5, 10, 20$. Kết luận "chồng lấn mạnh" có thay đổi không?
4. Loại các điểm noise khỏi phần học của mỗi fold, rồi huấn luyện XGBoost (không lấy mẫu). PR-AUC thay đổi thế nào? (Gợi ý: viết vòng lặp CV bằng tay như Bước 20, Chương 1.)
5. Kết hợp Borderline-2 với phương án dời ngưỡng: tìm ngưỡng tốt nhất bằng out-of-fold cho mô hình có Borderline-2. F1 trên tập test có vượt 0,543 không?

Mức 3 – Nâng cao.

6. Lặp lại Bước 14 với `LogisticRegression` và với `DecisionTreeClassifier(max_depth=5)` thay cho XGBoost. Lợi thế của Borderline-SMOTE có xuất hiện với các mô hình đơn giản hơn không?

7. Đọc về SVM-SMOTE (`imblearn.over_sampling.SVMSMOTE`), một biến thể dùng các vector hỗ trợ của SVM để xác định vùng biên. Đưa nó vào so sánh ở Bước 10.

3.12 Đáp án phần Tự kiểm tra

Câu 1.

Giảm. Để là noise, *tất cả m* láng giềng phải thuộc lớp đa số. Với *m* lớn, ta nhìn ra xa hơn, nên khả năng gặp ít nhất một điểm thiểu số tăng lên, và điều kiện "tất cả" khó thỏa mãn hơn. Bài tập 1 cho phép kiểm chứng.

Câu 2.

Quá vội. Thí nghiệm chỉ dùng một bộ dữ liệu thật và chủ yếu một mô hình (XGBoost). Kết luận đúng hơn là: "*Trên Credit Default, với XGBoost*, Borderline-SMOTE không tốt hơn SMOTE, và cả hai đều kém phương án dời ngưỡng." Với mô hình đơn giản hơn, hoặc dữ liệu có ít lớp thiểu số hơn, kết quả có thể khác (Bài tập 6).

Thuật ngữ của chương

Tiếng Việt	Tiếng Anh	Nghĩa ngắn gọn
Vùng biên	borderline	Vùng tiếp giáp giữa hai lớp
Điểm an toàn	safe instance	Điểm thiểu số có ít láng giềng thuộc lớp đa số
Điểm nguy hiểm	danger instance	Điểm thiểu số có ít nhất một nửa láng giềng thuộc lớp đa số
Điểm nhiễu	noise instance	Điểm thiểu số có mọi láng giềng thuộc lớp đa số
Chồng lấn lớp	class overlap	Hai lớp chiếm cùng một vùng trong không gian đặc trưng
Hạt giống	seed	Điểm gốc dùng để sinh mẫu tổng hợp

Tài liệu đọc thêm

Bài báo gốc của Han et al. [9]. Tổng quan về học từ dữ liệu mất cân bằng của He and Garcia [10], trong đó có thảo luận về vai trò của vùng chồng lấn.

CHƯƠNG 4

ADASYN – Sinh mẫu thích nghi theo độ khó

Chương này giúp bạn làm được gì?

Sau khi làm theo hết các bước trong chương, bạn sẽ:

- tính được "độ khó" r_i của mỗi điểm thiểu số và chia "ngân sách" điểm tổng hợp theo độ khó đó;
- tự viết ADASYN bằng cách sửa vòng lặp của hàm SMOTE trong Chương 2;
- so sánh ba cách phân bổ ngân sách: *đều* (SMOTE), *có hoặc không* (Borderline-SMOTE) và *tỷ lệ với độ khó* (ADASYN);
- kiểm tra bằng thí nghiệm một điểm yếu lý thuyết của ADASYN: ưu tiên cả những điểm có thể là nhiễu.

4.1 Trước khi bắt đầu

Bạn cần gì?

Chương này dùng tiếp hàm `smote_tay` (Chương 2, Ô lệnh 7) và phép phân loại điểm thiểu số theo láng giềng (Chương 3, Bước 2–3). Không cần cài thêm thư viện nào.

Ý tưởng của chương

Ở hai chương trước, ta đã gặp hai cách chia "ngân sách" điểm tổng hợp cho các điểm thiểu số:

- **SMOTE**: mọi điểm nhận phần như nhau.
- **Borderline-SMOTE**: chỉ điểm danger được nhận, các điểm khác nhận 0. Đây là quyết định "có hoặc không".

ADASYN (*ADAptive SYNthetic sampling*) của He et al. [11] đi theo con đường ở giữa: mỗi điểm nhận một phần **tỷ lệ với độ khó** của nó. Điểm càng bị lớp đa số bao vây thì càng được sinh nhiều điểm mới.

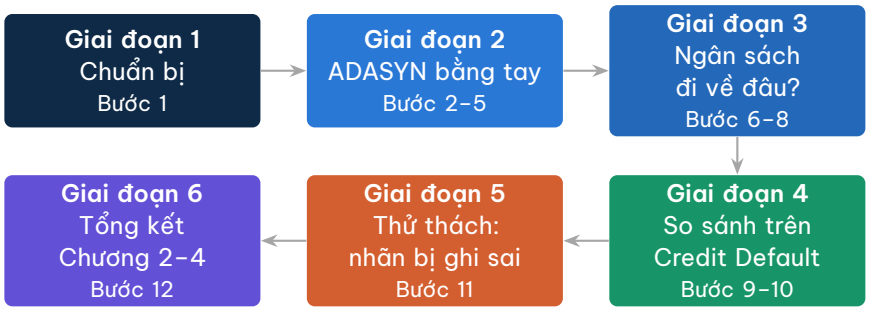
Lớp học phụ đạo

Hãy nghĩ đến một lớp học phụ đạo với tổng số giờ cố định:

- SMOTE chia đều giờ cho mọi học sinh.
- Borderline-SMOTE chỉ phụ đạo cho nhóm học sinh "trung bình".

- ADASYN chia giờ *tỷ lệ thuận với mức yếu*: em yếu nhất được nhiều giờ nhất.

Cách chia của ADASYN nghe rất hợp lý, trừ một trường hợp: nếu "em yếu nhất" thực ra là do *ghi nhầm điểm*, thì phụ đạo thêm cũng vô ích. ADASYN không phân biệt được điểm khó thật với điểm bị ghi sai nhãn.



Hình 4.1: Lộ trình của Chương 4: 6 giai đoạn, 12 bước.

4.2 Giai đoạn 1 – Chuẩn bị

4.2.1 Bước 1. Nạp dữ liệu

Mục đích: có dữ liệu thật và dữ liệu đồ chơi giống hai chương trước.

Ô lệnh 1 · Nạp thư viện, dữ liệu Credit Default và bộ dữ liệu "đồ chơi" `code/ch04/s01_chuan_bi.py`

```
import sys
sys.path.append("code")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from common import credit_split, xgb_clf

pd.set_option("display.width", 80)
pd.set_option("display.precision", 3)
SEED = 42

X_train, X_test, y_train, y_test = credit_split()
X_toy, y_toy = make_classification(n_samples=600, n_features=2, n_redundant=0,
                                   n_clusters_per_class=1, weights=[0.9, 0.1],
                                   class_sep=1.0, random_state=3)
print("Credit -- train:", X_train.shape, "| Đồ chơi:", X_toy.shape, np.bincount(y_toy))
```

Kết quả

```
Credit -- train: (24000, 23) | Đồ chơi: (600, 2) [534  66]
```

Đọc kết quả. Hai bộ dữ liệu giống hệt Chương 2 and 3. Ô lệnh nạp sẵn `StandardScaler` vì ta sẽ cần chuẩn hóa trước khi tính láng giềng trên dữ liệu thật.

4.3 Giai đoạn 2 – ADASYN bằng tay

4.3.1 Bước 2. Đo độ khó của một điểm

Mục đích: hiểu đại lượng r_i, "nguyên liệu" chính của ADASYN.

Với mỗi điểm thiểu số $\mathbf{x}_i$, tìm $K = 5$ láng giềng gần nhất trong **toàn bộ** dữ liệu, rồi đếm số láng giềng thuộc lớp đa số, gọi là Δ_i. **Độ khó** của điểm là

$$r_i = \frac{\Delta_i}{K}, \qquad 0 \le r_i \le 1. \tag{4.1}$$

Ô lệnh 2 · Đo "độ khó" r_i của một điểm thiểu số bằng 5 láng giềng code/ch04/s02_do_kho.py

```
K = 5
nn_all = NearestNeighbors(n_neighbors=K + 1).fit(X_toy)     # tìm trong TOÀN BỘ dữ liệu
min_idx = np.where(y_toy == 1)[0]

for i in min_idx[[1, 32, 21]]:                               # ba điểm đã gặp ở Chương 3
    nb = nn_all.kneighbors(X_toy[[i]], return_distance=False)[0][1:]
    delta = (y_toy[nb] == 0).sum()                           # số láng giềng thuộc lớp đa
    ↪ số
    print(f"Điểm {i:3d}: nhãn láng giềng = {y_toy[nb]} -> Delta = {delta}, r = {delta /
    ↪ K:.1f}")
```

Kết quả

```
Điểm  23: nhãn láng giềng = [1 1 1 1 1] -> Delta = 0, r = 0.0
Điểm 279: nhãn láng giềng = [1 0 0 1 1] -> Delta = 2, r = 0.4
Điểm 162: nhãn láng giềng = [0 0 0 0 0] -> Delta = 5, r = 1.0
```

Đọc kết quả. Ba điểm này chính là ba ví dụ ở Bước 2 của Chương 3, nhưng giờ chỉ xét 5 láng giềng thay vì 10:

- Điểm 23: không có láng giềng nào thuộc lớp đa số, nên $r = 0/5 = 0$. Đây là điểm *dễ*.
- Điểm 279: 2 trong 5 láng giềng thuộc lớp đa số, nên $r = 2/5 = 0{,}4$.
- Điểm 162: cả 5 láng giềng đều thuộc lớp đa số, nên $r = 5/5 = 1$. Đây là điểm *khó nhất*, hoặc là nhiễu.

Khác với Borderline-SMOTE (chỉ chia thành ba loại), r_i là một con số liên tục từ 0 đến 1.

4.3.2 Bước 3. Chia ngân sách theo độ khó

Mục đích: biến độ khó r_i thành số điểm tổng hợp g_i mà mỗi điểm sẽ sinh ra.

ADASYN chia ngân sách theo ba bước:

1. **Tổng ngân sách**: $G = n_0 - n_1$, đủ để hai lớp bằng nhau. Với bộ đồ chơi, $G = 534 - 66 = 468$.
2. **Chuẩn hóa độ khó** để tổng bằng 1:

$$\hat{r}_i = \frac{r_i}{\sum_j r_j}. \tag{4.2}$$

3. **Số điểm sinh từ mỗi điểm**:

$$g_i = \hat{r}_i \times G \quad \text{(làm tròn thành số nguyên)}. \tag{4.3}$$

Ô lệnh 3 · Chia "ngân sách" G điểm tổng hợp cho từng điểm thiểu số

code/ch04/s03_phan_bo_ngan_sach.py

```
def adasyn_weights(X, y, K=5):
    """Tính r_i (độ khó), r_hat_i (đã chuẩn hóa) và g_i (số điểm sẽ sinh từ điểm i)."""
    G = (y == 0).sum() - (y == 1).sum()                    # tổng số điểm cần sinh
    nb = NearestNeighbors(n_neighbors=K + 1).fit(X).kneighbors(
        X[y == 1], return_distance=False)[:, 1:]
    r = (y[nb] == 0).sum(axis=1) / K
    r_hat = r / r.sum()                                    # tổng các r_hat bằng 1
    g = np.rint(r_hat * G).astype(int)                     # làm tròn thành số nguyên
    return r, r_hat, g, G

r, r_hat, g, G = adasyn_weights(X_toy, y_toy)
print(f"G = {G}, tổng các r_i = {r.sum():.1f}, tổng các g_i = {g.sum()}")
print(pd.DataFrame({"r_i": r, "r_hat_i": r_hat, "g_i": g}).iloc[[1, 32, 21]])
print("Số điểm có g_i = 0:", (g == 0).sum(), "/", len(g), "| g_i lớn nhất:", g.max())
```

Kết quả

```
G = 468, tổng các r_i = 17.2, tổng các g_i = 464
    r_i  r_hat_i  g_i
1   0.0    0.000    0
32  0.4    0.023   11
21  1.0    0.058   27
Số điểm có g_i = 0: 44 / 66 | g_i lớn nhất: 27
```

Đọc kết quả. Tổng độ khó của 66 điểm là $\sum r_j = 17{,}2$. Tính lại bằng tay cho ba điểm ví dụ:

- Điểm 23 (vị trí 1): $\hat{r} = 0/17{,}2 = 0$, nên $g = 0$. Điểm này **không sinh điểm mới nào**.
- Điểm 279 (vị trí 32): $\hat{r} = 0{,}4/17{,}2 = 0{,}023$, nên $g = 0{,}023 \times 468 \approx 10{,}9$, làm tròn thành 11.
- Điểm 162 (vị trí 21): $\hat{r} = 1/17{,}2 = 0{,}058$, nên $g = 0{,}058 \times 468 \approx 27{,}2$, làm tròn thành 27.

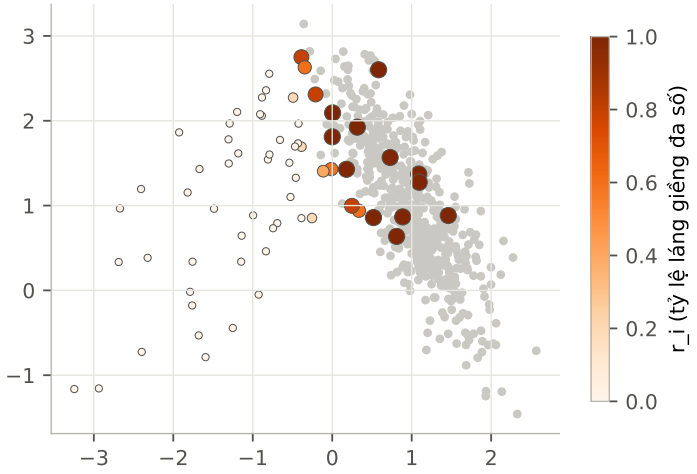
Tổng các g_i là 464 chứ không phải 468, do làm tròn. Dòng cuối cho thấy 44 trên 66 điểm có $g_i = 0$: **hai phần ba số điểm thiểu số không được dùng**. Ngân sách dồn vào khoảng 22 điểm có láng giềng thuộc lớp đa số.

4.3.3 Bước 4. Nhìn ngân sách trên mặt phẳng

Mục đích: thấy bằng mắt ADASYN "ưu tiên" những điểm nào.

Ô lệnh 4 · Vẽ: màu = độ khó r_i, kích thước = số điểm g_i được sinh code/ch04/s04_ve_trong_so.py

```
X_min = X_toy[y_toy == 1]
fig, ax = plt.subplots(figsize=(5, 3.4))
ax.scatter(*X_toy[y_toy == 0].T, s=8, color="#c9c8c2")
sc = ax.scatter(*X_min.T, c=r, cmap="Oranges", vmin=0, vmax=1, s=10 + g * 1.5,
                edgecolor="#52514e", linewidth=0.4)
fig.colorbar(sc, ax=ax, label="r_i (tỷ lệ láng giềng đa số)", shrink=0.85)
plt.savefig("figures/ch04_s04_weights.pdf", bbox_inches="tight")
```



Hình 4.2: Màu thể hiện độ khó r_i (càng đậm càng khó), kích thước thể hiện số điểm g_i được sinh từ mỗi điểm thiểu số. Các điểm nhạt và nhỏ ở bên trái là các điểm dễ, có $g_i = 0$.

Đọc kết quả. Các điểm to và đậm nằm ở vùng giữa, sát lớp đa số. Một số trong đó nằm lọt hẳn giữa đám điểm xám; theo Chương 3, đó là những điểm noise. Đây chính là chỗ ADASYN khác Borderline-SMOTE: Borderline-SMOTE *loại bỏ* điểm noise, còn ADASYN cho chúng *nhiều điểm nhất*.

4.3.4 Bước 5. Tự viết ADASYN

Mục đích: thấy rằng ADASYN chỉ khác SMOTE ở cách tổ chức vòng lặp.

Ô lệnh 5 · Tự viết ADASYN: mỗi điểm sinh đúng g_i điểm mới code/ch04/s05_tu_viet.py

```
def adasyn_tay(X_min, g, k=5, seed=SEED):
    rng = np.random.default_rng(seed)
    _, idx = NearestNeighbors(n_neighbors=k + 1).fit(X_min).kneighbors(X_min)
    new = []
    for i in range(len(X_min)):                  # KHÁC SMOTE: duyệt từng điểm,
        for _ in range(g[i]):                    # mỗi điểm sinh đúng g_i lần
            j = rng.choice(idx[i][1:])
            lam = rng.uniform(0, 1)
            new.append(X_min[i] + lam * (X_min[j] - X_min[i]))
```

```
    return np.array(new)

X_syn = adasyn_tay(X_min, g)
print("Số điểm sinh ra:", len(X_syn))
top = np.argsort(-g)[:3]
print("Ba điểm sinh nhiều nhất:", {int(i): int(g[i]) for i in top},
      "-> chiếm", f"{g[top].sum() / g.sum():.0%}", "tổng số điểm mới")
```

Kết quả

```
Số điểm sinh ra: 464
Ba điểm sinh nhiều nhất: {13: 27, 11: 27, 21: 27} -> chiếm 17% tổng số điểm mới
```

Đọc kết quả. So sánh với `smote_tay` của Chương 2:

- SMOTE: một vòng lặp chạy G lần, mỗi lần chọn *ngẫu nhiên* một điểm thiểu số.
- ADASYN: hai vòng lặp lồng nhau. Vòng ngoài duyệt *từng* điểm thiểu số i; vòng trong sinh đúng g_i điểm mới từ điểm đó.

Phần nội suy ((2.1)) giữ nguyên. Ba điểm khó nhất, mỗi điểm sinh 27 điểm mới, đã chiếm 17% toàn bộ điểm tổng hợp.

Thuật toán 3: ADASYN [11]

Đầu vào: Dữ liệu đã chuẩn hóa; số láng giềng K
Đầu ra: Tập điểm tổng hợp S_{syn}

$G \leftarrow n_0 - n_1$;
for *mỗi điểm thiểu số* $\mathbf{x}_i$ **do**
  $r_i \leftarrow$ (số láng giềng thuộc lớp đa số trong K láng giềng) $/\, K$; // Bước 2
$g_i \leftarrow \text{round}\big(r_i / \sum_j r_j \times G\big)$ với mọi i ; // Bước 3
for *mỗi điểm thiểu số* $\mathbf{x}_i$ **do**
  for $t = 1$ **to** g_i **do**
    Chọn ngẫu nhiên láng giềng thiểu số $\mathbf{x}_{\text{lg}}$ của $\mathbf{x}_i$; lấy λ trong $(0, 1)$;
    Thêm $\mathbf{x}_i + \lambda(\mathbf{x}_{\text{lg}} - \mathbf{x}_i)$ vào S_{syn};
return S_{syn}

Tự kiểm tra

Câu 1. Nếu *mọi* điểm thiểu số đều có $r_i = 0$ (không điểm nào có láng giềng thuộc lớp đa số), công thức ở Bước 3 sẽ gặp vấn đề gì? Tình huống này nói gì về dữ liệu?

4.4 Giai đoạn 3 – Ngân sách đi về đâu?

4.4.1 Bước 6. So sánh ba thuật toán bằng hình

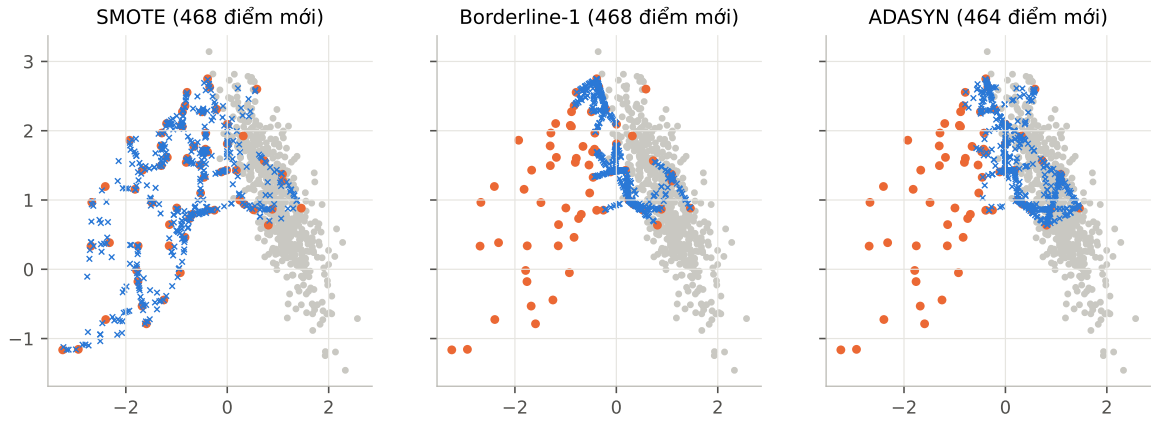
Mục đích: đặt SMOTE, Borderline-SMOTE và ADASYN cạnh nhau.

Ô lệnh 6 · So sánh SMOTE, Borderline-1 và ADASYN (thư viện imblearn)

code/ch04/s06_so_sanh_hinh.py

```
from imblearn.over_sampling import SMOTE, BorderlineSMOTE, ADASYN

samplers = {"SMOTE": SMOTE(random_state=SEED),
            "Borderline-1": BorderlineSMOTE(kind="borderline-1", random_state=SEED),
            "ADASYN": ADASYN(n_neighbors=5, random_state=SEED)}
fig, axes = plt.subplots(1, 3, figsize=(10, 3.2), sharex=True, sharey=True)
for ax, (name, s) in zip(axes, samplers.items()):
    Xr, yr = s.fit_resample(X_toy, y_toy)
    ax.scatter(*X_toy[y_toy == 0].T, s=6, color="#c9c8c2")
    ax.scatter(*X_min.T, s=12, color="#eb6834")
    ax.scatter(*Xr[len(X_toy):].T, s=8, marker="x", color="#2a78d6", linewidth=0.7)
    ax.set_title(f"{name} ({len(Xr) - len(X_toy)} điểm mới)")
plt.savefig("figures/ch04_s06_compare.pdf", bbox_inches="tight")
```



Hình 4.3: Điểm tổng hợp (dấu × xanh) của ba thuật toán trên cùng dữ liệu đồ chơi.

Đọc kết quả. ADASYN cho hình ảnh "trung gian": giống Borderline-1, nó gần như bỏ trống vùng an toàn bên trái; nhưng khác Borderline-1, nó sinh rất nhiều điểm ở vùng sâu bên trong lớp đa số (bên phải), quanh các điểm có $r_i = 1$.

4.4.2 Bước 7. Đếm ngân sách theo nhóm độ khó

Mục đích: biến quan sát ở Bước 6 thành con số.

Để so sánh công bằng, ta chia 66 điểm thiểu số thành ba nhóm theo r_i (cùng dùng 5 láng giềng), rồi tính tỷ lệ điểm tổng hợp xuất phát từ mỗi nhóm theo ba cách phân bổ.

Ô lệnh 7 · Ngân sách đi về đâu? So sánh ba cách phân bổ (cùng dùng 5 láng giềng)

code/ch04/s07_ngan_sach_ve_dau.py

```
groups = pd.cut(r, bins=[-0.01, 0.49, 0.99, 1.0], labels=["r < 0,5", "0,5 ⩽ r < 1", "r
↪  = 1"])
share = pd.DataFrame({
    "Số điểm": pd.Series(groups).value_counts().sort_index(),
    "SMOTE": pd.Series(np.ones(len(r)), index=groups).groupby(level=0,
    ↪  observed=False).sum() / len(r),
```

```
    "Kiểu Borderline": pd.Series(((r ⩾ 0.5) & (r < 1)).astype(float), index=groups)
                    .groupby(level=0, observed=False).sum() / ((r ⩾ 0.5) & (r <
                    ↪ 1)).sum(),
    "ADASYN": pd.Series(g, index=groups).groupby(level=0, observed=False).sum() /
    ↪ g.sum(),
})
print(share.round(2))
```

Kết quả

	Số điểm	SMOTE	Kiểu Borderline	ADASYN
r < 0,5	48	0.73	0.0	0.06
0,5 ⩽ r < 1	6	0.09	1.0	0.25
r = 1	12	0.18	0.0	0.70

Đọc kết quả. Mỗi cột cộng lại bằng 1 (tức 100% ngân sách):

- **SMOTE**: chia theo số điểm. Nhóm dễ ($r < 0{,}5$) có 48/66 điểm nên nhận 73% ngân sách.
- **Kiểu Borderline**: dồn 100% cho nhóm giữa, bỏ qua cả nhóm dễ lẫn nhóm $r = 1$.
- **ADASYN**: dồn 70% ngân sách cho 12 điểm có $r = 1$, tức những điểm mà cả 5 láng giềng đều thuộc lớp đa số.

Với 5 láng giềng, có 12 điểm có $r = 1$. Con số này nhiều hơn 9 điểm noise ở Chương 3, vì ở đó ta dùng 10 láng giềng: càng nhìn ít láng giềng, càng dễ "toàn là lớp đa số".

Vậy ADASYN và Borderline-SMOTE có quan điểm **trái ngược nhau** về những điểm có $r = 1$: Borderline-SMOTE coi chúng là nhiễu và bỏ qua; ADASYN coi chúng là điểm khó nhất và ưu tiên nhất.

4.4.3 Bước 8. Trên dữ liệu thật

Mục đích: xem ADASYN sẽ dồn ngân sách thế nào trên Credit Default.

Ô lệnh 8 · Trên Credit Default: phân bố r_i và ngân sách ADASYN đi về đâu

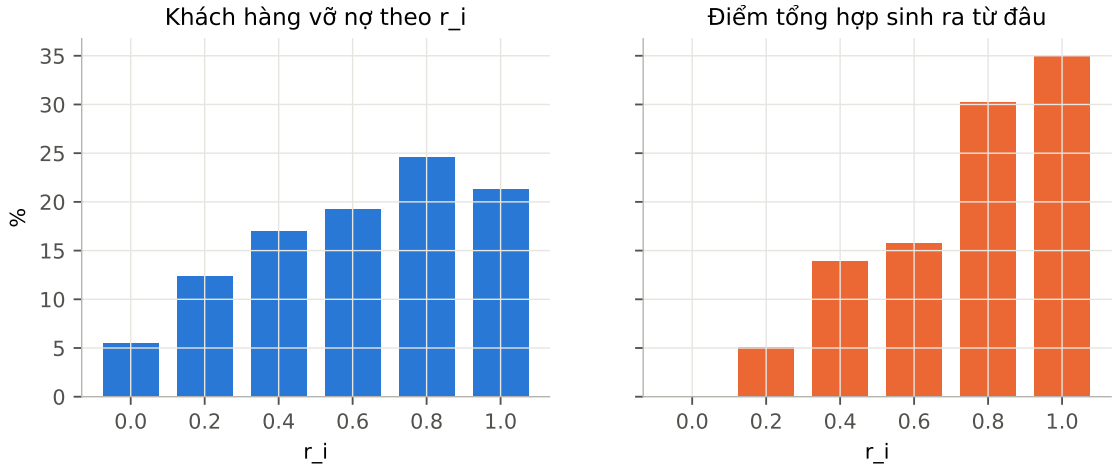
code/ch04/s08_credit_r.py

```
Z_train = StandardScaler().fit_transform(X_train)
rc, _, gc, Gc = adasyn_weights(Z_train, y_train.values)
tab = pd.DataFrame({"% khách hàng vỡ nợ":
↪ pd.Series(rc).value_counts(normalize=True).sort_index() * 100,
                    "% điểm tổng hợp": pd.Series(gc).groupby(rc).sum() / gc.sum() * 100})
tab.index.name = "r_i"
print(tab.round(1))

fig, ax = plt.subplots(1, 2, figsize=(8, 2.8), sharey=True)
ax[0].bar(tab.index, tab.iloc[:, 0], width=0.15, color="#2a78d6")
ax[0].set_title("Khách hàng vỡ nợ theo r_i"); ax[0].set_ylabel("%")
ax[1].bar(tab.index, tab.iloc[:, 1], width=0.15, color="#eb6834")
ax[1].set_title("Điểm tổng hợp sinh ra từ đâu")
for a in ax:
    a.set_xlabel("r_i")
plt.savefig("figures/ch04_s08_credit_r.pdf", bbox_inches="tight")
```

Kết quả

```
     % khách hàng vỡ nợ  % điểm tổng hợp
r_i
0.0                 5.5              0.0
0.2                12.4              5.1
0.4                17.0             13.9
0.6                19.3             15.8
0.8                24.6             30.2
1.0                21.3             35.0
```



Hình 4.4: Trái: tỷ lệ khách hàng vỡ nợ theo r_i ($K = 5$). Phải: tỷ lệ điểm tổng hợp được sinh từ mỗi nhóm r_i.

Đọc kết quả.

- $21{,}3\%$ khách hàng vỡ nợ có $r_i = 1$: cả 5 láng giềng gần nhất đều là người không vỡ nợ. Nhóm này nhận 35% **ngân sách**, nhiều hơn mọi nhóm khác.
- $5{,}5\%$ khách hàng có $r_i = 0$, tức các "ca dễ", không nhận điểm nào.
- Nhóm $r_i = 0{,}8$ và $r_i = 1$ cộng lại nhận $30{,}2\% + 35{,}0\% = 65{,}2\%$ ngân sách.

Hai cột phải "nghiêng" hẳn về phía r lớn so với hai cột trái: đó là đặc điểm thiết kế của ADASYN.

4.5 Giai đoạn 4 – So sánh trên Credit Default

4.5.1 Bước 9. Đánh giá chéo

Mục đích: so sánh ADASYN với SMOTE và với không lấy mẫu theo đúng quy trình.

Ô lệnh 9 · Đánh giá chéo: không lấy mẫu, SMOTE, ADASYN (2 mô hình) `code/ch04/s09_cv.py`

```python
from imblearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.linear_model import LogisticRegression

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

```
scoring = {"PR-AUC": "average_precision", "F1": "f1", "Recall": "recall", "Precision":
↪  "precision"}

def make(model, sampler):
    steps = [("scale", StandardScaler())]
    if sampler is not None:
        steps.append(("sampler", sampler))
    return Pipeline(steps + [("model", model)])

strategies = {"Không lấy mẫu": None, "SMOTE": samplers["SMOTE"], "ADASYN":
↪  samplers["ADASYN"]}
cv_rows = {}
for mname, mk in {"LogReg": lambda: LogisticRegression(max_iter=2000), "XGBoost":
↪  xgb_clf}.items():
    for sname, s in strategies.items():
        r_ = cross_validate(make(mk(), s), X_train, y_train, cv=skf, scoring=scoring)
        cv_rows[f"{mname} | {sname}"] = {k: r_["test_" + k].mean() for k in scoring}
print(pd.DataFrame(cv_rows).T)
```

Kết quả

	PR-AUC	F1	Recall	Precision
LogReg | Không lấy mẫu	0.506	0.366	0.246	0.714
LogReg | SMOTE	0.503	0.478	0.652	0.377
LogReg | ADASYN	0.499	0.462	0.688	0.348
XGBoost | Không lấy mẫu	0.558	0.476	0.369	0.671
XGBoost | SMOTE	0.539	0.512	0.459	0.580
XGBoost | ADASYN	0.534	0.508	0.457	0.571

Đọc kết quả. Hàm `make` giống hệt hai chương trước, chỉ thay sampler.

- Với XGBoost, ADASYN và SMOTE **gần như ngang nhau**: F1 0,508 so với 0,512, PR-AUC 0,534 so với 0,539. Các chênh lệch nhỏ hơn độ lệch chuẩn giữa các fold (khoảng 0,01).
- Với LogReg, ADASYN tăng Recall mạnh hơn SMOTE (0,688 so với 0,652) nhưng Precision và F1 thấp hơn.
- Mọi kỹ thuật lấy mẫu đều làm *giảm* PR-AUC so với không lấy mẫu (0,558 với XGBoost).

4.5.2 Bước 10. Ảnh hưởng của K

Mục đích: biết tham số chính của ADASYN thay đổi kết quả ra sao.

Ô lệnh 10 · Ảnh hưởng của số láng giềng K (XGBoost + ADASYN) `code/ch04/s10_tham_so_K.py`

```
rows = []
for K_ in [3, 5, 10, 20]:
    r_ = cross_validate(make(xgb_clf(), ADASYN(n_neighbors=K_, random_state=SEED)),
                        X_train, y_train, cv=skf, scoring=scoring)
    rows.append({"K": K_, **{k: r_["test_" + k].mean() for k in scoring}})
print(pd.DataFrame(rows).set_index("K"))
```

Kết quả

```
    PR-AUC     F1  Recall  Precision
K
3    0.535  0.512   0.474      0.556
5    0.534  0.508   0.457      0.571
10   0.532  0.504   0.447      0.576
20   0.534  0.499   0.430      0.595
```

Đọc kết quả. Xu hướng giống SMOTE ở Chương 2 (Bước 15): khi K tăng từ 3 lên 20, Recall giảm từ 0,474 xuống 0,430, Precision tăng từ 0,556 lên 0,595, còn PR-AUC gần như đứng yên quanh 0,534. Một lần nữa, tham số chỉ thay đổi *sự đánh đổi*, không thay đổi khả năng xếp hạng.

4.6 Giai đoạn 5 – Thử thách: nhãn bị ghi sai

4.6.1 Bước 11. Khi một phần dữ liệu train bị ghi nhãn sai

Mục đích: kiểm tra bằng thí nghiệm điểm yếu lý thuyết của ADASYN: nếu có nhiều nhãn sai, ADASYN có bị ảnh hưởng nặng hơn các kỹ thuật khác không?

Dữ liệu thực tế hiếm khi hoàn hảo: nhân viên nhập sai, hệ thống ghi nhầm. Ta mô phỏng tình huống này bằng cách *cố ý* lật nhãn của 5% khách hàng không vỡ nợ trong tập train thành "vỡ nợ". Những khách hàng bị lật nhãn này nằm giữa đám người không vỡ nợ, nên có r_i cao. Theo lý thuyết, ADASYN sẽ dồn nhiều ngân sách cho họ. Tập test giữ nguyên nhãn đúng.

Ô lệnh 11 · Thử thách: dữ liệu train có NHÃN SAI. Kỹ thuật nào chịu đựng tốt nhất?

code/ch04/s11_nhieu_nhan.py

```
from sklearn.metrics import average_precision_score, f1_score

rng = np.random.default_rng(SEED)
y_noisy = y_train.copy()
flip = rng.choice(np.where(y_train == 0)[0], size=int(0.05 * (y_train == 0).sum()),
↪  replace=False)
y_noisy.iloc[flip] = 1                    # 5% người KHÔNG vỡ nợ bị ghi nhầm thành vỡ nợ
print(f"Đã lật nhãn {len(flip)} khách hàng (chỉ trong tập train; tập test giữ nguyên)")

rows = {}
for sname, s in {"Không lấy mẫu": None, **samplers}.items():
    res = {}
    for tag, yy in [("sạch", y_train), ("nhiễu", y_noisy)]:
        p = make(xgb_clf(), s).fit(X_train, yy).predict_proba(X_test)[:, 1]
        res[f"PR-AUC {tag}"] = average_precision_score(y_test, p)
        res[f"F1 {tag}"] = f1_score(y_test, (p ⩾ 0.5).astype(int))
    res["Giảm F1"] = res["F1 sạch"] - res["F1 nhiễu"]
    rows[sname] = res
print(pd.DataFrame(rows).T.round(3).to_string())
```

Kết quả

```
Đã lật nhãn 934 khách hàng (chỉ trong tập train; tập test giữ nguyên)
               PR-AUC sạch  F1 sạch  PR-AUC nhiễu  F1 nhiễu  Giảm F1
```

```
Không lấy mẫu        0.556    0.467        0.557     0.477   -0.010
SMOTE                0.547    0.507        0.543     0.512   -0.005
Borderline-1         0.535    0.502        0.535     0.509   -0.007
ADASYN               0.538    0.505        0.539     0.508   -0.003
```

Đọc kết quả. Với mỗi kỹ thuật, ta huấn luyện hai lần (nhãn sạch và nhãn nhiễu) rồi chấm điểm trên cùng tập test sạch. Cột cuối là mức giảm F1; số âm nghĩa là F1 *tăng* khi có nhiễu.

- **PR-AUC gần như không đổi** với mọi kỹ thuật (chênh không quá 0,004). Khả năng xếp hạng của XGBoost khá vững trước 5% nhãn sai.
- **F1 tăng nhẹ** với mọi kỹ thuật. Lý do: 934 nhãn bị lật thành "vỡ nợ" làm tăng tỷ lệ lớp dương trong tập train, nên mô hình dự đoán "vỡ nợ" nhiều hơn, giống như hạ ngưỡng nhẹ.
- **ADASYN không bị ảnh hưởng nặng hơn** các kỹ thuật khác.

Vậy trong thí nghiệm này, điểm yếu lý thuyết của ADASYN *không* biểu hiện thành thiệt hại đo được. Một cách giải thích: Credit Default vốn đã có 21% khách hàng vỡ nợ với $r_i = 1$ (Bước 8). Thêm vài trăm điểm "nhiễu nhân tạo" không thay đổi đáng kể bức tranh vốn đã rất chồng lấn.

Bài học về phương pháp

Một điểm yếu *có thể xảy ra về lý thuyết* chưa chắc đã xảy ra trên dữ liệu cụ thể của bạn. Ngược lại, một ưu điểm lý thuyết cũng chưa chắc biểu hiện (Chương 3, Bước 14). Cách duy nhất để biết là **thiết kế thí nghiệm kiểm chứng**, như ta vừa làm.

4.7 Giai đoạn 6 – Tổng kết ba chương về lấy mẫu

4.7.1 Bước 12. Bảng tổng kết trên tập test

Mục đích: đặt tất cả các kỹ thuật của Chương 2 and 3 và chương này cạnh nhau.

Ô lệnh 12 · Bảng tổng kết Chương 2-4 trên tập test (XGBoost) code/ch04/s12_tong_ket.py

```
from sklearn.metrics import recall_score, precision_score

final = {}
for sname, s in {"Không lấy mẫu": None, **samplers,
                 "Borderline-2": BorderlineSMOTE(kind="borderline-2",
                  ↪  random_state=SEED)}.items():
    p = make(xgb_clf(), s).fit(X_train, y_train).predict_proba(X_test)[:, 1]
    final[sname] = (p, 0.5)
    if s is None:
        final["Không lấy mẫu, t = 0,26"] = (p, 0.26)
summary = {}
for name, (p, t) in final.items():
    pred = (p ⩾ t).astype(int)
    summary[name] = {"F1": f1_score(y_test, pred), "Recall": recall_score(y_test,
    ↪  pred),
                     "Precision": precision_score(y_test, pred),
                     "PR-AUC": average_precision_score(y_test, p)}
```

```
print(pd.DataFrame(summary).T.round(3))
```

Kết quả

	F1	Recall	Precision	PR-AUC
Không lấy mẫu	0.467	0.361	0.662	0.556
Không lấy mẫu, t = 0,26	0.543	0.584	0.508	0.556
SMOTE	0.507	0.453	0.576	0.547
Borderline-1	0.502	0.489	0.515	0.535
ADASYN	0.505	0.462	0.558	0.538
Borderline-2	0.511	0.515	0.507	0.530

Đọc kết quả. Bảng này tóm tắt ba chương:

- Bốn kỹ thuật lấy mẫu (SMOTE, Borderline-1, Borderline-2, ADASYN) cho F1 rất gần nhau, từ 0,502 đến 0,511. Chúng chỉ khác nhau ở *cách* đánh đổi Recall với Precision.
- Cả bốn đều *giảm* PR-AUC so với mô hình gốc (0,556).
- **Dời ngưỡng** của mô hình gốc về $t = 0{,}26$ cho F1 cao nhất (0,543) và Recall cao nhất (0,584), mà vẫn giữ nguyên PR-AUC.

Thông điệp chung của Chương 2–4

Trên dữ liệu này, **không kỹ thuật lấy mẫu nào vượt được phương án dời ngưỡng**. Chương 5 sẽ cho thấy lý do sâu xa: lấy mẫu, gán trọng số và dời ngưỡng đều là những cách khác nhau để thay đổi *cùng một thứ*, đó là "giá" tương đối giữa hai loại lỗi.

4.8 Tổng kết

Ưu điểm	Hạn chế
Phân bổ ngân sách liên tục theo độ khó, mềm dẻo hơn quy tắc "có hoặc không"	Ưu tiên cả điểm có $r_i = 1$, có thể là nhiễu
Có cơ sở lý thuyết rõ: dịch ranh giới về phía điểm khó	Ngân sách dồn vào ít điểm, dễ quá khớp khi lớp thiểu số rất nhỏ
Chỉ có một tham số chính (K)	Không xác định được khi mọi $r_i = 0$
	Trên Credit Default, không tốt hơn SMOTE

4.9 Những lỗi thường gặp

Lỗi thường gặp

1. **Nghĩ rằng "ADASYN mới hơn nên chắc chắn tốt hơn SMOTE".** Trên Credit Default, hai kỹ thuật tương đương (Bước 9).
2. **Dùng ADASYN trên dữ liệu có nhiều nhãn sai mà không kiểm tra.** Ở đây không thấy thiệt hại (Bước 11), nhưng với dữ liệu khác thì có thể có.
3. **Quên rằng số điểm sinh ra không đúng bằng** G do làm tròn (Bước 3). Tỷ lệ lớp sau ADASYN chỉ *xấp xỉ* $1:1$.

4. Các lỗi của Chương 2 vẫn áp dụng: chạy ADASYN ngoài pipeline, không chuẩn hóa, chỉ báo cáo F1 tại ngưỡng 0,5.

4.10 Khi viết bài báo khoa học

Gợi ý cho bài báo

- **Phân tích "ngân sách đi về đâu"** như Hình 4.4 giúp giải thích *cơ chế*, không chỉ báo cáo con số cuối cùng. Phản biện thường đánh giá cao những phân tích như vậy.
- **Thí nghiệm nhiễu nhãn có kiểm soát** như Bước 11 là một cách đơn giản để đánh giá độ bền của kỹ thuật. Có thể mở rộng với nhiều mức nhiễu (1%, 5%, 10%, 20%).
- **Mẫu câu**: *"ADASYN [11] allocates synthetic samples in proportion to local difficulty; on our data, 35% of synthetic samples originated from defaulters whose five nearest neighbours were all non-defaulters."*

4.11 Bài tập

Mức 1 – Làm quen.

1. Tính tay r_i, $\hat{r}_i$ và g_i cho một điểm thiểu số khác của bộ đồ chơi (ví dụ vị trí 10), rồi kiểm tra bằng hàm `adasyn_weights`.
2. Chạy lại Ô lệnh 3 với $K = 3$ và $K = 10$. Số điểm có $g_i = 0$ thay đổi thế nào?

Mức 2 – Vận dụng.

3. Viết phiên bản "ADASYN bỏ nhiễu": đặt $r_i = 0$ cho các điểm có $r_i = 1$, rồi chuẩn hóa lại. Vẽ kết quả trên bộ đồ chơi và so sánh với ADASYN gốc.
4. Dùng `FunctionSampler` của `imblearn` để đưa phiên bản ở Bài 3 vào pipeline, rồi so sánh với ADASYN gốc trên Credit Default.

Mức 3 – Nâng cao.

5. Lặp lại Bước 11 với mức nhiễu 1%, 10% và 20%. Vẽ PR-AUC theo mức nhiễu cho bốn kỹ thuật. Có kỹ thuật nào bền hơn hẳn không?
6. Dùng `SMOTEENN` và `SMOTETomek` của `imblearn` (lấy mẫu rồi làm sạch vùng chồng lấn). Chúng có cải thiện PR-AUC so với SMOTE không?

4.12 Đáp án phần Tự kiểm tra

Câu 1.

Khi mọi $r_i = 0$, mẫu số $\sum_j r_j = 0$, nên phép chia ở Bước 3 không xác định (chia cho 0). Thư viện `imblearn` sẽ báo lỗi. Tình huống này có nghĩa là không điểm thiểu số nào có láng giềng

thuộc lớp đa số: hai lớp *tách biệt hoàn toàn*. Khi đó bài toán vốn đã dễ, và cũng không cần cân bằng dữ liệu.

Thuật ngữ của chương

Tiếng Việt	Tiếng Anh	Nghĩa ngắn gọn
Độ khó cục bộ	local difficulty	Tỷ lệ láng giềng thuộc lớp đa số (r_i)
Ngân sách lấy mẫu	sampling budget	Tổng số điểm tổng hợp cần sinh (G)
Phân bổ thích nghi	adaptive allocation	Chia ngân sách tỷ lệ với độ khó
Nhiễu nhãn	label noise	Nhãn bị ghi sai trong dữ liệu

Tài liệu đọc thêm

Bài báo gốc của He et al. [11]. So sánh có hệ thống các kỹ thuật lấy mẫu trong tổng quan của He and Garcia [10].

CHƯƠNG 5

Class Weighting – Học có trọng số theo lớp

Chương này giúp bạn làm được gì?

Sau khi làm theo hết các bước trong chương, bạn sẽ:

- tự tính hàm mất mát log loss có trọng số cho từng khách hàng;
- kiểm chứng bằng code rằng *gán trọng số 3* cho một mẫu cũng chính là *nhân bản* mẫu đó thành 3 bản;
- dùng `class_weight` (scikit-learn) và `scale_pos_weight` (XGBoost);
- hiểu và kiểm chứng định lý quan trọng nhất của Phần II: **gán trọng số w tương đương với dời ngưỡng về $1/(1+w)$**;
- chọn ngưỡng quyết định từ *chi phí thực tế* của hai loại lỗi.

5.1 Trước khi bắt đầu

Ý tưởng của chương

Ba chương trước tác động vào *dữ liệu*: thêm mẫu cho lớp thiểu số. Chương này tác động vào *cách chấm điểm* khi huấn luyện: giữ nguyên dữ liệu, nhưng nói với mô hình rằng "mỗi lần dự đoán sai một khách hàng vỡ nợ, phạt nặng gấp w lần".

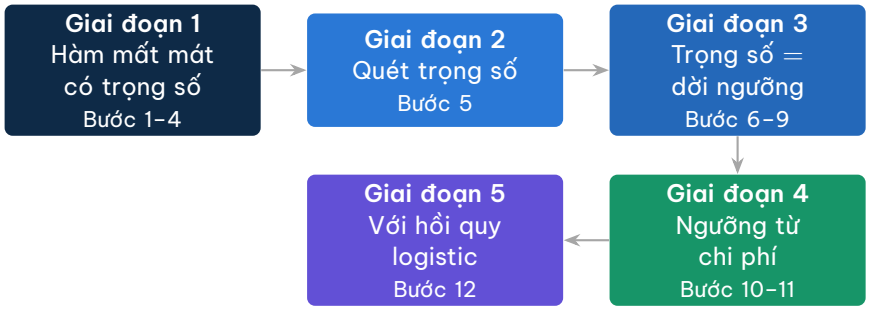
Nặng hơn thay vì nhiều hơn

Thay vì nhân bản mỗi khách hàng vỡ nợ thành w bản sao như Chương 2, ta chỉ giữ một bản, nhưng cho nó "nặng" gấp w lần. Kết quả về mặt toán học là như nhau (Bước 3 sẽ kiểm chứng), nhưng gán trọng số không tạo điểm giả, không tốn bộ nhớ, và không có yếu tố ngẫu nhiên.

Đây là trường hợp đơn giản nhất của **học nhạy chi phí** (*cost-sensitive learning*) [7]. Ý tưởng cốt lõi là các loại lỗi có hậu quả khác nhau:

- Cho một khách hàng sắp vỡ nợ vay tiếp (lỗi FN) có thể làm ngân hàng mất cả khoản vay.
- Từ chối nhầm một khách hàng tốt (lỗi FP) chỉ làm mất một phần lợi nhuận.

5.2 Giai đoạn 1 – Hàm mất mát có trọng số



Hình 5.1: Lộ trình của Chương 5: 5 giai đoạn, 12 bước.

5.2.1 Bước 1. Nạp dữ liệu

Mục đích: có tập train và test như các chương trước, và biết tỷ số giữa hai lớp.

Ô lệnh 1 · Nạp thư viện và dữ liệu code/ch05/s01_chuan_bi.py

```
import sys
sys.path.append("code")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from common import credit_split, xgb_clf

pd.set_option("display.width", 80)
pd.set_option("display.precision", 3)
SEED = 42

X_train, X_test, y_train, y_test = credit_split()
n0, n1 = (y_train == 0).sum(), (y_train == 1).sum()
print(f"Tập train: {n0} không vỡ nợ, {n1} vỡ nợ, tỷ số n0/n1 = {n0 / n1:.2f}")
```

Kết quả

```
Tập train: 18691 không vỡ nợ, 5309 vỡ nợ, tỷ số n0/n1 = 3.52
```

Đọc kết quả. Tỷ số $n_0/n_1 = 18\,691/5\,309 = 3{,}52$: cứ 1 khách hàng vỡ nợ thì có 3,52 khách hàng không vỡ nợ. Con số 3,52 sẽ xuất hiện rất nhiều lần trong chương này.

5.2.2 Bước 2. Tính tay hàm mất mát

Mục đích: hiểu "phạt nặng hơn" nghĩa là gì bằng những con số cụ thể.

Khi huấn luyện, mô hình cố gắng làm cho **hàm mất mát** (*loss*) nhỏ nhất. Với bài toán phân loại nhị phân, hàm mất mát phổ biến nhất là **log loss**. Với một khách hàng có nhãn y và xác suất dự đoán $\hat{p}$:

$$\text{mất mát} = -\big[\,y \log \hat{p} + (1-y)\log(1-\hat{p})\,\big]. \tag{5.1}$$

Nghĩa là: nếu $y = 1$, mất mát là $-\log \hat{p}$; nếu $y = 0$, mất mát là $-\log(1-\hat{p})$. Dự đoán càng sai, mất mát càng lớn.

Với trọng số, mất mát của mỗi khách hàng được nhân thêm w_1 (nếu $y = 1$) hoặc w_0 (nếu $y = 0$):

$$\mathcal{L}_w = -\sum_i \Big[w_1 \, y_i \log \hat{p}_i + w_0 \, (1 - y_i) \log(1 - \hat{p}_i) \Big]. \tag{5.2}$$

Ô lệnh 2 · Hàm mất mát log loss, có và không có trọng số, tính tay cho 4 khách hàng

code/ch05/s02_ham_mat_mat.py

```
y_true = np.array([1, 1, 0, 0])
p_hat = np.array([0.30, 0.80, 0.20, 0.60])      # xác suất vỡ nợ mà mô hình dự đoán
w = 3.5                                          # trọng số cho lớp vỡ nợ (lớp 1)

loss = -(y_true * np.log(p_hat) + (1 - y_true) * np.log(1 - p_hat))   # mất mát từng
↪  người
weight = np.where(y_true == 1, w, 1.0)
print(pd.DataFrame({"y": y_true, "p_hat": p_hat, "mất mát": loss.round(3),
                    "trọng số": weight, "mất mát x trọng số": (loss * weight).round(3)}))
print(f"\nTổng mất mát -- không trọng số: {loss.sum():.3f} | có trọng số: {(loss *
↪  weight).sum():.3f}")
```

Kết quả

```
   y  p_hat  mất mát  trọng số  mất mát x trọng số
0  1    0.3    1.204       3.5               4.214
1  1    0.8    0.223       3.5               0.781
2  0    0.2    0.223       1.0               0.223
3  0    0.6    0.916       1.0               0.916

Tổng mất mát -- không trọng số: 2.567 | có trọng số: 6.134
```

Đọc kết quả. Kiểm tra lại từng dòng với logarit tự nhiên (ln):

- Khách hàng 0: vỡ nợ ($y = 1$), mô hình chỉ cho $\hat{p} = 0{,}3$. Mất mát $= -\ln 0{,}3 = 1{,}204$. Nhân trọng số 3,5 được 4,214.
- Khách hàng 1: vỡ nợ, $\hat{p} = 0{,}8$ (đoán tốt). Mất mát $= -\ln 0{,}8 = 0{,}223$.
- Khách hàng 2: không vỡ nợ ($y = 0$), $\hat{p} = 0{,}2$. Mất mát $= -\ln(1 - 0{,}2) = 0{,}223$.
- Khách hàng 3: không vỡ nợ, $\hat{p} = 0{,}6$ (đoán sai). Mất mát $= -\ln 0{,}4 = 0{,}916$.

Không có trọng số, khách hàng 0 và khách hàng 3 (hai người bị đoán sai) đóng góp gần như ngang nhau (1,204 và 0,916). Có trọng số, khách hàng 0 đóng góp 4,214, gấp hơn 4 lần khách hàng 3. Để giảm tổng mất mát, mô hình sẽ "ưu tiên" sửa lỗi bỏ sót người vỡ nợ.

5.2.3 Bước 3. Trọng số bằng nhân bản

Mục đích: kiểm chứng bằng code rằng gán trọng số 3 và chép 3 bản cho cùng một mô hình.

Ô lệnh 3 · Kiểm chứng: trọng số 3 cho một mẫu = nhân bản mẫu đó thành 3 bản

code/ch05/s03_trong_so_bang_nhan_ban.py

```
Xs = StandardScaler().fit_transform(X_train)[:2000]      # lấy 2 000 dòng cho nhanh
ys = y_train.values[:2000]

# Cách A: trọng số 3 cho mọi khách hàng vỡ nợ
lr_a = LogisticRegression(max_iter=5000).fit(Xs, ys, sample_weight=np.where(ys == 1,
↪  3.0, 1.0))
# Cách B: chép mỗi khách hàng vỡ nợ thành 3 bản, không dùng trọng số
X_dup = np.vstack([Xs, Xs[ys == 1], Xs[ys == 1]])
y_dup = np.concatenate([ys, ys[ys == 1], ys[ys == 1]])
lr_b = LogisticRegression(max_iter=5000).fit(X_dup, y_dup)

print("5 hệ số đầu -- cách A:", lr_a.coef_[0][:5].round(4))
print("5 hệ số đầu -- cách B:", lr_b.coef_[0][:5].round(4))
print("Chênh lệch lớn nhất giữa hai cách:", f"{np.abs(lr_a.coef_ -
↪  lr_b.coef_).max():.1e}")
```

Kết quả

```
5 hệ số đầu -- cách A: [-0.1844 -0.0701 -0.1569 -0.0407  0.0926]
5 hệ số đầu -- cách B: [-0.1844 -0.0701 -0.1569 -0.0407  0.0926]
Chênh lệch lớn nhất giữa hai cách: 9.5e-14
```

Đọc kết quả.

- **Cách A** dùng tham số `sample_weight` của hàm `fit`: mỗi khách hàng vỡ nợ có trọng số 3, khách hàng khác có trọng số 1. Lệnh `np.where(ys == 1, 3.0, 1.0)` tạo đúng mảng trọng số đó.
- **Cách B** không dùng trọng số, mà dùng `np.vstack` để chép thêm 2 bản của mỗi khách hàng vỡ nợ (tổng cộng 3 bản).
- Hệ số học được của hai cách **trùng nhau** đến chữ số thứ 13 sau dấu phẩy. Chênh lệch $9{,}5 \times 10^{-14}$ chỉ là sai số làm tròn của máy tính.

Lý do: trong (5.2), "mất mát nhân 3" cũng chính là "cộng mất mát 3 lần". Đây là lý do gán trọng số còn được gọi là *nhân bản "ảo"*.

5.2.4 Bước 4. Trọng số "balanced"

Mục đích: biết cách scikit-learn tự tính trọng số để cân bằng hai lớp.

Ô lệnh 4 · Trọng số "balanced" của scikit-learn code/ch05/s04_balanced.py

```
from sklearn.utils.class_weight import compute_class_weight

w_bal = compute_class_weight("balanced", classes=np.array([0, 1]), y=y_train)
ratio = w_bal[1] / w_bal[0]
print(f"w0 = {w_bal[0]:.3f}, w1 = {w_bal[1]:.3f}, tỷ số w1/w0 = {ratio:.3f}")
print(f"Tổng trọng số lớp 0: {w_bal[0] * n0:.0f} | lớp 1: {w_bal[1] * n1:.0f}")
```

Kết quả

```
w0 = 0.642, w1 = 2.260, tỷ số w1/w0 = 3.521
Tổng trọng số lớp 0: 12000 | lớp 1: 12000
```

Đọc kết quả. Với tùy chọn `class_weight="balanced"`, scikit-learn tính trọng số của mỗi lớp c là

$$w_c = \frac{n}{2 \cdot n_c},$$

trong đó n là tổng số mẫu. Ví dụ: $w_1 = 24\,000/(2 \times 5\,309) = 2{,}260$ và $w_0 = 24\,000/(2 \times 18\,691) = 0{,}642$. Khi đó **tổng trọng số của hai lớp bằng nhau**: $2{,}260 \times 5\,309 = 0{,}642 \times 18\,691 = 12\,000$. Mô hình "nhìn thấy" hai lớp quan trọng ngang nhau.

Điều quan trọng là *tỷ số* $w_1/w_0 = 3{,}52$, đúng bằng n_0/n_1. Trong XGBoost, ta đặt trực tiếp tỷ số này qua tham số `scale_pos_weight`.

5.3 Giai đoạn 2 – Quét trọng số

5.3.1 Bước 5. Thử nhiều giá trị w

Mục đích: xem trọng số thay đổi kết quả của XGBoost như thế nào.

Ô lệnh 5 · Quét trọng số lớp dương w cho XGBoost (đánh giá chéo, 40 giây)

code/ch05/s05_quet_trong_so.py

```
from sklearn.model_selection import StratifiedKFold, cross_validate

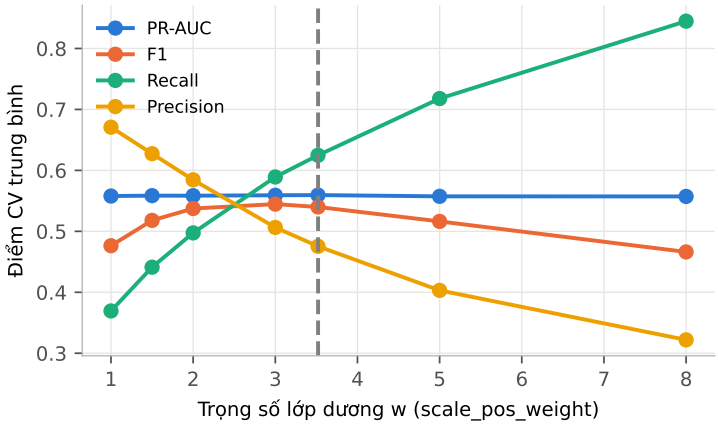
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
scoring = {"ROC-AUC": "roc_auc", "PR-AUC": "average_precision", "F1": "f1",
           "Recall": "recall", "Precision": "precision"}
rows = []
for w_ in [1, 1.5, 2, 3, round(ratio, 2), 5, 8]:
    r = cross_validate(xgb_clf(scale_pos_weight=w_), X_train, y_train, cv=skf,
    ↪  scoring=scoring)
    rows.append({"w": w_, **{k: r["test_" + k].mean() for k in scoring}})
sweep = pd.DataFrame(rows).set_index("w")
print(sweep)

ax = sweep[["PR-AUC", "F1", "Recall", "Precision"]].plot(marker="o", figsize=(5.2,
↪  2.9))
ax.axvline(ratio, ls="--", color="gray")
ax.set_xlabel("Trọng số lớp dương w (scale_pos_weight)"); ax.set_ylabel("Điểm CV trung
↪  bình")
plt.savefig("figures/ch05_s05_sweep.pdf", bbox_inches="tight")
```

Kết quả

w	ROC-AUC	PR-AUC	F1	Recall	Precision
1.00	0.783	0.558	0.476	0.369	0.671
1.50	0.784	0.559	0.518	0.441	0.627
2.00	0.783	0.558	0.537	0.497	0.584
3.00	0.783	0.559	0.545	0.589	0.506

```
3.52    0.783   0.559  0.540   0.625      0.475
5.00    0.783   0.557  0.516   0.718      0.403
8.00    0.782   0.557  0.466   0.845      0.322
```



Hình 5.2: Điểm CV của XGBoost theo trọng số lớp dương w. Đường nét đứt: $w = 3{,}52$ (“balanced”).

Đọc kết quả. Đây là kết quả quan trọng nhất của chương:

- **ROC-AUC và PR-AUC gần như không đổi**: ROC-AUC luôn trong khoảng 0,782–0,784, PR-AUC trong khoảng 0,557–0,559, với mọi w từ 1 đến 8. Trọng số không làm mô hình phân biệt hai lớp tốt hơn.
- **Recall tăng đều** theo w (từ 0,369 lên 0,845), còn **Precision giảm đều** (từ 0,671 xuống 0,322).
- **F1 cao nhất ở** $w = 3$ (0,545), rất gần giá trị “balanced” 3,52.

So với Chương 2 to 4: mọi kỹ thuật lấy mẫu đều làm PR-AUC *giảm* (từ 0,558 xuống 0,53–0,54). **Trọng số thì không.** Lý do: trọng số không tạo điểm giả trong vùng chồng lấn; nó chỉ đổi “giá” của các lỗi trên dữ liệu thật.

5.4 Giai đoạn 3 – Trọng số tương đương dời ngưỡng

5.4.1 Bước 6. Trọng số “uốn” thang xác suất

Mục đích: hiểu công thức cho biết trọng số w làm thay đổi xác suất dự đoán thế nào.

Gọi p là xác suất vỡ nợ mà mô hình *không trọng số* dự đoán cho một khách hàng. Có thể chứng minh rằng mô hình được huấn luyện với trọng số w (khi đủ dữ liệu và đủ linh hoạt) sẽ dự đoán:

$$p_w = \frac{w\,p}{w\,p + 1 - p}. \tag{5.3}$$

Phần chứng minh ở cuối bước này. Trước hết, hãy xem công thức này làm gì:

Ô lệnh 6 · Trọng số w "uốn" thang xác suất theo công thức p_w = w p / (w p + 1 - p)

code/ch05/s06_cong_thuc.py

```
p = np.array([0.05, 0.10, 0.221, 0.30, 0.50, 0.80])
w_ = ratio
p_w = w_ * p / (w_ * p + 1 - p)
print(pd.DataFrame({"p (không trọng số)": p, f"p_w (w = {w_:.2f})":
↪  p_w.round(3)}).to_string(index=False))
print(f"\nNgưỡng tương đương: p_w ⩾ 0,5  ⟺  p ⩾ 1/(1+w) = {1 / (1 + w_):.3f}")
```

Kết quả

```
 p (không trọng số)  p_w (w = 3.52)
              0.050           0.156
              0.100           0.281
              0.221           0.500
              0.300           0.601
              0.500           0.779
              0.800           0.934

Ngưỡng tương đương: p_w ⩾ 0,5  ⩤>  p ⩾ 1/(1+w) = 0.221
```

Đọc kết quả. Tính tay dòng thứ ba với $p = 0{,}221$ và $w = 3{,}52$:

$$p_w = \frac{3{,}52 \times 0{,}221}{3{,}52 \times 0{,}221 + 1 - 0{,}221} = \frac{0{,}778}{0{,}778 + 0{,}779} \approx 0{,}500.$$

Một khách hàng có xác suất thật 0,221 sẽ được mô hình có trọng số gán xác suất 0,5, tức đúng ngưỡng mặc định. Tổng quát:

$$p_w \geq 0{,}5 \iff p \geq \frac{1}{1+w}. \tag{5.4}$$

Định lý quan trọng nhất của Phần II

Huấn luyện với trọng số w rồi dùng ngưỡng 0,5 **cho cùng quyết định** với việc huấn luyện *không* trọng số rồi dùng ngưỡng $1/(1+w)$. Với $w = 3{,}52$, ngưỡng tương đương là 0,221, đúng bằng tỷ lệ vỡ nợ. Điều này không phải trùng hợp: trọng số "balanced" luôn đặt ngưỡng đúng bằng tỷ lệ lớp dương.
Hơn nữa, (5.3) là hàm *tăng*: p lớn hơn thì p_w cũng lớn hơn. Vì vậy trọng số không thay đổi *thứ tự* xếp hạng khách hàng, và đó là lý do ROC-AUC, PR-AUC không đổi ở Bước 5.

Chứng minh (dành cho bạn đọc muốn hiểu sâu). Xét một khách hàng cố định với xác suất vỡ nợ thật p. Nếu mô hình dự đoán q, mất mát kỳ vọng có trọng số là

$$\ell(q) = -\,w\,p \ln q - (1-p)\ln(1-q).$$

Mô hình tốt nhất chọn q làm $\ell(q)$ nhỏ nhất. Đạo hàm theo q và cho bằng 0:

$$-\frac{wp}{q} + \frac{1-p}{1-q} = 0 \implies wp(1-q) = (1-p)q \implies q = \frac{wp}{wp + 1 - p}. \qquad \square$$

5.4.2 Bước 7. Kiểm chứng trên dữ liệu thật

Mục đích: xem định lý có đúng với XGBoost trên Credit Default không.

Ô lệnh 7 · Kiểm chứng trên dữ liệu thật: trọng số w và ngưỡng 1/(1+w) cho cùng quyết định?

code/ch05/s07_kiem_chung.py

```
from sklearn.metrics import f1_score, recall_score, precision_score,
↪  average_precision_score
from scipy.stats import spearmanr

p1 = xgb_clf().fit(X_train, y_train).predict_proba(X_test)[:, 1]
↪  # w = 1
pw = xgb_clf(scale_pos_weight=ratio).fit(X_train, y_train).predict_proba(X_test)[:, 1]
↪  # w = 3,52
t_eq = 1 / (1 + ratio)

def report(p, t):
    pred = (p ⩾ t).astype(int)
    return {"F1": f1_score(y_test, pred), "Recall": recall_score(y_test, pred),
            "Precision": precision_score(y_test, pred), "PR-AUC":
             ↪  average_precision_score(y_test, p)}

print(pd.DataFrame({"w = 1, t = 0,5": report(p1, 0.5),
                    f"w = {ratio:.2f}, t = 0,5": report(pw, 0.5),
                    f"w = 1, t = {t_eq:.3f}": report(p1, t_eq)}).T.round(3))
print(f"\nTương quan hạng Spearman giữa hai bộ xác suất: {spearmanr(p1,
↪  pw).correlation:.4f}")
```

Kết quả

```
                     F1  Recall  Precision  PR-AUC
w = 1, t = 0,5    0.467   0.361      0.662   0.556
w = 3.52, t = 0,5 0.534   0.618      0.470   0.556
w = 1, t = 0.221  0.535   0.632      0.464   0.556

Tương quan hạng Spearman giữa hai bộ xác suất: 0.9863
```

Đọc kết quả. So sánh hai dòng cuối:

	F1	Recall	Precision
Trọng số $w = 3{,}52$, ngưỡng 0,5	0,534	0,618	0,470
Không trọng số, ngưỡng 0,221	0,535	0,632	0,464

Hai cách cho kết quả gần như trùng nhau. PR-AUC của cả ba dòng đều là 0,556. Tương quan hạng Spearman giữa hai bộ xác suất là 0,986 (1 là trùng thứ tự hoàn toàn). Hai mô hình không giống hệt nhau vì XGBoost dùng trọng số cả khi chọn cách chia cây, và có yếu tố ngẫu nhiên khi huấn luyện; nhưng về thực chất, đây là **cùng một mô hình với hai thang xác suất khác nhau**.

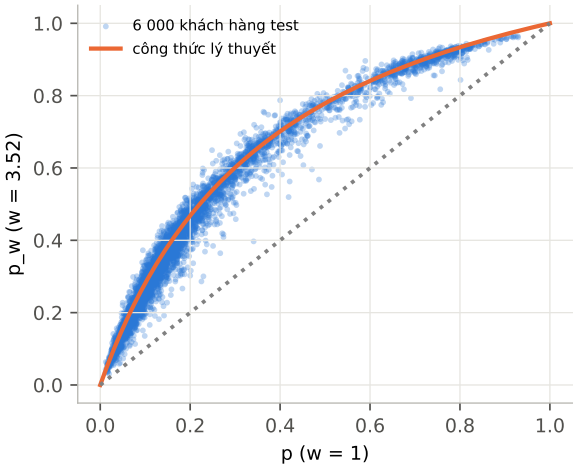
5.4.3 Bước 8. Nhìn định lý bằng hình

Mục đích: so sánh trực tiếp xác suất thật của hai mô hình với đường cong lý thuyết.

Ô lệnh 8 · Vẽ xác suất của mô hình có trọng số theo mô hình không trọng số

code/ch05/s08_ve_cong_thuc.py

```
grid = np.linspace(0, 1, 200)
fig, ax = plt.subplots(figsize=(4.2, 3.4))
ax.scatter(p1, pw, s=3, alpha=0.3, color="#2a78d6", label="6 000 khách hàng test")
ax.plot(grid, ratio * grid / (ratio * grid + 1 - grid), color="#eb6834", lw=2,
↪  label="công thức lý thuyết")
ax.plot([0, 1], [0, 1], ls=":", color="gray")
ax.set_xlabel("p (w = 1)"); ax.set_ylabel(f"p_w (w = {ratio:.2f})");
↪  ax.legend(fontsize=7)
plt.savefig("figures/ch05_s08_pw.pdf", bbox_inches="tight")
```



Hình 5.3: Mỗi chấm xanh là một khách hàng test: trục hoành là xác suất của mô hình $w = 1$, trục tung là xác suất của mô hình $w = 3{,}52$. Đường cam: (5.3). Đường chấm: $p_w = p$.

Đọc kết quả. Đám chấm xanh bám rất sát đường cam lý thuyết. Mọi chấm đều nằm *phía trên* đường chấm chéo: mô hình có trọng số luôn cho xác suất cao hơn. Mức đẩy lên lớn nhất ở vùng giữa (p khoảng 0,2–0,4), và nhỏ dần khi p gần 0 hoặc gần 1.

5.4.4 Bước 9. Cái giá phải trả: xác suất bị méo

Mục đích: thấy mặt trái của trọng số, và cách khắc phục.

Ô lệnh 9 · Cái giá phải trả: xác suất bị méo code/ch05/s09_xac_suat_bi_meo.py

```
print(f"Tỷ lệ vỡ nợ thật trên tập test : {y_test.mean():.3f}")
print(f"Xác suất TB -- w = 1           : {p1.mean():.3f}")
print(f"Xác suất TB -- w = {ratio:.2f}        : {pw.mean():.3f}")

# Khử trọng số: đảo ngược công thức ở Ô lệnh 6
p_back = pw / (pw + ratio * (1 - pw))
print(f"Xác suất TB sau khi khử trọng số: {p_back.mean():.3f}")
```

Kết quả

```
Tỷ lệ vỡ nợ thật trên tập test : 0.221
Xác suất TB -- w = 1           : 0.222
Xác suất TB -- w = 3.52        : 0.416
Xác suất TB sau khi khử trọng số: 0.213
```

Đọc kết quả.

- Mô hình $w = 1$ cho xác suất trung bình 0,222, khớp với tỷ lệ vỡ nợ thật 0,221. Ta nói mô hình được **hiệu chuẩn** tốt.
- Mô hình $w = 3{,}52$ cho xác suất trung bình 0,416, gần **gấp đôi** thực tế.

Nếu ngân hàng dùng xác suất này để ước tính tổn thất, họ sẽ dự phòng gần gấp đôi mức cần thiết.

Có thể **khử trọng số** bằng cách đảo ngược (5.3):

$$p = \frac{p_w}{p_w + w\,(1 - p_w)}.$$

Sau khi khử, xác suất trung bình là 0,213, gần lại với 0,221. Chương 25 sẽ trình bày các phương pháp hiệu chuẩn tổng quát hơn.

5.5 Giai đoạn 4 – Chọn ngưỡng từ chi phí

5.5.1 Bước 10. Ngưỡng tối ưu khi biết chi phí

Mục đích: chọn ngưỡng theo hậu quả thực tế của từng loại lỗi, thay vì dò thử cho F1 cao nhất.

Giả sử bỏ sót một khách hàng vỡ nợ tốn C_{FN} đồng, và báo động nhầm một khách hàng tốt tốn C_{FP} đồng. Với một khách hàng có xác suất vỡ nợ p:

- Nếu dự đoán "vỡ nợ": chỉ sai khi họ thực ra không vỡ nợ, nên chi phí kỳ vọng là $(1-p)\,C_{FP}$.
- Nếu dự đoán "không vỡ nợ": chỉ sai khi họ thực ra vỡ nợ, nên chi phí kỳ vọng là $p\,C_{FN}$.

Nên dự đoán "vỡ nợ" khi cách đó rẻ hơn:

$$p\,C_{FN} \;\geq\; (1-p)\,C_{FP} \quad\Longleftrightarrow\quad p \;\geq\; t^{\star} = \frac{1}{1 + C_{FN}/C_{FP}}. \tag{5.5}$$

So sánh với Bước 6: **trọng số w chính là tỷ lệ chi phí C_{FN}/C_{FP}**. Đây là kết quả kinh điển của Elkan [7].

Ô lệnh 10 · Chọn ngưỡng từ chi phí: t* = 1 / (1 + C_FN / C_FP) code/ch05/s10_chi_phi.py

```python
def total_cost(p, t, c_fn, c_fp=1):
    pred = (p >= t).astype(int)
    fn = int(((pred == 0) & (y_test == 1)).sum())    # bỏ sót người vỡ nợ
    fp = int(((pred == 1) & (y_test == 0)).sum())    # báo động nhầm
    return fn, fp, fn * c_fn + fp * c_fp

rows = []
for c in [1, 3, 5, 10]:
```

```
    t_star = 1 / (1 + c)
    fn, fp, cost = total_cost(p1, t_star, c)
    rows.append({"C_FN/C_FP": c, "t*": round(t_star, 3), "FN": fn, "FP": fp,
                 "Chi phí (t*)": cost, "Chi phí (t = 0,5)": total_cost(p1, 0.5, c)[2]})
print(pd.DataFrame(rows).to_string(index=False))
```

Kết quả

```
 C_FN/C_FP    t*  FN   FP  Chi phí (t*)  Chi phí (t = 0,5)
         1 0.500 848  245          1093               1093
         3 0.250 530  791          2381               2789
         5 0.167 357 1582          3367               4485
        10 0.091  98 3263          4243               8725
```

Đọc kết quả. Đơn vị chi phí: $C_{FP} = 1$. Đọc dòng $C_{FN}/C_{FP} = 5$ (bỏ sót một người vỡ nợ đắt gấp 5 lần báo động nhầm):

- Ngưỡng tối ưu theo (5.5): $t^\star = 1/(1+5) = 0{,}167$.
- Tại $t^\star$: bỏ sót 357 người, báo nhầm 1 582 người. Tổng chi phí $= 357 \times 5 + 1\,582 \times 1 = 1\,785 + 1\,582 = 3\,367$.
- Nếu giữ ngưỡng 0,5: tổng chi phí là 4 485, cao hơn 33%.

Với tỷ lệ chi phí 10, dùng đúng ngưỡng giúp **giảm hơn một nửa** tổng chi phí (4 243 so với 8 725). Mô hình không thay đổi gì; chỉ có quy tắc ra quyết định thay đổi.

5.5.2 Bước 11. Đường chi phí

Mục đích: nhìn tổng chi phí thay đổi theo ngưỡng, và kiểm tra ngưỡng lý thuyết có tốt thật không.

Ô lệnh 11 · Vẽ tổng chi phí theo ngưỡng (C_FN/C_FP = 5) code/ch05/s11_duong_chi_phi.py

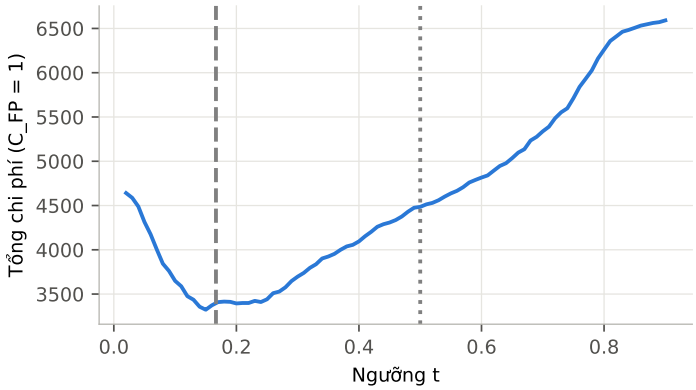
```
ts = np.linspace(0.02, 0.9, 89)
costs = [total_cost(p1, t, 5)[2] for t in ts]
print(f"Ngưỡng có chi phí thấp nhất trên tập test: {ts[np.argmin(costs)]:.2f}"
      f" (lý thuyết: {1 / 6:.3f})")

fig, ax = plt.subplots(figsize=(5, 2.7))
ax.plot(ts, costs)
ax.axvline(1 / 6, ls="--", color="gray"); ax.axvline(0.5, ls=":", color="gray")
ax.set_xlabel("Ngưỡng t"); ax.set_ylabel("Tổng chi phí (C_FP = 1)")
plt.savefig("figures/ch05_s11_cost.pdf", bbox_inches="tight")
```

Kết quả

```
Ngưỡng có chi phí thấp nhất trên tập test: 0.15 (lý thuyết: 0.167)
```

Đọc kết quả. Đường chi phí có dạng "thung lũng". Đáy thung lũng trên tập test ở $t = 0{,}15$, rất gần ngưỡng lý thuyết 0,167. Ngưỡng lý thuyết được tính *trước*, không cần nhìn tập test, mà vẫn rơi đúng vùng đáy. Đây là sức mạnh của việc lập luận từ chi phí: nếu mô hình cho xác suất đúng (hiệu chuẩn tốt), ta không cần dò thử.



Hình 5.4: Tổng chi phí trên tập test theo ngưỡng t, với $C_{FN}/C_{FP} = 5$. Đường nét đứt: ngưỡng lý thuyết $t^\star = 1/6$. Đường chấm: ngưỡng mặc định 0,5.

Gợi ý thực tế

Tách rời mô hình xác suất khỏi quy tắc ra quyết định. Mô hình nên cho xác suất đúng. Quy tắc ra quyết định (ngưỡng) nên do chuyên gia nghiệp vụ đặt ra dựa trên chi phí. Khi chi phí thay đổi (ví dụ lãi suất tăng), chỉ cần đổi ngưỡng, không cần huấn luyện lại mô hình.

5.6 Giai đoạn 5 – Với hồi quy logistic

5.6.1 Bước 12. Trọng số với một mô hình khác

Mục đích: kiểm tra kết luận có đúng với mô hình tuyến tính không.

Ô lệnh 12 · Với hồi quy logistic: class_weight cũng chỉ dịch chuyển sự đánh đổi

code/ch05/s12_logreg.py

```
rows = []
for w_ in [1, 2, 3, round(ratio, 2), 5]:
    lr = make_pipeline(StandardScaler(),
                       LogisticRegression(max_iter=2000, class_weight={0: 1, 1: w_}))
    r = cross_validate(lr, X_train, y_train, cv=skf, scoring=scoring)
    rows.append({"w": w_, **{k: r["test_" + k].mean() for k in scoring}})
print(pd.DataFrame(rows).set_index("w"))
```

Kết quả

w	ROC-AUC	PR-AUC	F1	Recall	Precision
1.00	0.726	0.506	0.366	0.246	0.714
2.00	0.726	0.505	0.502	0.430	0.603
3.00	0.726	0.504	0.502	0.554	0.459
3.52	0.726	0.504	0.481	0.645	0.383
5.00	0.726	0.503	0.418	0.778	0.286

Đọc kết quả. Với hồi quy logistic, ta dùng `class_weight={0: 1, 1: w}`. Kết quả giống hệt XGBoost về bản chất:

- ROC-AUC giữ đúng 0,726; PR-AUC gần như không đổi (0,503–0,506).
- Recall tăng từ 0,246 lên 0,778, Precision giảm từ 0,714 xuống 0,286.
- F1 cao nhất ở $w = 2$ và $w = 3$ (0,502).

So với SMOTE ở Chương 2: LogReg + SMOTE cho F1 0,478 và PR-AUC 0,503, đều không hơn trọng số $w = 2$ hay 3.

5.7 Tổng kết

Ưu điểm	Hạn chế
Không tạo dữ liệu giả, không có yếu tố ngẫu nhiên, không tốn bộ nhớ	Làm méo xác suất dự đoán; cần khử trọng số hoặc hiệu chuẩn lại
Liên hệ trực tiếp với chi phí nghiệp vụ	Một số mô hình (như kNN) không hỗ trợ trọng số
Không làm giảm PR-AUC như các kỹ thuật lấy mẫu	Về lý thuyết tương đương dời ngưỡng, nên hiếm khi tốt hơn dời ngưỡng
Có sẵn trong hầu hết thư viện	

Ba điều cần nhớ nhất

1. Trọng số w cho một lớp **tương đương nhân bản** các mẫu của lớp đó w lần (Bước 3).
2. Trọng số w **tương đương dời ngưỡng** về $1/(1+w)$, nên không thay đổi ROC-AUC, PR-AUC (Bước 5–8).
3. Ngưỡng tốt nhất được suy ra từ **chi phí**: $t^\star = 1/(1 + C_{FN}/C_{FP})$ (Bước 10).

5.8 Những lỗi thường gặp

Lỗi thường gặp

1. **Dùng xác suất của mô hình có trọng số như xác suất thật**, ví dụ để tính dự phòng rủi ro hoặc báo cho khách hàng (Bước 9).
2. **Vừa gán trọng số, vừa SMOTE, vừa dời ngưỡng.** Ba hiệu ứng cộng dồn, và mô hình nghiêng quá mức về lớp thiểu số.
3. **Chọn w bằng cách dò F1 trên tập test.** Hãy dò bằng CV, hoặc tốt hơn là suy ra từ chi phí.
4. **Nhầm tên tham số**: scikit-learn dùng `class_weight`; XGBoost dùng `scale_pos_weight`; LightGBM dùng cả `class_weight` và `scale_pos_weight`. `GradientBoostingClassifier` của scikit-learn không có `class_weight`, phải truyền `sample_weight` vào `fit`.

5.9 Khi viết bài báo khoa học

Gợi ý cho bài báo

- **Đóng khung bài toán bằng chi phí.** Thay vì "chúng tôi dùng class weighting để xử lý mất cân bằng", hãy viết: *"With a cost ratio of* $C_{FN}/C_{FP} = 5$ *estimated from [nguồn], the Bayes-optimal threshold is* $t^\star = 1/6$ *[7]."* Cách viết này chặt chẽ hơn nhiều.
- **Đánh giá bằng chi phí kỳ vọng** bên cạnh F1. Có thể vẽ đường "chi phí theo tỷ lệ C_{FN}/C_{FP}" cho nhiều phương pháp: đây là phân tích độ nhạy được phản biện đánh giá cao.
- **Báo cáo độ hiệu chuẩn** (xác suất trung bình, Brier score) khi dùng trọng số.
- Một hướng nghiên cứu nhỏ nhưng sạch sẽ: so sánh *có kiểm soát* bốn chiến lược (lấy mẫu, trọng số, dời ngưỡng, kết hợp) trên nhiều bộ dữ liệu có mức mất cân bằng khác nhau.

5.10 Bài tập

Mức 1 – Làm quen.

1. Tính tay p_w theo (5.3) cho $p = 0{,}4$ với $w = 2$ và $w = 5$.
2. Lặp lại Bước 3 với trọng số 2 (chép 2 bản). Hệ số có còn trùng nhau không?
3. Với $C_{FN}/C_{FP} = 4$, ngưỡng tối ưu là bao nhiêu? Tính tổng chi phí trên tập test tại ngưỡng đó.

Mức 2 – Vận dụng.

4. Tìm w cho F1 cao nhất bằng CV (thử w từ 2 đến 4, bước 0,25). So sánh F1 trên tập test với phương án dời ngưỡng $t = 0{,}26$ của Chương 2.
5. Vẽ lại hình ở Bước 8 cho $w = 8$. Đám chấm còn bám sát đường lý thuyết không?

Mức 3 – Nâng cao.

6. Giả sử chi phí bỏ sót của mỗi khách hàng bằng chính dư nợ `BILL_AMT1` của họ, còn chi phí báo nhầm là hằng số 5 000. Chi phí khi đó *khác nhau cho từng người*. Hãy đặt ngưỡng riêng cho từng khách hàng theo (5.5) và tính tổng chi phí.
7. Đọc King and Zeng [14] về hiệu chỉnh hệ số chặn của hồi quy logistic cho dữ liệu sự kiện hiếm, và liên hệ với (2.2) của Chương 2.

Thuật ngữ của chương

Tiếng Việt	Tiếng Anh	Nghĩa ngắn gọn
Hàm mất mát	loss function	Đại lượng mô hình cố làm nhỏ nhất khi học
Trọng số lớp	class weight	Hệ số nhân mất mát của từng lớp
Trọng số mẫu	sample weight	Hệ số nhân mất mát của từng mẫu
Học nhạy chi phí	cost-sensitive learning	Học có tính đến chi phí khác nhau của các loại lỗi
Ma trận chi phí	cost matrix	Bảng chi phí của TN, FP, FN, TP
Khử trọng số	reweighting correction	Đưa xác suất về thang không trọng số

Tài liệu đọc thêm

Elkan [7] là nền tảng lý thuyết của học nhạy chi phí. King and Zeng [14] bàn về hồi quy logistic cho sự kiện hiếm. van den Goorbergh et al. [21] phân tích tác hại của các kỹ thuật cân bằng lên độ hiệu chuẩn.

CHƯƠNG 6

Focal Loss – Tập trung vào mẫu khó

Chương này giúp bạn làm được gì?

Sau khi làm theo hết các bước trong chương, bạn sẽ:

- tự tính focal loss cho từng mẫu và hiểu vai trò của hai tham số γ và α;
- viết hàm focal loss trong PyTorch và kiểm tra nó bằng một trường hợp đặc biệt đã biết đáp án;
- xây dựng và huấn luyện một mạng nơ-ron nhiều tầng (MLP) đơn giản cho dữ liệu bảng;
- phân biệt hai cơ chế: *giảm trọng số mẫu dễ* (γ) và *trọng số lớp* (α);
- giải thích vì sao focal loss *không* phải "thuốc chữa" mất cân bằng cho dữ liệu bảng.

6.1 Trước khi bắt đầu

Bạn cần gì?

Focal loss là một *hàm mất mát*, nên nó cần một mô hình được huấn luyện bằng đạo hàm (gradient), như mạng nơ-ron. Chương này dùng thư viện **PyTorch**:

Cửa sổ lệnh (đã kích hoạt môi trường ảo)

```
pip install torch
```

Bạn không cần biết trước về mạng nơ-ron; Bước 5 sẽ giải thích những gì cần thiết. Bạn cũng nên nhớ hàm mất mát log loss ở Chương 5 (Bước 2).

Nguồn gốc của focal loss

Focal loss được Lin et al. [16] đề xuất cho bài toán *phát hiện vật thể* trong ảnh. Một bộ phát hiện xét khoảng 100 000 ô ứng viên trên mỗi bức ảnh, nhưng chỉ vài ô chứa vật thể. Gần như tất cả các ô còn lại là nền trời, mặt đường, bức tường: những mẫu âm *cực kỳ dễ*. Mỗi mẫu dễ chỉ đóng góp một lượng mất mát rất nhỏ, nhưng vì có quá nhiều mẫu dễ, tổng của chúng lấn át tín hiệu từ vài mẫu khó.

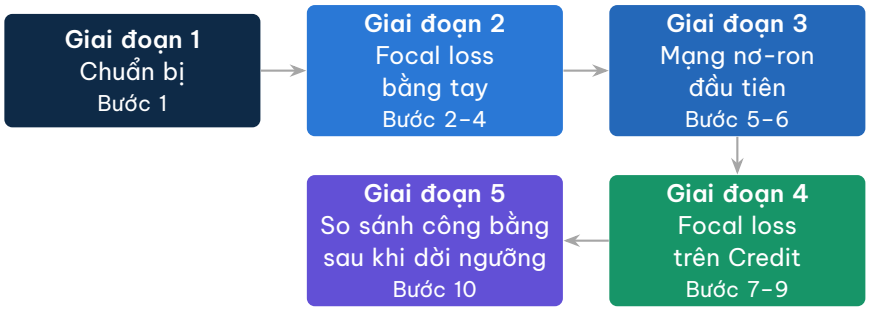
Bớt quan tâm đến những câu đã làm đúng

Focal loss nhân thêm một hệ số $(1 - p_t)^\gamma$ vào log loss, trong đó p_t là xác suất mô hình gán cho *nhãn đúng*:

- Mẫu đã được dự đoán đúng và tự tin (p_t gần 1): hệ số gần 0, nên gần như bị bỏ qua.

- Mẫu khó hoặc bị dự đoán sai (p_t nhỏ): hệ số gần 1, nên được giữ nguyên.

Lưu ý quan trọng: γ giảm trọng số theo **độ dễ**, không theo **lớp**. Một khách hàng vỡ nợ "dễ đoán" cũng bị giảm trọng số như một khách hàng tốt "dễ đoán".



Hình 6.1: Lộ trình của Chương 6: 5 giai đoạn, 10 bước.

6.2 Giai đoạn 1 – Chuẩn bị

6.2.1 Bước 1. Nạp dữ liệu và biến đổi đặc trưng

Mục đích: chuẩn bị dữ liệu cho mạng nơ-ron, loại mô hình nhạy với thang đo hơn nhiều so với mô hình cây.

Ô lệnh 1 · Nạp thư viện, dữ liệu và biến đổi đặc trưng cho mạng nơ-ron `code/ch06/s01_chuan_bi.py`

```
import sys
sys.path.append("code")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.preprocessing import QuantileTransformer
from common import credit_split

pd.set_option("display.width", 80)
pd.set_option("display.precision", 3)
SEED = 42
torch.set_num_threads(2)

X_train, X_test, y_train, y_test = credit_split()
qt = QuantileTransformer(output_distribution="normal", random_state=0).fit(X_train)
Xtr, Xte = qt.transform(X_train), qt.transform(X_test)      # mảng NumPy đã biến đổi
print("PyTorch:", torch.__version__, "| Xtr:", Xtr.shape, "| Xte:", Xte.shape)
print("BILL_AMT1 trước biến đổi: min =", X_train["BILL_AMT1"].min(), ", max =",
 ↪  X_train["BILL_AMT1"].max())
print("BILL_AMT1 sau biến đổi  : min = %.2f, max = %.2f" % (Xtr[:, 11].min(), Xtr[:,
 ↪  11].max()))
```

Kết quả

```
PyTorch: 2.14.1+cu130 | Xtr: (24000, 23) | Xte: (6000, 23)
BILL_AMT1 trước biến đổi: min = -165580.0 , max = 964511.0
BILL_AMT1 sau biến đổi  : min = -5.20, max = 5.20
```

Đọc kết quả. (Ô lệnh này có thể mất nửa phút ở lần đầu, do nạp thư viện PyTorch.)

Các biến tiền như `BILL_AMT1` có "đuôi rất dài" (Chương 1, Bước 11): giá trị từ −165 580 đến 964 511. Mạng nơ-ron học kém với dữ liệu như vậy. Ta dùng `QuantileTransformer` với tùy chọn `output_distribution="normal"` để "nắn" mỗi đặc trưng về dạng phân phối chuẩn: giá trị nhỏ nhất thành khoảng −5,2, lớn nhất thành khoảng +5,2, phần lớn khách hàng nằm quanh 0. Cách làm này bền hơn `StandardScaler` khi có nhiều giá trị cực lớn, vì nó dựa trên *thứ hạng* chứ không dựa trên trung bình và độ lệch chuẩn.

Lưu ý: `qt` chỉ được `fit` trên tập train, giống mọi bước tiền xử lý khác (Chương 1, Bước 21).

6.3 Giai đoạn 2 – Focal loss bằng tay

6.3.1 Bước 2. Tính tay cho 4 khách hàng

Mục đích: thấy hệ số $(1-p_t)^\gamma$ "tắt tiếng" các mẫu dễ như thế nào.

Với nhãn $y \in \{0,1\}$ và xác suất dự đoán p, định nghĩa

$$p_t = \begin{cases} p & \text{nếu } y = 1, \\ 1-p & \text{nếu } y = 0, \end{cases} \tag{6.1}$$

tức xác suất mô hình gán cho *nhãn đúng*. Log loss viết gọn là $\mathrm{CE} = -\log p_t$. Focal loss là

$$\mathrm{FL} = -(1-p_t)^\gamma \log p_t = (1-p_t)^\gamma \times \mathrm{CE}. \tag{6.2}$$

Ô lệnh 2 · Tính tay focal loss cho 4 khách hàng code/ch06/s02_tinh_tay.py

```
y = np.array([1, 1, 0, 0])
p = np.array([0.95, 0.30, 0.10, 0.70])          # xác suất vỡ nợ mô hình dự đoán
p_t = np.where(y == 1, p, 1 - p)                # xác suất gán cho NHÃN ĐÚNG
ce = -np.log(p_t)                               # cross-entropy (log loss)

tab = pd.DataFrame({"y": y, "p": p, "p_t": p_t, "CE": ce})
for g in [1, 2]:
    tab[f"(1-p_t)^{g}"] = (1 - p_t) ** g
    tab[f"FL g={g}"] = (1 - p_t) ** g * ce
print(tab.round(3).to_string())
```

Kết quả

```
   y     p   p_t     CE  (1-p_t)^1  FL g=1  (1-p_t)^2  FL g=2
0  1  0.95  0.95  0.051       0.05   0.003      0.003   0.000
1  1  0.30  0.30  1.204       0.70   0.843      0.490   0.590
2  0  0.10  0.90  0.105       0.10   0.011      0.010   0.001
3  0  0.70  0.30  1.204       0.70   0.843      0.490   0.590
```

Đọc kết quả.

- **Khách hàng 0** ($y = 1$, $p = 0{,}95$): đoán đúng và rất tự tin, $p_t = 0{,}95$. Log loss đã nhỏ (0,051); với $\gamma = 2$, hệ số $(1 - 0{,}95)^2 = 0{,}0025$ nên focal loss gần như bằng 0.
- **Khách hàng 2** ($y = 0$, $p = 0{,}1$): cũng là mẫu dễ, $p_t = 1 - 0{,}1 = 0{,}9$. Với $\gamma = 2$, hệ số $= 0{,}01$, focal loss $\approx 0{,}001$.
- **Khách hàng 1 và 3**: bị đoán sai, $p_t = 0{,}3$. Hệ số $(1 - 0{,}3)^2 = 0{,}49$, focal loss $= 0{,}49 \times 1{,}204 = 0{,}590$. Mẫu khó vẫn giữ được khoảng một nửa mất mát.

Tỷ lệ giữa mẫu khó và mẫu dễ thay đổi rất mạnh: với log loss, khách hàng 1 "nặng" gấp $1{,}204/0{,}105 \approx 11$ lần khách hàng 2; với focal loss $\gamma = 2$, con số này là $0{,}590/0{,}001$, tức gấp vài trăm lần.

Hãy để ý rằng khách hàng 1 (vỡ nợ) và khách hàng 3 (không vỡ nợ) có mất mát *bằng nhau*. Focal loss chỉ quan tâm mẫu khó hay dễ, *không quan tâm mẫu thuộc lớp nào*.

Tự kiểm tra

Câu 1. Một khách hàng không vỡ nợ ($y = 0$) được dự đoán $p = 0{,}5$. Tính p_t, log loss và focal loss với $\gamma = 2$. Hệ số giảm là bao nhiêu?

6.3.2 Bước 3. Nhìn hình dạng hàm mất mát

Mục đích: thấy ảnh hưởng của γ trên toàn bộ khoảng p_t.

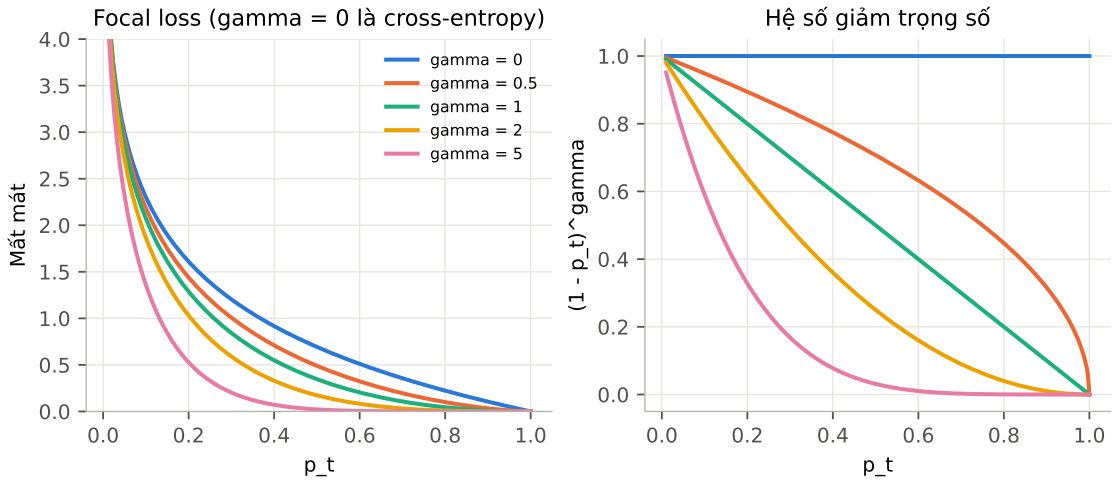
Ô lệnh 3 · Vẽ focal loss theo p_t với các giá trị gamma khác nhau code/ch06/s03_ve_duong_cong.py

```
pt = np.linspace(0.01, 1, 300)
fig, ax = plt.subplots(1, 2, figsize=(8.2, 3))
for g, c in zip([0, 0.5, 1, 2, 5], ["#2a78d6", "#eb6834", "#1baf7a", "#eda100",
↪  "#e87ba4"]):
    ax[0].plot(pt, -(1 - pt) ** g * np.log(pt), color=c, label=f"gamma = {g}")
    ax[1].plot(pt, (1 - pt) ** g, color=c)
ax[0].set_ylim(0, 4); ax[0].set_xlabel("p_t"); ax[0].set_ylabel("Mất mát");
↪  ax[0].legend(fontsize=7)
ax[0].set_title("Focal loss (gamma = 0 là cross-entropy)")
ax[1].set_xlabel("p_t"); ax[1].set_ylabel("(1 - p_t)^gamma"); ax[1].set_title("Hệ số
↪  giảm trọng số")
plt.savefig("figures/ch06_s03_curves.pdf", bbox_inches="tight")
```

Đọc kết quả. Các đường cong đều bắt đầu cao ở bên trái (mẫu bị đoán sai) và giảm về 0 ở bên phải (mẫu đoán đúng). γ càng lớn, đường càng "sụp" nhanh về 0 khi p_t tăng. Với $\gamma = 5$, chỉ những mẫu có p_t dưới khoảng 0,5 mới còn đóng góp đáng kể.

6.3.3 Bước 4. Viết focal loss bằng PyTorch

Mục đích: có hàm focal loss dùng được để huấn luyện mạng nơ-ron, và kiểm tra nó đúng.



Hình 6.2: Trái: focal loss theo p_t với các giá trị γ; đường $\gamma = 0$ chính là log loss. Phải: hệ số $(1 - p_t)^\gamma$. Với $\gamma = 2$, một mẫu có $p_t = 0{,}9$ chỉ còn 1% trọng số so với log loss.

Ô lệnh 4 · Viết focal loss bằng PyTorch và kiểm tra với trường hợp gamma = 0

code/ch06/s04_ham_focal.py

```
def focal_loss(logits, y, gamma=2.0, alpha=None):
    """FL = -alpha_t * (1 - p_t)^gamma * log(p_t), lấy trung bình trên các mẫu."""
    ce = F.binary_cross_entropy_with_logits(logits, y, reduction="none")   # =
    ↪ -log(p_t), ổn định
    p = torch.sigmoid(logits)
    p_t = p * y + (1 - p) * (1 - y)
    loss = (1 - p_t) ** gamma * ce
    if alpha is not None:                          # trọng số lớp: alpha cho lớp 1, 1 -
    ↪ alpha cho lớp 0
        loss = (alpha * y + (1 - alpha) * (1 - y)) * loss
    return loss.mean()

z = torch.tensor([2.0, -0.5, -2.0, 0.8])       # "logit": điểm số trước khi qua sigmoid
yt = torch.tensor([1.0, 1.0, 0.0, 0.0])
print("BCE của PyTorch       :", round(F.binary_cross_entropy_with_logits(z,
↪ yt).item(), 4))
print("focal_loss, gamma = 0 :", round(focal_loss(z, yt, gamma=0).item(), 4))
print("focal_loss, gamma = 2 :", round(focal_loss(z, yt, gamma=2).item(), 4))
```

Kết quả

```
BCE của PyTorch       : 0.5998
focal_loss, gamma = 0 : 0.5998
focal_loss, gamma = 2 : 0.2346
```

Đọc kết quả. Đọc hàm `focal_loss` từng dòng:

- Đầu vào là **logit** z, tức "điểm số" của mô hình trước khi qua hàm sigmoid. Xác suất là $p = \sigma(z) = 1/(1 + e^{-z})$ (Chương 1, Bước 15).
- Hàm `binary_cross_entropy_with_logits` với tùy chọn `reduction="none"` tính $-\log p_t$ cho *từng* mẫu. Hàm này ổn định về số học: tính trực tiếp `torch.log(torch.sigmoid(z))` có thể ra $-\infty$ khi z rất âm.
- Hai dòng tiếp theo tính p_t và nhân với hệ số $(1 - p_t)^\gamma$, đúng như (6.2).
- Nếu có α, mỗi mẫu được nhân thêm **trọng số lớp**: α cho lớp 1, $1 - \alpha$ cho lớp 0.

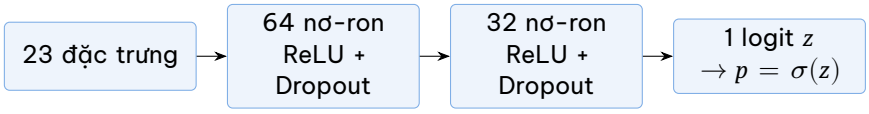
Kiểm tra: với $\gamma = 0$, hệ số $(1 - p_t)^0 = 1$, nên focal loss phải bằng đúng log loss. Kết quả: cả hai đều là 0,5998. Với $\gamma = 2$, mất mát giảm còn 0,2346 vì các mẫu dễ bị "tắt tiếng". Đây là thói quen tốt: *mỗi khi tự viết một hàm, hãy kiểm tra nó với một trường hợp đặc biệt đã biết đáp án.*

6.4 Giai đoạn 3 – Mạng nơ-ron đầu tiên

6.4.1 Bước 5. Xây và huấn luyện một MLP

Mục đích: có một mô hình huấn luyện bằng gradient, để có thể thay hàm mất mát tùy ý.

Mạng nơ-ron nhiều tầng (MLP, *Multi-Layer Perceptron*) có thể hiểu là nhiều lớp hồi quy logistic xếp chồng lên nhau. Mạng của ta có ba tầng:



Ô lệnh 5 · Mạng nơ-ron nhiều tầng (MLP) và hàm huấn luyện `code/ch06/s05_mang_no_ron.py`

```python
class MLP(nn.Module):
    def __init__(self, d_in):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d_in, 64), nn.ReLU(), nn.Dropout(0.1),
                                 nn.Linear(64, 32), nn.ReLU(), nn.Dropout(0.1),
                                 nn.Linear(32, 1))
    def forward(self, x):
        return self.net(x).squeeze(-1)          # trả về logit, mỗi mẫu một số

def train_mlp(X, y, loss_fn, epochs=25, batch=256, lr=1e-3, seed=SEED, verbose=False):
    torch.manual_seed(seed)
    Xt = torch.tensor(np.asarray(X), dtype=torch.float32)
    yt = torch.tensor(np.asarray(y), dtype=torch.float32)
    model = MLP(Xt.shape[1])
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
```

```
    for epoch in range(1, epochs + 1):
        model.train()
        perm = torch.randperm(len(Xt))          # xáo trộn dữ liệu mỗi epoch
        for i in range(0, len(Xt), batch):      # chia thành các lô (mini-batch) 256 mẫu
            idx = perm[i:i + batch]
            opt.zero_grad()
            loss = loss_fn(model(Xt[idx]), yt[idx])
            loss.backward()                     # tính đạo hàm
            opt.step()                          # cập nhật trọng số
        if verbose and epoch % 5 == 0:
            print(f"Epoch {epoch:2d}: mất mát của lô cuối = {loss.item():.4f}")
    model.eval()
    return model

@torch.no_grad()
def predict_proba(model, X):
    return torch.sigmoid(model(torch.tensor(np.asarray(X),
    ↪  dtype=torch.float32))).numpy()

bce = lambda z, y: F.binary_cross_entropy_with_logits(z, y)
mlp_bce = train_mlp(Xtr, y_train, bce, verbose=True)
```

Kết quả

```
Epoch  5: mất mát của lô cuối = 0.4521
Epoch 10: mất mát của lô cuối = 0.4304
Epoch 15: mất mát của lô cuối = 0.4110
Epoch 20: mất mát của lô cuối = 0.3907
Epoch 25: mất mát của lô cuối = 0.4536
```

Đọc kết quả. Ô lệnh dài nhất của chương; hãy đọc từng phần:

- **Lớp `MLP`**: `nn.Linear(23, 64)` là một tầng tuyến tính (như hồi quy, nhưng cho ra 64 con số thay vì 1). `nn.ReLU()` giữ số dương và đổi số âm thành 0, giúp mạng học được quan hệ phi tuyến. `nn.Dropout(0.1)` ngẫu nhiên "tắt" 10% nơ-ron mỗi lần học, giúp giảm quá khớp.
- **Hàm `train_mlp`**: mỗi **epoch** là một lượt đi qua toàn bộ dữ liệu. Dữ liệu được chia thành các **lô** (*mini-batch*) 256 mẫu. Với mỗi lô, máy thực hiện bốn bước:
  1. tính dự đoán `model(...)`;
  2. tính mất mát `loss_fn(...)`; đây là chỗ ta "cắm" focal loss vào;
  3. `loss.backward()` tính đạo hàm của mất mát theo mọi trọng số;
  4. `opt.step()` cập nhật trọng số theo hướng làm giảm mất mát (thuật toán Adam).
- **Hàm `predict_proba`**: đưa dữ liệu mới qua mạng để lấy logit, rồi dùng hàm `torch.sigmoid` để đổi logit thành xác suất. Dòng `@torch.no_grad()` phía trên hàm báo cho PyTorch không cần tính đạo hàm khi dự đoán, giúp chạy nhanh hơn.

Mất mát giảm dần qua các epoch (từ 0,45 xuống khoảng 0,39). Con số ở epoch 25 tăng lại là bình thường: đây chỉ là mất mát của *một lô* cuối cùng, nên dao động theo những khách hàng tình cờ rơi vào lô đó.

6.4.2 Bước 6. Đánh giá MLP

Mục đích: có mốc so sánh cho các hàm mất mát khác, và so MLP với XGBoost.

Ô lệnh 6 · Đánh giá MLP (hàm mất mát BCE) trên tập test code/ch06/s06_danh_gia.py

```
from sklearn.metrics import (roc_auc_score, average_precision_score, f1_score,
                             recall_score, precision_score, brier_score_loss)

def report(p, y_true=y_test, t=0.5):
    pred = (p >= t).astype(int)
    return {"ROC-AUC": roc_auc_score(y_true, p), "PR-AUC":
    ↪  average_precision_score(y_true, p),
            "F1": f1_score(y_true, pred), "Recall": recall_score(y_true, pred),
            "Precision": precision_score(y_true, pred), "Brier":
            ↪  brier_score_loss(y_true, p),
            "p TB": p.mean()}

p_bce = predict_proba(mlp_bce, Xte)
print(pd.Series(report(p_bce)).round(3).to_string())
```

Kết quả

```
ROC-AUC      0.770
PR-AUC       0.544
F1           0.471
Recall       0.370
Precision    0.649
Brier        0.137
p TB         0.219
```

Đọc kết quả. Ngoài các chỉ số quen thuộc, ô lệnh in thêm hai đại lượng về *chất lượng xác suất*:

- **Brier score** $= \frac{1}{n}\sum_i (\hat{p}_i - y_i)^2$: trung bình bình phương sai lệch giữa xác suất và nhãn thật. Càng nhỏ càng tốt; 0,137 là mức khá tốt với bài toán này.
- **p TB**: xác suất dự đoán trung bình, 0,219, rất gần tỷ lệ vỡ nợ thật 0,221. MLP với BCE được hiệu chuẩn tốt.

So với XGBoost ở Chương 1 (ROC-AUC 0,778, PR-AUC 0,556), MLP hơi kém hơn (ROC-AUC 0,770, PR-AUC 0,544). Điều này phù hợp với nhận định chung: trên dữ liệu bảng, mô hình cây thường mạnh hơn mạng nơ-ron [8]. Trong chương này, ta dùng MLP chỉ vì focal loss cần một mô hình huấn luyện bằng gradient.

6.5 Giai đoạn 4 – Focal loss trên Credit Default

6.5.1 Bước 7. Focal loss kéo xác suất về phía 0,5

Mục đích: dự đoán trước, bằng toán, focal loss sẽ làm gì với xác suất.

Log loss có một tính chất đẹp: nếu xác suất thật là p, mô hình tốt nhất sẽ dự đoán *đúng* p. Focal loss có giữ tính chất này không? Ta tìm bằng số: với mỗi p, thử mọi q từ 0 đến 1 và chọn

q làm mất mát kỳ vọng nhỏ nhất:

$$q^\star(p) = \text{giá trị } q \text{ làm nhỏ nhất } \Big[-p\,(1-q)^\gamma \log q \;-\; (1-p)\,q^\gamma \log(1-q)\Big]. \tag{6.3}$$

Ô lệnh 7 · Nếu xác suất thật là p, focal loss "muốn" dự đoán bao nhiêu? code/ch06/s07_keo_ve_05.py

```
q = np.linspace(1e-4, 1 - 1e-4, 200001)
rows = {}
for g in [0, 1, 2, 5]:
    rows[f"gamma = {g}"] = [q[np.argmin(-p_ * (1 - q) ** g * np.log(q) - (1 - p_) * q
    ↪  ** g * np.log(1 - q))]
                            for p_ in [0.05, 0.2, 0.4, 0.6, 0.9]]
print(pd.DataFrame(rows, index=["p = 0,05", "p = 0,2", "p = 0,4", "p = 0,6", "p =
↪  0,9"]).T.round(3))
```

Kết quả

```
           p = 0,05  p = 0,2  p = 0,4  p = 0,6  p = 0,9
gamma = 0     0.050    0.200    0.400    0.600    0.900
gamma = 1     0.162    0.314    0.443    0.557    0.774
gamma = 2     0.250    0.373    0.462    0.538    0.694
gamma = 5     0.371    0.438    0.482    0.518    0.597
```

Đọc kết quả. Mỗi ô là xác suất mà focal loss "muốn" dự đoán khi xác suất thật là p:

- Với $\gamma = 0$ (log loss), $q^\star = p$ đúng như mong đợi.
- Với $\gamma = 2$: xác suất thật 0,05 bị đẩy lên 0,25; xác suất thật 0,9 bị kéo xuống 0,69. **Mọi xác suất bị kéo về phía** 0,5: mô hình trở nên kém tự tin một cách có hệ thống.
- Nhưng **phía của** 0,5 **được giữ nguyên**: $q^\star > 0{,}5$ khi và chỉ khi $p > 0{,}5$.

Tính chất thứ hai cho một dự đoán bất ngờ: γ **sẽ không làm tăng Recall tại ngưỡng** 0,5, vì những khách hàng đang ở dưới 0,5 vẫn ở dưới 0,5. Kết quả này phù hợp với phân tích lý thuyết của Charoenphakdee et al. [3]. Bước 8 sẽ kiểm chứng.

6.5.2 Bước 8. So sánh tám hàm mất mát

Mục đích: kiểm chứng dự đoán của Bước 7, và tách riêng tác dụng của γ với α.

Ô lệnh 8 · So sánh 8 hàm mất mát bằng đánh giá chéo 5-fold (khoảng 2–3 phút)

code/ch06/s08_cv.py

```
from sklearn.model_selection import StratifiedKFold

w_bal = (y_train == 0).sum() / (y_train == 1).sum()
losses = {
    "BCE": bce,
    "BCE có trọng số": lambda z, y: F.binary_cross_entropy_with_logits(z, y,
    ↪  pos_weight=torch.tensor(w_bal)),
    "FL gamma=0.5": lambda z, y: focal_loss(z, y, 0.5),
    "FL gamma=1": lambda z, y: focal_loss(z, y, 1.0),
    "FL gamma=2": lambda z, y: focal_loss(z, y, 2.0),
    "FL gamma=5": lambda z, y: focal_loss(z, y, 5.0),
```

```
    "FL gamma=2, alpha=0.25": lambda z, y: focal_loss(z, y, 2.0, 0.25),
    "FL gamma=2, alpha=0.75": lambda z, y: focal_loss(z, y, 2.0, 0.75),
}
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
cv_rows = {}
for name, loss_fn in losses.items():
    folds = []
    for tr, va in skf.split(X_train, y_train):
        qt_f = QuantileTransformer(output_distribution="normal",
        ↪  random_state=0).fit(X_train.iloc[tr])
        m = train_mlp(qt_f.transform(X_train.iloc[tr]), y_train.iloc[tr], loss_fn)
        folds.append(report(predict_proba(m, qt_f.transform(X_train.iloc[va])),
        ↪  y_train.iloc[va]))
    cv_rows[name] = pd.DataFrame(folds).mean()
cv_df = pd.DataFrame(cv_rows).T[["PR-AUC", "F1", "Recall", "Precision", "Brier", "p
↪  TB"]]
print(cv_df)
```

Kết quả

	PR-AUC	F1	Recall	Precision	Brier	p TB
BCE	0.535	0.487	0.390	0.651	0.136	0.218
BCE có trọng số	0.535	0.533	0.637	0.459	0.181	0.422
FL gamma=0.5	0.536	0.487	0.389	0.651	0.139	0.264
FL gamma=1	0.535	0.487	0.388	0.655	0.147	0.304
FL gamma=2	0.533	0.485	0.386	0.653	0.165	0.361
FL gamma=5	0.533	0.485	0.386	0.651	0.201	0.431
FL gamma=2, alpha=0.25	0.538	0.079	0.042	0.735	0.150	0.279
FL gamma=2, alpha=0.75	0.534	0.537	0.599	0.488	0.200	0.455

Đọc kết quả. (Ô lệnh này huấn luyện $8 \times 5 = 40$ mạng nơ-ron, mất khoảng 2–3 phút.) Trong mỗi fold, `QuantileTransformer` được `fit` lại chỉ trên phần học, để tránh rò rỉ. Đọc bảng theo từng nhóm:

Tham số γ không thay đổi quyết định. So các dòng "FL gamma=..." (không có α) với dòng BCE:

- F1 gần như không đổi: 0,487, 0,487, 0,487, 0,485, 0,485.
- Recall dao động trong khoảng 0,386–0,390; PR-AUC trong khoảng 0,533–0,536.

Đúng như Bước 7 dự đoán. Xác suất trung bình tăng dần từ 0,218 lên 0,431, không phải vì mô hình dự đoán nhiều người vỡ nợ hơn, mà vì đa số khách hàng có xác suất thấp bị "kéo lên" về phía 0,5. Do xác suất bị méo, Brier score xấu đi rõ rệt: từ 0,136 lên 0,201.

Tham số α mới là thứ thay đổi quyết định.

- $\alpha = 0{,}75$ (ưu tiên lớp vỡ nợ): F1 $= 0{,}537$, Recall $= 0{,}599$. Kết quả này gần với BCE có trọng số (F1 $= 0{,}533$), vì α *chính là* một trọng số lớp như ở Chương 5.
- $\alpha = 0{,}25$, giá trị mặc định trong bài báo gốc: **Recall sụp xuống** 0,042, F1 chỉ còn 0,079. Lý do: $\alpha = 0{,}25$ cho lớp vỡ nợ trọng số 0,25 và lớp kia 0,75, tức *giảm* tầm quan trọng của lớp thiểu số xuống còn 1/3!

Tự kiểm tra

Câu 2. Với $\alpha = 0{,}75$, mỗi khách hàng vỡ nợ có trọng số gấp bao nhiêu lần một khách hàng không vỡ nợ? So sánh với trọng số $w = n_0/n_1 = 3{,}52$ ở Chương 5.

Vì sao bài báo gốc dùng $\alpha = 0{,}25$?

Trong bài toán phát hiện vật thể, γ đã "tắt tiếng" rất mạnh hàng chục nghìn mẫu âm dễ. Lúc đó lớp dương trở nên "quá nặng", và Lin et al. [16] cần $\alpha < 0{,}5$ để kéo lại cân bằng. Bê nguyên giá trị đó sang dữ liệu bảng là một sai lầm nghiêm trọng, và rất hay gặp.

6.5.3 Bước 9. Nhìn phân bố xác suất

Mục đích: thấy bằng mắt focal loss "nén" xác suất về 0,5.

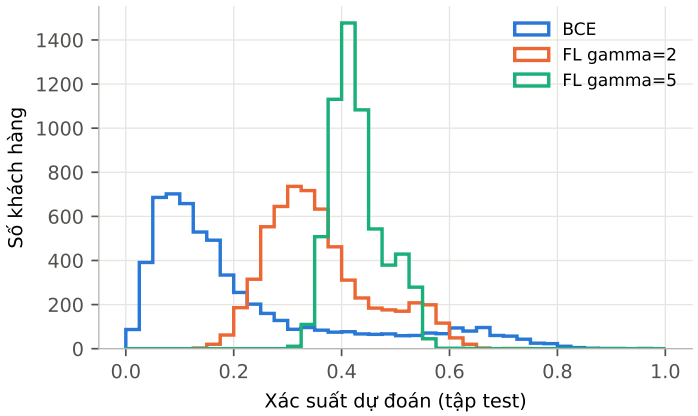
Ô lệnh 9 · Phân bố xác suất dự đoán trên tập test `code/ch06/s09_phan_bo_xac_suat.py`

```
probs = {"BCE": p_bce}
for name in ["FL gamma=2", "FL gamma=5"]:
    probs[name] = predict_proba(train_mlp(Xtr, y_train, losses[name]), Xte)

fig, ax = plt.subplots(figsize=(5, 2.9))
bins = np.linspace(0, 1, 41)
for (name, p_), c in zip(probs.items(), ["#2a78d6", "#eb6834", "#1baf7a"]):
    ax.hist(p_, bins=bins, histtype="step", lw=1.6, color=c, label=name)
ax.set_xlabel("Xác suất dự đoán (tập test)"); ax.set_ylabel("Số khách hàng");
↪  ax.legend()
plt.savefig("figures/ch06_s09_proba.pdf", bbox_inches="tight")
print({k: round(float(v.mean()), 3) for k, v in probs.items()})
```

Kết quả

```
{'BCE': 0.219, 'FL gamma=2': 0.361, 'FL gamma=5': 0.43}
```



Hình 6.3: Phân bố xác suất dự đoán trên tập test của ba hàm mất mát.

Đọc kết quả. Với BCE, xác suất trải rộng từ 0 đến khoảng 0,8, phần lớn khách hàng ở vùng 0,05–0,2. Với focal loss $\gamma = 2$, đám đông dịch sang vùng 0,25–0,35. Với $\gamma = 5$, gần như mọi khách hàng bị dồn vào khoảng hẹp quanh 0,4. Tuy vậy, *thứ tự* của các khách hàng gần như

không đổi, nên PR-AUC không đổi.

Vì sao focal loss hiệu quả với ảnh nhưng không với dữ liệu bảng?

Trong phát hiện vật thể, tỷ lệ mẫu âm trên mẫu dương có thể lên tới $1000:1$, và phần lớn mẫu âm có $p_t > 0{,}99$. Hệ số $(1-p_t)^2 < 10^{-4}$ loại bỏ chúng rất hiệu quả. Với Credit Default, tỷ lệ chỉ là $3{,}5:1$, và Chương 3 cho thấy 61% khách hàng vỡ nợ nằm ở vùng chồng lấn. Ở đây không có "biển mẫu dễ" nào để loại bỏ. Vấn đề của dữ liệu này là *chồng lấn*, không phải *mẫu dễ lấn át*.

6.6 Giai đoạn 5 – So sánh công bằng sau khi dời ngưỡng

6.6.1 Bước 10. Chọn ngưỡng riêng cho từng hàm mất mát

Mục đích: trả lời câu hỏi cuối cùng: nếu mỗi hàm mất mát được dùng với *ngưỡng tốt nhất của nó*, chúng còn khác nhau không?

So sánh tại ngưỡng 0,5 chưa thật công bằng: focal loss và trọng số làm méo thang xác suất theo những cách khác nhau. Ta tách 20% tập train làm **tập kiểm định** (*validation set*), tìm ngưỡng cho F1 cao nhất trên tập này cho từng mô hình, rồi mới chấm điểm trên tập test.

Ô lệnh 10 · Công bằng hơn: chọn ngưỡng tốt nhất cho TỪNG hàm mất mát rồi mới so sánh

code/ch06/s10_doi_nguong.py

```
from sklearn.model_selection import train_test_split

X_a, X_v, y_a, y_v = train_test_split(X_train, y_train, test_size=0.2,
 ↪  stratify=y_train,
                                      random_state=SEED)       # tách 20% train làm tập
                                       ↪  kiểm định
qt_a = QuantileTransformer(output_distribution="normal", random_state=0).fit(X_a)
grid = np.round(np.arange(0.05, 0.951, 0.01), 2)
rows = {}
for name in ["BCE", "BCE có trọng số", "FL gamma=2", "FL gamma=2, alpha=0.75"]:
    m = train_mlp(qt_a.transform(X_a), y_a, losses[name])
    p_v = predict_proba(m, qt_a.transform(X_v))
    t_best = grid[np.argmax([f1_score(y_v, (p_v ⩾ t).astype(int)) for t in grid])]
    p_te = predict_proba(m, qt_a.transform(X_test))
    pred = (p_te ⩾ t_best).astype(int)
    rows[name] = {"Ngưỡng t*": t_best, "F1 test": f1_score(y_test, pred),
                  "Recall": recall_score(y_test, pred), "PR-AUC":
                   ↪  average_precision_score(y_test, p_te)}
print(pd.DataFrame(rows).T.round(3))
```

Kết quả

	Ngưỡng t*	F1 test	Recall	PR-AUC
BCE	0.23	0.527	0.575	0.537
BCE có trọng số	0.53	0.525	0.560	0.526
FL gamma=2	0.40	0.525	0.562	0.533
FL gamma=2, alpha=0.75	0.51	0.531	0.552	0.533

Đọc kết quả.

- **Ngưỡng tốt nhất rất khác nhau**: 0,23 cho BCE, 0,40 cho focal loss $\gamma = 2$, khoảng 0,5 cho hai mô hình có trọng số lớp. Mỗi hàm mất mát tạo ra một "thang xác suất" riêng.
- **Nhưng sau khi dời ngưỡng, F1 gần như bằng nhau**: từ 0,525 đến 0,531. Mọi khác biệt ở Bước 8 gần như biến mất.
- PR-AUC của BCE (0,537) vẫn bằng hoặc cao hơn các hàm mất mát khác.

Thông điệp chung của toàn bộ Phần II

Từ Chương 2 đến chương này, ta đã thử lấy mẫu (SMOTE, Borderline-SMOTE, ADASYN), trọng số lớp, và focal loss. Trên dữ liệu Credit Default, chúng đều có *tác dụng chính* giống nhau: **dịch chuyển sự đánh đổi giữa Precision và Recall**, gần như tương đương với việc dời ngưỡng quyết định. Không kỹ thuật nào làm mô hình *phân biệt* hai lớp tốt hơn (PR-AUC không tăng).
Vì vậy, trước khi dùng một kỹ thuật "xử lý mất cân bằng" phức tạp, hãy thử giải pháp đơn giản nhất: **giữ mô hình gốc, chọn ngưỡng theo chi phí hoặc theo tập kiểm định**. Muốn PR-AUC tăng thật sự, ta cần thông tin mới: đặc trưng tốt hơn (Phần III) hoặc mô hình tốt hơn (Phần IV, V).

Khi nào nên dùng focal loss cho dữ liệu bảng?

- Khi mất cân bằng rất nặng (dưới 1%) *và* có rất nhiều mẫu âm dễ, ví dụ phát hiện gian lận giao dịch.
- Khi mạng nơ-ron sâu bị *quá tự tin*: Mukhoti et al. [17] cho thấy focal loss giúp hiệu chuẩn tốt hơn trong trường hợp này. MLP của ta vốn đã hiệu chuẩn tốt với BCE (Brier 0,136), nên focal loss chỉ làm xấu đi.

Thử tự làm

Sửa Ô lệnh 10 để thêm focal loss $\gamma = 5$. Ngưỡng tốt nhất của nó cao hay thấp hơn 0,40? Vì sao? (Gợi ý: xem xác suất trung bình ở Bước 8.)

6.7 Tổng kết

Ưu điểm	Hạn chế
Rất hiệu quả khi có lượng lớn mẫu âm dễ (phát hiện vật thể, gian lận cực hiếm)	Kéo mọi xác suất về 0,5, làm hỏng hiệu chuẩn
Cài đặt đơn giản, vài dòng code	Hai tham số γ, α tác động theo những cách rất khác nhau, dễ nhầm
Có thể giảm hiện tượng quá tự tin của mạng sâu	Không dùng trực tiếp được cho mô hình cây (cần tự viết đạo hàm)
	Trên Credit Default, không tốt hơn BCE sau khi dời ngưỡng

Ba điều cần nhớ nhất

1. γ giảm trọng số mẫu **dễ**; α là trọng số **lớp**. Chỉ α thay đổi quyết định tại ngưỡng 0,5.
2. Focal loss với $\gamma > 0$ kéo xác suất về 0,5; giá trị $\alpha = 0{,}25$ mặc định *giảm* tầm quan trọng của lớp thiểu số.
3. Sau khi chọn ngưỡng phù hợp, BCE, BCE có trọng số và focal loss cho kết quả gần như nhau trên dữ liệu bảng.

6.8 Những lỗi thường gặp

Lỗi thường gặp

1. **Dùng $\alpha = 0{,}25$ mặc định** mà không hiểu rằng nó giảm trọng số lớp thiểu số (Bước 8).
2. **Kết luận "focal loss tốt hơn BCE"** trong khi thực ra chỉ α (tức trọng số lớp) tạo ra khác biệt. Cần thí nghiệm tách riêng γ và α.
3. **So sánh các hàm mất mát tại cùng ngưỡng 0,5.** Mỗi hàm mất mát tạo thang xác suất khác nhau; hãy so sánh sau khi chọn ngưỡng (Bước 10) và bằng PR-AUC.
4. **Dùng xác suất của mô hình focal loss** như xác suất thật khi chưa hiệu chuẩn lại.
5. **So sánh MLP + focal loss với XGBoost + không gì cả** rồi kết luận về focal loss. Hãy so sánh trong cùng một loại mô hình.
6. **Tự tính log của sigmoid** bằng hai bước riêng thay vì dùng hàm có sẵn trong PyTorch (Bước 4), gây lỗi số học khi z rất âm.

6.9 Khi viết bài báo khoa học

Gợi ý cho bài báo

- **Thí nghiệm tách biến (ablation) là bắt buộc**: BCE, BCE + α, FL(γ), FL(γ) + α. Nếu chỉ báo cáo "FL($\gamma = 2, \alpha = 0{,}25$)" so với BCE, người đọc không biết thành phần nào gây ra khác biệt.
- **So sánh sau khi chọn ngưỡng** trên tập kiểm định, như Bước 10, để loại bỏ ảnh hưởng của việc "méo" thang xác suất.
- **Báo cáo Brier score** hoặc một chỉ số hiệu chuẩn khác, vì focal loss thay đổi độ hiệu chuẩn.
- **Mẫu câu**: *"To disentangle the focusing effect from class re-weighting, we evaluated focal loss with $\gamma \in \{0.5, 1, 2, 5\}$ both with and without the balancing factor α; decision thresholds were tuned on a held-out validation split."*

6.10 Bài tập

Mức 1 – Làm quen.

1. Tính tay focal loss với $\gamma = 1$ và $\gamma = 5$ cho bốn khách hàng ở Bước 2.

2. Huấn luyện MLP với 50 epoch thay vì 25. PR-AUC trên tập test thay đổi thế nào?

Mức 2 – Vận dụng.

3. Giải số (6.3) với $\gamma = 2$ trên lưới p từ 0,01 đến 0,99, rồi vẽ $q^\star(p)$. Dùng đường cong này để "khử" xác suất của mô hình focal loss về thang thật, rồi tính lại Brier score.

4. Lặp lại Bước 10 với tất cả 8 hàm mất mát của Bước 8. Có hàm mất mát nào vượt hẳn BCE sau khi dời ngưỡng không?

Mức 3 – Nâng cao.

5. Tạo phiên bản Credit Default có tỷ lệ vỡ nợ chỉ 1% (bỏ bớt người vỡ nợ *trong tập train*), rồi lặp lại Bước 8. Lợi ích của γ có xuất hiện không?

6. Viết focal loss làm hàm mục tiêu tùy chỉnh cho LightGBM: cần đạo hàm bậc một và bậc hai theo logit. Gợi ý: dùng `torch.autograd` để kiểm tra công thức đạo hàm bạn tự tính.

6.11 Đáp án phần Tự kiểm tra

Câu 1.
$p_t = 1 - 0{,}5 = 0{,}5$. Log loss $= -\log 0{,}5 \approx 0{,}693$. Hệ số $(1-0{,}5)^2 = 0{,}25$, nên focal loss $= 0{,}25 \times 0{,}693 \approx 0{,}173$. Ngay cả một mẫu "nửa đúng nửa sai" cũng bị giảm 75% mất mát; chỉ những mẫu bị đoán sai rõ rệt (p_t nhỏ) mới giữ được phần lớn mất mát.

Câu 2.
Lớp vỡ nợ nhận $\alpha = 0{,}75$, lớp kia nhận $1 - \alpha = 0{,}25$, nên tỷ lệ là $0{,}75/0{,}25 = 3$ lần. Con số này gần với $w = 3{,}52$, giải thích vì sao FL($\gamma = 2$, $\alpha = 0{,}75$) và BCE có trọng số cho F1 và Recall gần nhau ở Bước 8. Ngược lại, $\alpha = 0{,}25$ cho tỷ lệ $1/3$: lớp vỡ nợ bị *giảm* trọng số.

Thuật ngữ của chương

Tiếng Việt	Tiếng Anh	Nghĩa ngắn gọn
Mạng nơ-ron nhiều tầng	multi-layer perceptron (MLP)	Nhiều tầng tuyến tính xen kẽ hàm kích hoạt
Logit	logit	Điểm số trước khi qua hàm sigmoid
Lượt huấn luyện	epoch	Một lần đi qua toàn bộ dữ liệu train
Lô	mini-batch	Một nhóm nhỏ mẫu dùng cho một lần cập nhật
Tham số tập trung	focusing parameter (γ)	Mức giảm trọng số mẫu dễ
Tập kiểm định	validation set	Phần dữ liệu tách từ train để chọn ngưỡng/tham số
Brier score	Brier score	Trung bình bình phương sai lệch giữa xác suất và nhãn
Thí nghiệm tách biến	ablation study	Bỏ/thêm từng thành phần để đo tác dụng riêng

Tài liệu đọc thêm

Bài báo gốc của Lin et al. [16]. Phân tích lý thuyết về ước lượng xác suất với focal loss của Charoenphakdee et al. [3]. Focal loss và hiệu chuẩn mạng nơ-ron trong Mukhoti et al. [17]. Vì sao mô hình cây vẫn mạnh trên dữ liệu bảng: Grinsztajn et al. [8].

Tài liệu tham khảo

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.

[3] Nontawat Charoenphakdee, Jayakorn Vongkulbhisal, Nuttapong Chairatanakul, and Masashi Sugiyama. On focal loss for class-posterior probability estimation: A theoretical perspective. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5202–5211, 2021.

[4] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16: 321–357, 2002.

[5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[6] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[7] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, pages 973–978, 2001.

[8] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*, 35:507–520, 2022.

[9] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. In *Advances in Intelligent Computing (ICIC 2005)*, volume 3644 of *Lecture Notes in Computer Science*, pages 878–887. Springer, 2005.

[10] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.

[11] Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks*, pages 1322–1328, 2008.

[12] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012.

[13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.

[14] Gary King and Langche Zeng. Logistic regression in rare events data. *Political Analysis*, 9 (2):137–163, 2001.

[15] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.

[16] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.

[17] Jishnu Mukhoti, Viveka Kulharia, Amartya Sanyal, Stuart Golodetz, Philip H. S. Torr, and Puneet K. Dokania. Calibrating deep neural networks using focal loss. In *Advances in Neural Information Processing Systems 33*, pages 15288–15299, 2020.

[18] Claude Nadeau and Yoshua Bengio. Inference for the generalization error. *Machine Learning*, 52(3):239–281, 2003.

[19] R. Kelley Pace and Ronald Barry. Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3):291–297, 1997.

[20] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3): e0118432, 2015.

[21] Ruben van den Goorbergh, Maarten van Smeden, Dirk Timmerman, and Ben Van Calster. The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, 29(9):1525–1534, 2022.

[22] I-Cheng Yeh and Che-hui Lien. The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2):2473–2480, 2009.